COMPARATIVE ANALYSIS OF PATHFINDING FOR STATIC ROUTE PRE-PLANNING

A Thesis Presented to the Faculty, College of Computer Studies
Laguna State Polytechnic University
Los Baños Campus
Los Baños, Laguna

In partial fulfillment of the requirement for the degree
Bachelor of Science in Computer Science
Specialized in Intelligent Systems

MELVIN JAMES R. CALAMBA
LEX ANNE M. DE LEON
NICOLE M. SURALTA

GENE MARCK B. CATEDRILLA

JUNE 2026

APPROVAL SHEET

The thesis entitled **"COMPARATIVE ANALYSIS OF PATHFINDING FOR STATIC ROUTE PRE-PLANNING"** prepared and submitted by Melvin James R. Calamba, Lex Anne M. De Leon, and Nicole M. Suralta in partial fulfillment of the requirements for the degree, Bachelor of Science in Computer Science specialized in Intelligent Systems is at this moment recommended for approval and acceptance.

GENE MARCK B. CATEDRILLA
Adviser

Approved and Accepted by the Committee on Oral Examination with a grade of____.

MIKE PHILIP T. RAMOS
Member

CAROLINA R. JOVAL
Member

JOCELYN O. PADALLAN
Member

JEFFERSON L. LERIOS
Member

Accepted in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science specialized in Intelligent Systems.

LOYD S. ECHALAR, DIT
Associate Dean, College of Computer Studies

OFELIA B. MANINGAS, EdD.
Chairperson, Research and Development Office

_____
Date Signed

RESEARCH CONTRIBUTION NO.	

ACKNOWLEDGEMENT

The researchers extend their heartfelt gratitude to all who supported and guided them throughout this study. They express their deepest thanks to Almighty God for His constant guidance and provision.

The researchers also express their heartfelt appreciation to the University President, Dr. Mario R. Briones, and the OIC Campus Director, Engr. Manuel Luis R. Alvarez, for their exceptional leadership and for fostering an environment that encourages academic excellence.

Their gratitude is also extended to the Associate Dean of the College of Computer Studies, Dr. Loyd S. Echalar, for his valuable advice and continuous encouragement.

The researchers are especially thankful to their mentor, Mr. Gene Marck B. Catedrilla, for his unwavering support, insightful advice, and expert guidance throughout the development of this thesis.

Finally, the researchers would like to express their heartfelt appreciation to the validators who took the time to assess, review, and provide valuable feedback on the system and research findings. Their expertise, constructive comments, and thoughtful suggestions greatly contributed to the improvement and credibility of this study.

THE RESEARCHERS

DEDICATION

This project is dedicated to Almighty God, whose guidance, wisdom, and protection have been a continuous source of strength and inspiration throughout this journey.

Mr. Joel Calamba & Mrs. Imelda Calamba

Mr. Sydney De Leon & Mrs. Jelna De Leon

Mr. Michael Suralta & Mrs. Ailyn Suralta

For their constant love, unwavering support, and sacrifices—both emotional and financial—which helped the researchers stay focused and complete this work. Their belief in the researchers was essential to the success of this study.

The researchers also extend their heartfelt gratitude to their friends and siblings for their steadfast support and motivation during challenging moments, reminding them to remain focused and persevere despite the obstacles encountered along the way.

M.J.R.C

L.A.M.D.L.

N.M.S.

ABSTRACT

In emergency situations, every second counts, and determining the fastest and most reliable route is crucial to saving lives and minimizing damage. This study compares four pathfinding algorithms such as A*, Dijkstra's Algorithm, Ant Colony Optimization (ACO), and Greedy Best-First Search (GBFS) to identify the most suitable approach for emergency response routing. A localized simulation of Los Baños, Laguna was developed to model real-world conditions with dynamic disruptions such as road blockages, floods, and earthquakes. The algorithms were evaluated based on path cost, computation time, and adaptability to changing environments. Statistical validation using the Paired Samples T-Test revealed no significant differences ($p > 0.05$) in performance across most conditions, except for execution time under earthquake scenarios ($p < 0.001$), indicating sensitivity to severe disruptions. Dijkstra's Algorithm consistently achieved the most optimal paths (average cost = 297.17; execution time = 0.0008s), followed by A*, which offered comparable accuracy with faster computation. ACO demonstrated adaptability but required longer processing (0.2774s), while GBFS was the fastest (0.0101s) but least accurate. Overall, Dijkstra's Algorithm proved to be the most reliable and efficient for time-critical routing, contributing to the design of intelligent, adaptive routing systems for real-time emergency response.

Keywords: *Pathfinding Algorithms, Dijkstra's Algorithm, A* Search, Ant Colony Optimization, Greedy Best-First Search, Emergency Response, Route Planning*

TABLE OF CONTENTS

Preliminaries	***Page***
Title Page	i
Approval Sheet	ii
Acknowledgement	iii
Dedication	iv
Abstract	v
Table of Contents	vi
List of Figures	viii
List of Tables	x
Definition of Terms	xii

CHAPTER

I	**INTRODUCTION**	
	Background of the Study	1
	Objectives	4
	Significance of the Study	5
	Scope and Limitations	6
II	**REVIEW OF RELATED LITERATURE AND STUDIES**	
	Related Literature	8
	Literature Matrix	30
	Synthesis and Relevance of the Study	33
	Conceptual Framework	34
	Summary of Algorithms	35
III	**METHODOLOGY**	
	Research Design	37
	Materials	38
	Libraries	39
	Hardware	40
	Model Development	41
	Algorithm Analysis	42
	Simulation Design	44
	Flowchart	45
	Simulation Parameters	46
	Data Acquisition Implementation	47
	Algorithm Implementation	47
	Theorems	48
	Pseudocode	49
	Model Analysis and Evaluation	51
	Validation Method	54

Ethical Consideration 55

IV RESULTS AND DISCUSSIONS

Develop Model 56
Model Simulation Setup 63
Dataset Visualization 65
Model Experimental Runs 66
Summary Performance of Algorithms 73
Model Simulation Results 78
Trade-offs in Algorithm Performance 82
Comparative Analysis 83
Overall Comparison of Algorithms 85
Paired T-Test 86
Discussion 90
System Limitations and Constraints 91

V SUMMARY, CONCLUSIONS AND RECOMMENDATIONS

Summary 95
Conclusion 96
Recommendations 98

REFERENCES
APPENDICES

Appendix A. Letter of Approval
Appendix B. Certificate of Acceptance
Appendix C. Relevant Source Code
Appendix D. Evaluation Tool
Appendix E. Sample Input / Output
Appendix F. Curriculum Vitae
Appendix G. User’s Guide
Appendix H. Grammarly and Turnitin Report
Appendix I. Publishable Paper (IEEE format)

LIST OF FIGURES

Figure	Title	Page
1	Conceptual Framework	35
2	Flow of A* Algorithm	42
3	Flow of Ant Colony Optimization	43
4	Flow Dijkstra's Algorithm	43
5	Flow of Greedy Best-First Search	44
6	Flowchart	46
7	Pathfinding Algorithm Race System	57
8	Pathfinding Algorithm Race System w/o Road Blockages	58
9	Pathfinding Algorithm Race System w/ Simulate Flood	59
10	Pathfinding Algorithm Race System w/ Simulate Fire	60
11	Pathfinding Algorithm Race System w/ Simulate Earthquake	60
12	Earthquake Simulation Interface Showing Algorithm Failure under Severe Network Disruption	61
13	ACO Results under Earthquake Scenario	62
14	Los Banos Laguna Dataset	63
15	Dataset Visualization of Los Baños Road Network	65
16	Distribution of Road Segment Segment Distance and Travel Duration	66
17	Algorithm Performance across Experimental Runs Under No Road Blockages	68
18	Comparative Performance of Pathfinding Algorithm	69
19	Comparative Average Results of Pathfinding Algorithm Under Dynamic Road Blockage	71

20	Cost and Execution Time Trends of Pathfinding Algorithms Under Dynamic Road Blockages	72
21	Average Performance across All Conditions	85

LIST OF TABLES

No.	Title	Page
1	Matrix of the Review of Related Literature	30
2	Short Summary of Algorithms	36
3	Laptop Specification	41
4	Performance Metrics Table	53
5	Performance of Algorithms on Experimental Runs	67
6	Performance of Algorithms on Experimental Runs with blockages	70
7	Summary Performance of Algorithms under Normal Conditions	73
8	Summary Performance of Algorithms under Dynamic Conditions	74
9	Summary Performance of Algorithms under Earthquake Disaster	75
10	Summary Performance of Algorithms under Fire Disaster	76
11	Summary Performance of Algorithms under Flood Disaster	77
12	Key Results Summary (under Normal Conditions)	79
13	Key Results Summary (under Road Blockages)	79
14	Trade-offs in Algorithm Performance	82
15	Summary of Comparative Findings	84
16	Paired Sample T-test Results (Normal Vs. Fire)	86
17	Paired Sample T-test Results (Normal Vs. Dynamic)	87
18	Paired Sample T-test Results (Normal Vs. Earthquake)	88

19 Paired Samples T-test Results (Normal Vs. Flood) 89

DEFINITION OF TERMS

A*	A heuristic-based pathfinding algorithm that combines the benefits of Dijkstra's algorithm and Best-First Search by evaluating paths with a cost function $f(n)=g(n)+h(n)$, where g is the cost so far and h is a heuristic estimate to the goal.
Ant Colony Optimization (ACO)	A metaheuristic inspired by the foraging behavior of ants, using simulated pheromones to probabilistically find optimal or near-optimal paths through a graph.
Discrete Event Simulation (DES)	A modeling approach where system behavior is represented as a sequence of discrete events occurring at specific points in simulated time; used here to test how algorithms respond to sudden changes like roadblocks.
Dijkstra's Algorithm	A classic pathfinding algorithm that systematically explores all nodes to guarantee the shortest path in a weighted graph with non-negative edge costs.
Emergency	A critical and unexpected situation that demands urgent intervention to protect human life, property, or the environment, including incidents such as disasters, accidents, and other life-threatening events.
Global Positioning System (GPS)	A satellite-based navigation system providing real-time location information to guide routing and tracking of vehicles or individuals.
Greedy Best-First Search (GBFS)	A heuristic-driven search algorithm that prioritizes exploration of the node closest to the goal according to the heuristic value, often leading to faster but not always optimal paths.
Parameter	A configurable variable or setting that influences how an algorithm behaves,

	such as heuristic weights, pheromone evaporation rates, or speed limits.
Pathfinding Algorithms	Computational methods designed to find routes between points on a map or graph, optimizing criteria such as shortest distance, fastest time, or least cost.
Road Layout	The arrangement or map of roads, intersections, and pathways within a given area, defining possible routes for vehicles or pedestrians.

CHAPTER 1

INTRODUCTION

Background of the Study

Emergencies come in many forms, ranging from natural disasters like earthquakes, hurricanes, and floods to man-made crises such as accidents, medical emergencies, and terrorist attacks. Regardless of the type, all emergencies share one defining feature: unpredictability and urgency. When an emergency strikes, every second counts, and even minor delays in response can have devastating consequences. Research highlights that efficient and timely response systems are essential for minimizing the loss of life and property damage. According to Zhou et al. (2018), the ability of first responders to act quickly and effectively depends significantly on the efficiency of routing systems, the accessibility of real-time information, and coordination among multiple agencies. Time remains the most critical factor, the faster and more efficiently responders act, the more lives can be saved.

Emergency situations also present a wide range of challenges that complicate response efforts. Studies have provided an analysis of the challenges that arise during emergency situations, emphasizing their dynamic and unpredictable nature (Shahrah & Al-Mashari, 2017). Roads can be blocked by debris, traffic congestion can cause gridlock, and sudden weather changes may render planned routes ineffective. Traditional response strategies, which rely on pre-established routes and manual decision-making, often fail to adapt to these rapid changes.

Over the years, several technological solutions have been developed to assist in emergency operations, particularly in optimizing the movement of responders and resources. Among these, Geographic Information Systems (GIS) have emerged as one of the most valuable tools. GIS enables emergency planners to map road networks, analyze traffic, and allocate resources more efficiently in real time (Matveev et al., 2018). Although GIS improves route selection using real-time data on road blockages and traffic, it remains limited by its reliance on static algorithms that cannot quickly adapt to sudden environmental changes. As a result, many GIS-based systems are still reactive rather than proactive, adjusting routes only after disruptions occur.

Pathfinding algorithms have long been studied in computer science to solve routing problems in fields such as robotics, logistics, and gaming. These algorithms use heuristics and optimization models to identify the shortest or most cost-effective route between two points (Kyrkou et al., 2022). For emergency operations, these algorithms can help responders adjust routes in real time, improving efficiency and reducing response delays. Several well-known algorithms such as A*, Dijkstra's, Greedy Best-First Search (GBFS), and Ant Colony Optimization (ACO) have shown potential for such applications. The A* algorithm efficiently balances speed and accuracy through heuristic estimation (Foead et al., 2021). Dijkstra's Algorithm guarantees optimality but becomes computationally demanding in large, dynamic networks. GBFS offers faster route computation but often sacrifices path accuracy, while ACO, inspired by ant foraging behavior, demonstrates adaptability and learning capability in changing environments (Konatowski & Pawłowski, 2018).

Integrating these algorithms into modern response systems can greatly reduce travel time and enhance operational efficiency. Responders can ensure that vehicles are dispatched efficiently to priority areas by dynamically adjusting routes based on real-time conditions such as traffic, road closures, and hazards. The ability to process and react to data instantaneously is critical in high-stakes environments, as static algorithms often fail to adapt when road conditions change within seconds (Rachmawati & Gustin, 2020).

However, few studies have conducted direct comparative evaluations of multiple pathfinding algorithms under simulated, real-time emergency conditions. Most prior research focuses on static or theoretical applications rather than dynamic, event-driven simulations (Zhu et al., 2020; Kyrkou et al., 2022). This lack of comparative experimentation under realistic conditions leaves a gap in understanding how algorithms perform when exposed to real-time disruptions and environmental changes..

Unlike previous comparative studies that focused on static or single-scenario evaluations, this research introduces multi-scenario, event-driven simulation that dynamically models real-world emergencies such as floods, fires, and earthquakes. Through the use of Discrete Event Simulation, the system replicates time-dependent disruptions like road blockages and changing accessibility, allowing for more realistic and comprehensive assessment of algorithm performance. This novel approach enables the study to capture not only route efficiency but also each algorithm’s adaptability and resilience under rapidly changing conditions like factors often overlooked in traditional pathfinding research.

The reviewed literature reveals a strong focus on individual path-finding algorithms and their applications in routing, navigation, and crisis management contexts. Studies discuss how heuristics like A* Search, Dijkstra's Algorithm, and Greedy Best-First Search are widely used due to their efficiency in traversing graphs and generating routes for robots, ambulances, and unmanned systems. Meanwhile, bio-inspired techniques such as Ant Colony Optimization are gaining attention for their adaptability in dynamic and uncertain environments. The literature also highlights the growing importance of integrating real-time data including traffic conditions, blocked roads, and environmental updates into routing systems (Zhang et al., 2021). However, few studies have conducted direct comparative evaluations of multiple path-finding algorithms under simulated, real-time emergency conditions, where variables such as road blockages, network disruptions, and evolving hazards are systematically introduced.

Objectives of the Study

The primary objective of this study is to conduct a comprehensive comparative analysis of various path-finding algorithms used in emergency response system. Specially, the study aims to:

1. Identify and utilize appropriate datasets that represent static road network characteristics for route-planning, including road layout, connectivity, and predefined constraints relevant to emergency scenarios.
2. Implement and adapt several path finding algorithms such as A*, Ant Colony Optimization, Dijkstra's, and Greedy Best-First Search to,

evaluating their performance in scenarios involving obstacle avoidance, blocked paths, and time-sensitive navigation.

3. Evaluate and compare the performance of A*, Ant Colony Optimization, Dijkstra' algorithm, and Greedy Best-First Search using measurable metrics such as computation time, path optimality, success rate, adaptability to changes, and resistance to obstacles, along with evaluation tools like accuracy, to identify which algorithm performs best in time-critical emergency situations.

4. Validate a statistical performance differences among pathfinding algorithms through paired-sample t-test analysis.

Significance of the Study

This study is significant in improving path accuracy, route efficiency, and algorithmic reliability for static route pre-planning in emergency and critical service operations. Effective pre-planned routing supports emergency services such as ambulances, fire trucks, and rescue units by ensuring that optimal routes are identified in advance, which can reduce response delays and improve preparedness. This research addresses gaps in understanding how different pathfinding approaches perform under predefined road constraints and simulated disruptions by comparing the performance of A*, Dijkstra’s Algorithm, Ant Colony Optimization, and Greedy Best-First Search.

From an institutional perspective, the findings can assist local government units, disaster risk reduction and management offices (DRRMOs), and urban planners in selecting appropriate routing algorithms for emergency

preparedness, infrastructure planning, and contingency route design. The results may support better logistical coordination, improved road network planning, and the development of resilient transportation strategies for disaster-prone areas.

From a technical and academic standpoint, this study contributes to the field of algorithm analysis and intelligent transportation systems by providing a structured comparison of classical, heuristic, and metaheuristic pathfinding algorithms under controlled conditions. The findings can serve as a reference for researchers and system developers in designing efficient pre-routing models and can inform future studies that integrate dynamic data, real-time decision support, and smart city transportation frameworks.

Scope and Limitation

This study conducts a comparative analysis of four pathfinding algorithms for static route pre-planning using a localized road network dataset of Los Baños, Laguna, representing intersections and road segments. The simulation focuses on predefined network conditions and fixed constraints to evaluate each algorithm’s path efficiency, computation time, and consistency in identifying optimal routes. Multiple experimental runs were performed to ensure result reliability, with performance assessed using metrics such as path cost, execution time, and path length.

The study is limited to static urban and suburban road networks and does not incorporate real-time traffic conditions, environmental sensors, or human decision-making factors. Dynamic events such as live road closures,

traffic fluctuations, and emergency responder behavior were excluded to maintain algorithmic isolation and controlled comparison. Future studies may extend this work by integrating dynamic data sources, real-time updates, larger-scale networks, and multi-objective optimization to improve applicability to real-world emergency routing and smart transportation systems.

CHAPTER 2

THEORETICAL FRAMEWORK OF THE STUDY

Review Related Literature and Studies

Bio-inspired Algorithms

Ant Colony Optimization (ACO) is a bio-inspired algorithm that simulates the behavior of ants in finding the shortest path between their colony and food sources. It was first introduced as a probabilistic method that uses pheromone trails to guide artificial ants toward optimal solutions (Dorigo & Gambardella, 1997). The study was originally applied to the Traveling Salesman Problem and laid the foundation for applying ACO to real-world routing issues, including dynamic environments such as emergency scenarios. A hybrid algorithm based on Ant Colony Optimization, enhanced with a threshold-accepting technique, was later proposed to address emergency roadway repair routing problems (Yan & Shih, 2012). This study demonstrated that ACO, particularly when hybridized, can adapt to dynamic and uncertain conditions such as those encountered after the 1999 Chi-Chi earthquake in Taiwan. The ACO approach proved effective in managing complex networks and finding near-optimal paths while considering multiple constraints such as road damage severity and repair resource availability (Yan & Shih, 2012).

It has been shown to be flexible and efficient even in situations with complex variable spaces by extending its principles to continuous optimization domains (Socha & Dorigo, 2008). Their research highlights that ACO is a suitable option for emergency routing in scenarios where conditions change

rapidly, due to its ability to navigate unpredictable and dynamic environments. The ACO paradigm has also been explored in the context of combinatorial optimization problems, introducing the ant metaheuristic and demonstrating its effectiveness in solving complex tasks such as quadratic assignment and frequency assignment problems (Maniezzo et al., 2003). These studies provide valuable insights into the versatility and practical applications of ACO across various optimization contexts.

ACO has matured into a powerful tool for solving complex optimization problems in fields such as routing, scheduling, and network design (Akhtar, 2019). Recent studies emphasize ACO's superiority in navigating high-dimensional search spaces, often outperforming traditional methods due to its adaptive and parallelizable nature (Blum, 2024). Hybrid approaches have also been developed to integrate machine learning, enhancing convergence speed and solution quality (Wang & Han, 2021). Furthermore, research illustrates ACO's practical utility in engineering design, underscoring its flexibility, scalability, and robustness in dynamic environments (Sun et al., 2022).

Pathfinding using Ant Colony Algorithm

Ant Colony Optimization has also been utilized to develop real-time evacuation routing systems designed to function in post-earthquake scenarios (Mochio & Kitahara, 2016). The researchers' approach addressed the challenge of recalculating optimal paths in environments where traditional routes were often compromised due to infrastructure damage such as building collapses, fires, and blocked roads. The hybrid ACO model allowed for quick adaptation to environmental changes, making it a suitable solution for real-time

decision-making in post-disaster conditions. The study also highlighted ACO's ability to handle uncertainty and dynamic changes, which are inherent disaster environments where information is incomplete and constantly evolving (Mochio & Kitahara, 2016). Effective routing for emergency scenarios such as evacuation and emergency vehicle navigation is essential for minimizing damage and saving lives. Pathfinding algorithms play a crucial role in ensuring that the quickest and safest routes are selected in highly dynamic and uncertain environments, such as those caused by natural disasters or accidents (Zhao & Cheng, 2015).

It is frequently applied to pathfinding problems due to its ability to discover optimal or near-optimal paths in dynamic environments. Studies have explored grid-based pathfinding in simulated environments, where ACO has proven efficient in reducing path length and computation time (Eido & Ibrahim, 2025). Early research introduced a global path-planning framework based on ACO, which laid the groundwork for later improvements (Wen & Cai, 2006). More recent studies demonstrate ACO's adaptability to real-time terrain changes in grid-based navigation (Biswas et al., 2023), while others have optimized ACO for urban navigation, achieving reduced computational overhead (Ronghua et al., 2024).

Heuristic Algorithms

A* Algorithm

On the other hand, the A* algorithm is a widely used heuristic-based search method known for its optimality and efficiency in static, well-defined

environments. It guarantees the shortest path if the heuristic used is admissible (Hart et al., 1968). However, its performance can degrade in highly dynamic scenarios unless extended through variants such as D* or real-time replanning methods (Koenig & Likhachev, 2002). In the context of emergency response, where road conditions and accessibility can rapidly change, A* may require frequent recalculations, potentially affecting overall response time. The D* (Dynamic A*) algorithm was proposed as an improvement to A* to address this limitation, allowing real-time replanning and making it more suitable for unpredictable and dynamic environments (Stentz, 1994).

Foundational insights into its mechanisms are discussed in early studies (Lester, 2005), while later enhancements demonstrate improved performance in large-scale systems (Foead et al., 2021). Research has also provided a structured classification of A* variants, highlighting its adaptability across different applications (Rios & Chaimowicz, 2010). More recent innovations include the integration of A* into hardware-limited systems (You et al., 2023) and the use of machine learning to refine heuristic functions, showcasing A*'s evolving role in intelligent and adaptive pathfinding systems (Yan, 2023). Routing for emergency response is a critical component of disaster management, as rapid access to affected areas can save lives and minimize damage. Various algorithms have been developed to optimize route planning under such high-pressure scenarios. Among these, Ant Colony Optimization (ACO) and the A* algorithm are two of the most widely studied techniques, each offering distinct strengths (Dorigo & Stützle, 2004; Hart et al., 1968). The effectiveness of each algorithm depends largely on the characteristics of the

disaster environment. ACO demonstrates strong adaptability and robustness in handling uncertainty, making it well-suited for routing in post-disaster areas where real-time updates and probabilistic decision-making are essential (Dorigo & Gambardella, 1997). In contrast, A* performs more efficiently in stable, well-defined environments where deterministic planning is sufficient and computational speed is prioritized (Hart et al., 1968; Koenig & Likhachev, 2002).

Pathfinding using A* Algorithm

In his book Planning Algorithms, LaValle provides a comprehensive analysis of path planning strategies, examining both probabilistic and deterministic methods such as A* and ACO (LaValle, 2006). The discussion highlights the advantages and limitations of these algorithms across various contexts, offering a strong theoretical foundation for selecting the most appropriate approach based on operational requirements and environmental conditions.

A* remains a core algorithm in modern navigation systems, particularly in robotics and gaming applications. Its effectiveness in real-time decision-making has been demonstrated in various domains. For instance, the algorithm has been applied in real-time strategy games to simulate intelligent NPC movement (Cui & Shi, 2011). The utility of A* in dynamic terrains and uncertain environments has also been examined in several studies, including recent research hosted by Universitas Gadjah Mada (Radianto, 2024) and application involving biomedical navigation and robotic systems (Zhao et al., 2024). This balance is especially relevant in emergency response scenarios, where timely and reliable navigation is critical.

Furthermore, comprehensive reviews, such as one published in the International Journal of Computer Science and Mobile Computing, illustrate various implementation strategies of the A* algorithm across different environments (Sharma, 2015). Other studies available in biomedical research archives further extend its applicability to non-traditional pathfinding contexts, including healthcare logistics and medical transport systems (Lermen & Reinert, 2000).

Dijkstra's Algorithm

Dijkstra's Algorithm, introduced in 1956, remains a foundational technique for finding the shortest paths in graphs with non-negative edge weights. It has been widely applied in transportation and communication networks because of its deterministic accuracy (Parungao et al., 2018). Its efficiency compared to other classical algorithms has also been emphasized in recent research (Khan, 2023). To enhance scalability, several studies have proposed memory-efficient variants (Javaid, 2013) and heuristic-enhanced modifications suitable for large graph scenarios (Bing & Lai, 2022). Additionally, educational implementations have demonstrated the algorithm's pedagogical value in promoting computational thinking and algorithmic understanding (Cevizci, 2018).

In real-time routing applications, efficiency can be improved by integrating Dijkstra's Algorithm with other network optimization techniques (Rexin & Porter, 2020). This integration allows the algorithm to consider multiple routing factors such as traffic congestion, variable loads, and alternative paths like key elements in large-scale network systems. Researchers have also

provided valuable insights into enhancing its performance for modern communication infrastructures, particularly within Internet of Things (IoT) and telecommunication networks (Narnavre et al., 2024).

Several studies have examined the limitations of Dijkstra's Algorithm, particularly its challenges with memory utilization and fixed parameters that constrain efficiency (Parmanand et al., 2012). Their research further proposes enhancements to improve the algorithm's effectiveness in large-scale data applications, emphasizing the need for faster computations and reduced memory usage.

Pathfinding with Dijkstra's Algorithm

Beyond its theoretical significance, Dijkstra's Algorithm has been extensively applied in real-world pathfinding scenarios, particularly for optimizing route selection. In terrain navigation systems for robotic agents, the algorithm has been integrated to determine the most efficient paths in complex and dynamic environments (Wang et al., 2022). This application demonstrates Dijkstra's continued relevance in modern autonomous systems that require precise and adaptive routing.

In the field of transportation, Dijkstra's Algorithm has been proven effective in optimizing road traffic routing (Fan & Shi, 2010). Their study focused on minimizing congestion and reducing travel times by applying the algorithm to traffic flow analysis. Through this approach, Dijkstra's Algorithm successfully identified the most efficient routes within road networks, helping to alleviate delays in urban environments. Further research explored its integration into

intelligent traffic management systems, emphasizing how the algorithm can dynamically adjust travel routes based on real-time traffic conditions to enhance travel efficiency (Chen et al., 2014).

Dijkstra's Algorithm has also proven valuable in large-scale simulation environments. Its performance has been evaluated within smart city simulations focused on vehicle routing in interconnected transport grids (Rosita et al., 2019). The study demonstrated how Dijkstra's Algorithm effectively optimized vehicle flow across urban road networks by accounting for factors such as road closures and congestion. In a related application, the algorithm was utilized in campus navigation systems, where it was adapted to support hybrid indoor and outdoor routing for improved mobility within university environments (Suardinata et al., 2022).

Greedy Best-First Search Algorithm

Greedy Best-First Search (GBFS) employs heuristic evaluation to prioritize exploration toward the goal, often favoring speed over guaranteed optimality. Its role in accelerating state-space traversal has been highlighted in recent studies (Setyobudhi, 2022). Further analyses emphasize GBFS's sensitivity to heuristic selection and domain characteristics, establishing a foundation for future algorithmic improvements (Heusner et al., 2019). Performance benchmarks also demonstrate the algorithm's competitiveness in planning and navigation tasks, particularly when guided by well-designed heuristic functions (Xie et al., 2014). However, despite its computational efficiency, GBFS may generate suboptimal routes in complex or highly constrained environments, limiting its reliability for time-critical applications.

The sample complexity of learning heuristic functions for GBFS and A* has been explored in recent research (Sakaue & Oki, 2022). Their study provides upper and lower bounds on the pseudo-dimensions of these algorithms, demonstrating that heuristic quality has a significant impact on search efficiency. The findings emphasize that while both algorithms benefit from well-designed heuristics, GBFS performance can decline substantially when suboptimal heuristics are applied.

Recent research has explored how sample generation strategies can enhance the performance of GBFS when guided by learned heuristic functions (Bettker et al., 2024). The study identifies key factors such as sample distribution and cost-to-goal estimation that significantly influence the algorithm's efficiency. The authors demonstrate how these elements affect the overall effectiveness of the search process by analyzing various sample generation methods. Their findings indicate that optimizing sample generation can substantially improve GBFS performance.

Pathfinding with Greedy Best-First Search

Greedy Best-First Search has been extensively utilized in pathfinding applications across various domains, including robotics, video games, and navigation systems. Its efficiency in optimizing unmanned aerial vehicle (UAV) trajectories within dynamic environments has been demonstrated, achieving faster computation times compared to exhaustive search methods (Jesuthas & Somaskandan, 2022). In computer gaming, GBFS is recognized as a lightweight yet effective algorithm for achieving real-time performance (Mehta et al., 2015). Similarly, recent studies highlight its success in guiding

autonomous agents through grid-based maps, where it consistently outperformed random walk and breadth-first search methods under heuristic-driven conditions (Lawande et al., 2022).

Recent research presented at the NeurIPS conference has introduced machine learning–enhanced search algorithms that integrate GBFS with learned heuristics to improve navigation efficiency in high-dimensional state spaces (Wayahdi et al., 2021). Comparative evaluations with A* and Dijkstra's Algorithm reaffirms GBFS's advantage in computational speed, though this benefit often comes at the expense of optimality, depending on the precision of the heuristic function (Sakaue & Oki, 2022).

Applications of Ant Colony Optimization on Medical Emergency

The Vehicle Routing Problem with Time Windows (VRPTW) plays a crucial role in emergency scenarios where timely responses are essential. An Improved Ant Colony Optimization (IACO) algorithm was introduced to address this problem, aiming to minimize total vehicle mileage while considering capacity and customer time constraints (Ibrahim et al., 2022). The proposed approach incorporated local search and mutation processes to enhance solution quality, demonstrating effectiveness in generating routes that comply with strict time windows without exceeding vehicle limitations. In the field of emergency logistics, efficient relief supply distribution is equally vital. An enhanced version of the ACO algorithm, termed Fish-Swarm Ant Colony Optimization, was proposed to improve emergency logistics routing (Ding, 2011). Simulation results revealed that this method outperformed the standard

ACO, producing higher-quality solutions that reduced delivery times while accounting for road conditions and varying urgency levels.

In life-critical situations, optimizing routes for emergency services is vital to reducing response times and saving lives. An enhanced ACO algorithm was developed to improve emergency vehicle routing under uncertain traffic conditions, resulting in faster arrival times (Morin et al., 2023). Similarly, the integration of ACO with GIS for real-time emergency dispatching has been explored, highlighting its potential for dynamic route recalculation (Singh et al., 2017). Recent advancements include a framework designed to prioritize ambulance pathfinding during peak hours, adapting ACO specifically for medical emergency systems (Wang, 2025). Furthermore, a multi-objective ACO model has been proposed to balance time efficiency and safety in route selection, an approach particularly valuable in dense urban environments (Graf et al., 2017).

Applications of Ant Colony Optimization on Fire and Evacuation Emergency

Ant Colony Optimization has shown significant potential in navigating complex and hazardous environments, particularly during fire emergencies. Its integration with fire simulation models has been demonstrated to enhance multiscale feature fusion and high-resolution representation work to enhance human pose estimation systems performance in evacuation route planning and safety assessments (Goodwin et al., 2015). More recent advancements have introduced an improved ACO-based framework for guiding fire rescue robots, enabling them to navigate unpredictable and dynamic fire environments

through adaptive pathfinding strategies (Zhang et al., 2024). These studies highlight ACO's strength in responding to rapidly changing and high-risk scenarios by enhancing real-time decision-making and route optimization in emergency conditions.

Building on previous research, sensor data has been integrated with ACO to guide evacuees through real-time hazard maps, enhancing situational awareness and decision-making during fire evacuations (Li et al., 2024). This approach demonstrated how ACO can effectively utilize real-time environmental data to improve evacuation efficiency and safety. In related work, adaptive path strategies were explored for indoor fire scenarios, showing how ACO dynamically adjusts to shifting fire locations or blocked pathways (Xu et al., 2022). Additionally, ACO has been applied to optimize fire station response times, improving the routing of fire trucks and overall emergency intervention efficiency (Zarrinpanjeh et al., 2020). Collectively, these studies underscore ACO's versatility and effectiveness in enhancing operational decision-making during fire-related emergencies.

Building on previous research, sensor data has been integrated with Ant Colony Optimization to guide evacuees through real-time hazard maps, enhancing situational awareness and decision-making during fire evacuations (Li et al., 2024). This approach demonstrated how ACO can effectively utilize real-time environmental data to improve evacuation efficiency and safety. In related work, adaptive path strategies were explored for indoor fire scenarios, showing how ACO dynamically adjusts to shifting fire locations or blocked pathways (Xu et al., 2022). Additionally, ACO has been applied to optimize fire

station response times, improving the routing of fire trucks and overall emergency intervention efficiency (Zarrinpanjeh et al., 2020). Collectively, these studies underscore ACO's versatility and effectiveness in enhancing operational decision-making during fire-related emergencies.

Expanding the applicability of Ant Colony Optimization (ACO) in emergency management, recent studies have demonstrated its effectiveness across diverse disaster scenarios. ACO has been utilized to model seismic evacuations with real-time constraints, optimizing routes under time pressure and dynamic obstacles (Muhammad & Wu, 2023). Similarly, its use in flood evacuation modeling has shown promise in simulating large-scale population movements and identifying safe, efficient escape routes (Yamamoto & Li, 2018). Earlier research also applied ACO to urban evacuation planning, emphasizing its scalability and adaptability to complex environments with high population density, heavy traffic, and infrastructural challenges (Forcael et al., 2014). Collectively, these studies highlight ACO's robustness in managing real-time evacuation planning for multi-hazard urban scenarios.

Applications of A* Algorithm on Medical Emergency

Recent advancements have adapted the A* algorithm for urban rescue operations, enabling responders to determine the fastest routes even when infrastructure is partially damaged. Simulations demonstrated that an enhanced version of A*, which integrates dynamic road data and prioritizes high-risk zones, significantly outperforms traditional static routing techniques (Liu et al., 2023). The A* algorithm continues to be widely employed in emergency pathfinding because of its efficiency in identifying optimal routes within dynamic

and constraint-heavy environments. In disaster scenarios characterized by blocked roads and unpredictable travel conditions, A* remains highly effective in recalculating optimal paths in real time (Hart et al., 1968).

In emergency medical operations, pathfinding algorithms such as A* play a crucial role in minimizing response times and improving service efficiency. One study demonstrated the application of A* in healthcare logistics, optimizing ambulance routing within complex urban environments (Nordin et al., 2011). Subsequent research expanded on this work by introducing real-time route adjustment mechanisms for use during large-scale health crises (Nordin et al., 2012). More recent advancements have integrated the medical Internet of Things (IoT) with A*, enabling intelligent decision-making and dynamic coordination in time-sensitive scenarios (Akhther et al., 2023). Additionally, A* has been applied in drone-assisted medical delivery and mobile emergency response systems, improving the speed and reliability of critical resource distribution (Michael & Xavier, 2018; Liu et al., 2024).

Applications of A* Algorithm on Fire Emergency

A recent case study integrated fire recognition technology with the A* algorithm for firefighting robots, demonstrating its capability to dynamically adjust paths based on real-time fire data (Deng et al., 2022). Building on this, newer studies have incorporated machine vision and Simultaneous Localization and Mapping (SLAM) to enhance autonomous navigation in smoke-filled or low-visibility environments. These approaches highlight the potential of combining A* with perception-based technologies to improve

navigation and decision-making in extreme conditions where traditional pathfinding algorithms may struggle (Li, 2024).

Further research has demonstrated how the A* algorithm can be adapted to dynamically adjust paths in real time by incorporating fire sensor data (Zhang & Yu, 2022). In the context of embedded systems for firefighting applications, lightweight A*-based approaches have also been developed to enhance performance on resource-constrained platforms. Such methods emphasize the need for algorithmic efficiency in real-time fire response, where both computational power and reaction speed are critical (Win, 2022).

Applications of A* Algorithm on Evacuation Emergency

Evacuation strategies during natural disasters such as floods and earthquakes demand rapid adaptation to constantly changing conditions and constraints. Within this context, the A* algorithm has been increasingly utilized to optimize evacuation routes and improve decision-making efficiency. For instance, its integration with real-time geospatial data has been applied to simulate flood evacuations, allowing for dynamic route adjustments based on environmental updates (Choe et al., 2023). Similarly, an earthquake drill simulation conducted at UGM demonstrated the use of A* for route optimization under conditions of restricted mobility and infrastructure damage (Radianto, 2024). These studies collectively highlight A*'s adaptability and effectiveness in facilitating real-time evacuation planning during large-scale disasters.

Further research has explored the use of the A* algorithm for multi-path evacuation planning in wildfire scenarios, demonstrating its capacity to adapt

evacuation routes in real time based on changing wind patterns and fire spread dynamics (Lee et al., 2022). In large-scale urban evacuations, A* has also been applied to optimize pedestrian movements in densely populated areas during earthquakes, improving overall evacuation efficiency (Zhang et al., 2021). Additionally, recent work combining A* drone technology has introduced innovative solutions for aerial evacuation during flood events. Drones equipped with real-time sensors and A*-based pathfinding were shown to identify alternative routes when ground access was obstructed by floodwaters, emphasizing the algorithm's versatility in integrating with emerging technologies for disaster response (Kim & Hwang, 2023).

Applications of Dijkstra's Algorithm on Medical Emergency

Dijkstra's Algorithm plays a major role in optimizing routes for emergency response, particularly in ensuring rapid access to medical facilities and the efficient delivery of medical supplies. Its effectiveness has been demonstrated in drone-assisted emergency response systems, where it enables autonomous path determination for medical deliveries in remote or inaccessible areas (Husain et al., 2022). This approach allows drones to navigate dynamic environments efficiently, ensuring the swift transport of life-saving equipment. Similarly, Dijkstra's Algorithm has been implemented in mobile emergency response systems to improve ambulance routing and enhance response times through continuous route adjustments (Ghazali et al., 2023). Further work has extended its application to emergency medical models that consider road types and congestion levels, optimizing route selection to minimize delays and prioritize the fastest paths for critical interventions (Kai et al., 2014).

Furthermore, Dijkstra's Algorithm has been effectively utilized within hospital information systems to streamline internal logistics and emergency workflows. It has been applied to optimize patient transfer routes and resource allocation, thereby enhancing the speed and coordination of in-hospital operations during emergencies (Eneh & Arinze, 2017). The algorithm also supports efficient management of patient movement, medical staff deployment, and the delivery of essential supplies, reducing delays in critical procedures. In a related study, Dijkstra's Algorithm was integrated into an IoT-enabled real-time tracking system for medical supply distribution, where it determined the most efficient routes for time-sensitive deliveries such as blood, organs, and vaccines (Ahmed et al., 2018). This integration demonstrated its reliability and precision in ensuring prompt transport even under dynamic or resource-constrained conditions.

Applications of Dijkstra's Algorithm on Fire Emergency

Dijkstra's Algorithm has also been widely applied in fire evacuation and response systems, where dynamic adaptability is critical. In one study, it was used to continuously update evacuation routes based on real-time data, accounting for changing conditions such as fire spread and blocked pathways (Mirahadi & McCabe, 2021). Building on this, an enhanced version of Dijkstra's Algorithm incorporated building-specific hazards into path cost calculations, improving accuracy in complex fire scenarios such as high-rise structures and large public facilities (Win, 2022). Similarly, the integration of Dijkstra's pathfinding into smart firefighting support systems has been explored to optimize the movement of fire trucks and emergency responders within fire-

affected zones (Rahayuda & Santiari, 2019). These studies collectively demonstrate Dijkstra's adaptability and reliability in real-time fire and emergency management contexts.

A decision support system for fire services was developed using Dijkstra's Algorithm to optimize routing for fire trucks, ambulances, and other emergency responders (Phyo & Sein, 2015). The model enabled fire departments to make more informed decisions under time constraints by accounting for road conditions, traffic, and other dynamic variables to minimize response times. This integration significantly improved the efficiency of emergency operations. In a related study, Dijkstra's Algorithm was applied in virtual fire safety training environments to enhance simulation realism and decision-making practice. The system calculated optimal evacuation routes during fire drills, allowing trainees to experience realistic, time-critical scenarios (Rahayuda & Santiari, 2021).

Applications of Dijkstra's Algorithm on Evacuation Emergency

Dijkstra's Algorithm has proven to be an effective tool for identifying efficient and safe evacuation routes in disaster scenarios such as floods and earthquakes. In a study on flood-prone areas, a GIS-based evacuation system integrated Dijkstra's Algorithm to utilize real-time data on topography and flood forecasts, enabling dynamic route adjustments and significantly reducing evacuation times by directing evacuees through safer, less affected areas (Mantoro et al., 2021). Similarly, a multi-hazard evacuation model incorporating Dijkstra's Algorithm was developed to simulate pedestrian movement during earthquakes, taking into account obstacles such as debris, building collapse,

and aftershocks. This model effectively calculated optimal evacuation paths in real time, improving both safety and efficiency in navigating complex post-disaster environments (Wang et al., 2025).

Dijkstra's Algorithm has also been extended to aerial rescue and logistics operations in disaster zones. In such scenarios, ground transportation is often restricted or obstructed, making aerial solutions crucial. Researchers demonstrated how integrating Dijkstra's Algorithm into drone-assisted emergency response systems enables efficient optimization of evacuation and delivery routes, ensuring timely assistance to affected areas (Husain et al., 2022).

Applications of Greedy Best-First Search on Medical Emergency

A study focusing on optimizing ambulance dispatch systems under time-critical conditions demonstrated the effectiveness of GBFS in rapidly identifying optimal routes using real-time data. The findings revealed that GBFS, when guided by accurate distance heuristics, significantly improved ambulance routing by reducing response times, particularly in congested urban settings. Its greedy mechanism prioritized minimizing perceived distance, enabling the nearest available ambulance to reach emergency sites more efficiently (Erdemir et al., 2010).

Applications of Greedy Best-First Search on Fire Emergency

A key study applied GBFS to simulate fire evacuation scenarios in large buildings, demonstrating its effectiveness in dynamically adapting to changing conditions (Shahidi et al., 2022). When integrated with real-time hazard data

such as fire and smoke spread information where GBFS significantly reduced evacuation times by continuously adjusting routes as conditions evolved. The algorithm offered a flexible solution for guiding evacuees away from immediate danger by optimizing paths based on updated hazard maps. Researchers evaluated GBFS's efficiency using various heuristic assumptions, including straight-line distances and travel times adjusted for environmental hazards. Their findings indicated that the algorithm's rapid convergence, which prioritizes minimizing perceived distance, resulted in faster evacuations, particularly in large, crowded, or unfamiliar environments. However, Shahidi et al. (2022) also emphasized that GBFS's dependence on heuristics can occasionally produce suboptimal routes in highly complex and rapidly changing fire conditions.

A study proposed that combined GBFS with Dijkstra's algorithm for fire evacuation planning, demonstrating improved balance between speed and optimality (Chen et al., 2021). The hybrid model leveraged GBFS's rapid heuristic-based exploration and Dijkstra's guaranteed optimal pathfinding to produce faster evacuation times while maintaining safe and efficient routes. The research highlighted how integrating GBFS with traditional graph-based algorithms enhances its reliability in highly dynamic fire environments, where both adaptability and path accuracy are critical (Chen et al., 2021).

Applications of Greedy Best-First Search on Evacuation Emergency

One key application of GBFS in disaster response is demonstrated in the thesis by (Xie, 2016) which introduced a rescue robot navigation system using GBFS in partially observable environments. The study focused on the effectiveness of GBFS in search-and-rescue missions where robots need to

navigate uncertain and dynamically changing environments, such as collapsed buildings after an earthquake or flooded areas. Despite these challenges, GBFS proved useful in guiding robots to optimal evacuation routes by using heuristic functions based on the available data, such as distance to safe zones or other navigation factors. (Xie, 2016) also acknowledged the limitations of GBFS in highly uncertain environments. While GBFS is efficient in terms of speed, it can lead to suboptimal paths if the heuristic is not well-calibrated or if there are significant changes in the environment that are not accounted for in the initial data. In the context of dynamic disaster scenarios, this can result in robots taking routes that are no longer the most optimal due to unforeseen obstacles or hazards.

Discrete Event Simulation in Emergency Response

Discrete Event Simulation has emerged as a valuable tool for modeling and improving emergency response systems. In the study by (Wei Lam et al., 2014), a DES model was developed for Singapore’s Emergency Medical Services using retrospective call data to evaluate various policy alternatives aimed at reducing ambulance response times. The study found that optimizing dispatch policies and reallocating ambulances could match the performance gains of adding new vehicles without incurring additional costs or reducing utilization efficiency. This demonstrated DES's potential for informed decision-making in operational strategies under resource constraints.

Similarly, (Debacker et al., 2016) introduced the SIMEDIS model, a DES framework designed to simulate disaster medical response systems. Unlike traditional analytic methods, SIMEDIS enables the parallel tracking of victim

health states and their interactions with available medical resources during mass casualty incidents. Applied to a simulated airplane crash scenario, the model provided a dynamic and structured approach to testing medical response plans and resource allocation strategies in complex disaster environments.

Hybrid and AI-Augmented Pathfinding Algorithms

Recent developments in AI have inspired hybrid pathfinding algorithms that combine classical methods such as A*, Dijkstra's, ACO with machine learning and evolutionary computation techniques. These approaches address the limitations of traditional algorithms, which often struggle with dynamic and uncertain environments. For instance, machine learning-enhance A* models can learn from historical or real-time data to refine their heuristic functions, enabling faster and more adaptive route recalculations in changing conditions. Similarly, Genetic-ACO hybrids merge pheromone-based learning with genetic optimization, improving convergence speed and solution quality in large-scale or disrupted networks.

This hybrid and AI-augmented algorithms represent a significant advancement toward intelligent, data-driven routing systems for real-world applications such as emergency response and intelligent transportation. These methods enable more predictive and context-aware decision-making compared to purely and context-aware decision-making compared to purely heuristic or metaheuristic techniques by integrating learning-based adaption and multi-objective optimization. Their potential to improve pathfinding accuracy, adaptability, and computational efficiency makes them a promising direction for future research and system enhancement.

Literature Matrix

Table 1. Literature Matrix

Author(s)	Title	Year	Method/ Algorithm	Findings	Key Variables / Performance Metrics
Eido W. M. & Ibrahim M. I.	Ant Colony Optimization for Traveling Salesman Problem: A Review	2025	ACO variants for Traveling Salesman Problem path optimization.	ACO is effective for dynamic and complex path environments .	Path cost, computation speed adaptability
Liu W. et al.	Efficient Path Planning for Emergency Medical Services with Stable Connectivity Demands	2024	Modified A* on damaged road maps.	Improved evacuation success rates by 20% compared to manual planning.	Evacuation success rate, computation time
Ronghua M. et al.	Improved Ant Colony Optimization for Safe Path Planning of AUV	2024	Improved ACO for AUV path planning with obstacle avoidance.	ACO adapts better in unpredictable environments than traditional methods.	Adaptability, obstacle avoidance accuracy
Biswas S. et al.	Grid-Based Pathfinding Using Ant Colony Optimization Algorithm	2023	ACO algorithm with grid preprocessing.	Improved efficiency and solution quality over basic ACO	Path cost, computation time
Ghazali J. M. et al.	An Application of the Dijkstra's Algorithm Model in Finding Shortest Traveling Time of Ambulance to	2023	Dijkstra's algorithm for ambulance travel time optimization.	Improved navigation efficiency under city conditions.	Travel time, route optimality

	Hospital Kuala Lumpur				
Husain Z. et al.	Search and Rescue in a Maze-like Environment with Ant and Dijkstra Algorithm	2022	Dijkstra's on 3D space for drones.	Fast and reliable emergency deliveries.	Speed, accuracy, reliability
Jesuthas & Somaskanda N.	Pathfinding and Planning in a 3D Environment: An Analysis using Bidirectional Versions of Dijkstra's, Weighted A*, and Greedy Best-First Search Algorithms	2022	Modified GBFS heuristic with adaptive learning.	Enhanced pathfinding speed and accuracy compared to standard GBFS.	Search speed, heuristic accuracy
Lawande S. R. et al.	A Systematic Review and Analysis of Intelligence-Based Pathfinding Algorithms in the Field of Video Games	2022	Simulation and performance testing on grid environments.	GBFS is faster but less reliable than A* in complex terrains.	Speed, reliability, path optimality
Wayahdi M. R. et al.	Greedy, A* and Dijkstra's Algorithms in Finding Shortest Path	2021	Practical testing on different weighted graphs.	GBFS is fast at finding solutions but tends to sacrifice optimality in favor of speed.	Computation time, path optimality
Yu J. et al.	A Novel Parallel Ant Colony Optimization Algorithm for Warehouse Path Planning	2020	Parallel Ant Colony Optimization for path planning.	Significant reduction in computation time for path planning.	Computation time, scalability

Ahmed S. et al.	Mobile-based Routes Network Analysis for emergency Response using an enhanced Dijkstra's Algorithm and AHP	2018	Enhanced Dijkstra's and AHP for mobile-based emergency response route analysis.	Dijkstra ensures safety-focused pathfinding in emergencies.	Safety, route efficiency
Michael T. & Xavier D.	Intelligent Ambulance Management System with A* Algorithm	2018	Real-time ambulance routing using enhanced A* algorithm with dynamic traffic integration.	Achieved efficient multi-ambulance dispatch coordination.	Response time, dynamic adaptability
Eneh A. & Arinze U.	Comparative Analysis and Implementation of Dijkstra's Shortest Path Algorithm for Emergency Response and logistic Planning	2017	Simulations using real city maps.	Dijkstra is reliable but A* is faster in urban navigation.	Speed, reliability
Metha P. et al	A Review on Algorithms for Pathfinding in Computer Games	2015	Review of pathfinding algorithms in computer games.	GBFS is efficient but may not guarantee the optimal path.	Search efficiency, path optimality
Sharma S. K. & Pal B. L.	Shortest Path Searching for Road Network using A* Algorithm	2015	Simulation compared grid environments by path cost and computation time.	A* was faster and more efficient, while Dijkstra's found the shortest path but with higher cost.	Path cost, computation time
Kai N. et al.	Shortest Path Analysis based on Dijkstra's Algorithm in Emergency	2014	Traditional Dijkstra with weighted road networks.	Dijkstra consistently finds the shortest route	Path length, execution speed

	Response System			but slower than A*.	
Nordin N. A. M. et al.	Finding Shortest Path of the Ambulance Routing: Interface of A* Algorithm	2012	GPS integration with A* algorithm.	Reduced response times even during peak traffic.	Response time, adaptability
Nordin N. A. M. et al.	An Application of the A* Algorithm on the Ambulance Routing	2011	A* combined with road network graphs.	A* provides quicker and adaptable paths in urban areas.	Path cost, real-time adaptability
Wen Z. & Cai Z.	Global Path Planning Approach based on Ant Colony Optimization Algorithm	2006	ACO algorithm implementation with environment modeling.	ACO provides near-optimal paths in a reasonable time.	Path optimality, processing time

Synthesis of the Study

The reviewed literature emphasizes the critical role of pathfinding algorithms in optimizing navigation, logistics, and emergency response systems. Algorithms such as A*, Dijkstra's, GBFS, and ACO have been widely explored due to their ability to compute efficient routes under various conditions. A* is recognized for balancing accuracy and computational efficiency, making it suitable for dynamic routing tasks like urban navigation and rescue operations. Dijkstra's Algorithm, while reliable in guaranteeing the shortest and most accurate path, tends to be slower when applied to larger and more complex networks. GBFS prioritizes speed and simplicity, generating faster results but often at the expense of optimality. Meanwhile, ACO, excels in

adaptive and uncertain environments, making it ideal for large-scale and dynamic applications such as disaster management and multi-agent routing.

The literature also highlights the increasing integration of real-time data sources, such as GPS, traffic updates, and environmental sensors, to enhance the performance of these algorithms in fast-changing conditions. Studies suggest that combining heuristic and metaheuristic approaches, or incorporating machine learning, can improve adaptability, reduce computation time, and produce near-optimal paths even in unpredictable environments. Overall, the effectiveness of each algorithm depends on the nature of the environment, data availability, and system complexity.

Conceptual Framework of the Study

This study follows a structured, multi-phase framework to compare the performance of A*, ACO, Dijkstra's Algorithm, and GBFS in dynamic emergency scenarios. It begins with data acquisition, collecting road network and traffic data, followed by preprocessing into graph structures and data splitting to ensure accurate and reliable evaluation.

The framework integrates four pathfinding algorithms within a Discrete Event Simulation that models real-world emergencies such as earthquakes, floods, and fires. It employs dynamic feedback loops that trigger automatic re-routing when roads become blocked, allowing algorithms to recompute optimal paths in real time. Each algorithm's performance is evaluated based on computation time, scalability, and adaptability, enabling a comparative assessment of their efficiency and responsiveness in emergency scenarios.

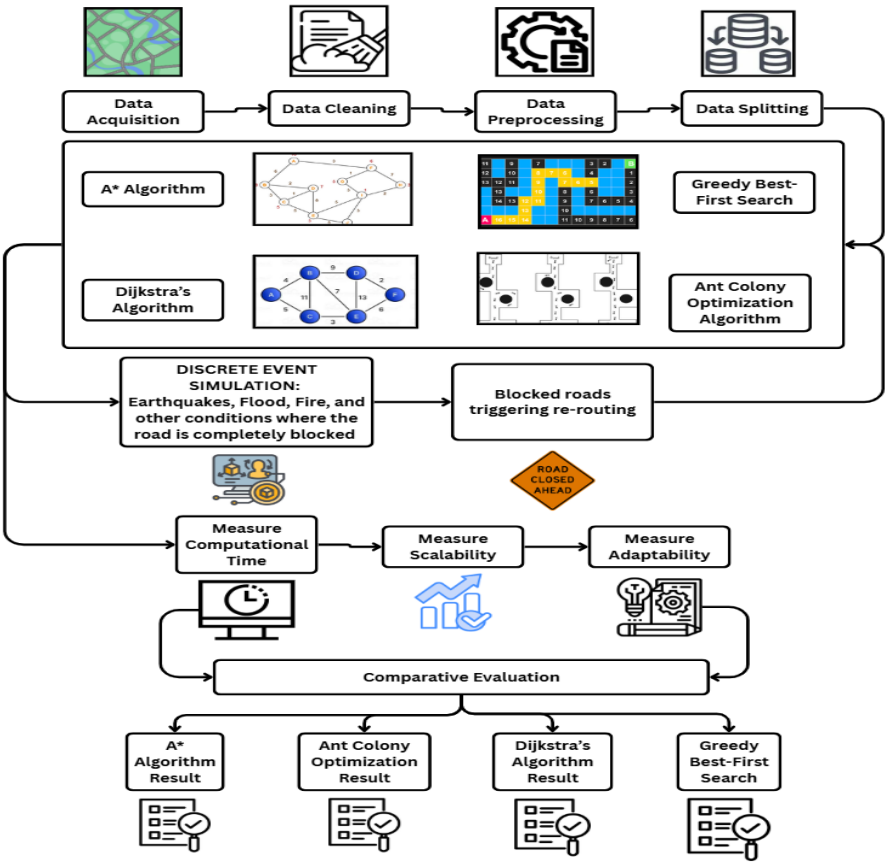


Figure 1. Conceptual Framework

Summary of Algorithm Strengths, Weaknesses, and Common Cases

The reviewed literature highlights that each pathfinding algorithm offers distinct advantages and limitations depending on the network conditions and objectives of the routing task. Heuristic algorithms such as A*, Dijkstra's, and Greedy Best-First Search emphasize efficiency and optimality, particularly in static environments. While bio-inspired approaches like Ant Colony Optimization emphasize adaptability and learning in complex, changing networks focus on adaptability and learning complex and evolving networks. This table summarizes the key strengths, weaknesses, and typical applications of the four algorithms considered in this study.

Table 2. Short Summary of Algorithms

Algorithm	Type	Strengths	Weaknesses	Common Use Cases
Dijkstra	Graph-based/ Heuristic	Guarantees shortest and optimal path; stable and reliable in static environments	Computationally heavy for large networks; less adaptive to changes	Transportation systems, navigation apps, static route planning
A*	Heuristic	Fast and efficient; balances accuracy and computation; supports dynamic recalculations	Performance depends on heuristic function quality	Real-time navigation, robotics, emergency routing
GBFS	Heuristic	Very fast; low computation time; easy to implement	Often produces suboptimal routes; ignores path cost	Quick approximations, low-priority routing tasks, simulation prototypes
ACO	Metaheuristic/ Bio-inspired	Adaptive to dynamic environments; learns from previous iterations; scalable	Slow convergence; high computation time	Disaster response systems, adaptive routing, logistics optimization

CHAPTER 3

METHODOLOGY

This chapter presents the methodological framework used to compare four pathfinding techniques such as A*, Ant Colony Optimization, Dijkstra's Algorithm, and Greedy Best-First Search within an emergency response simulation. The testing is conducted within a computer-based model that replicates actual rescue scenarios. In addition to software tools that carry out the runs and performance evaluation metrics that evaluate each algorithm's computational efficiency, path optimality, and flexibility in the presence of sudden simulation changes, it offers a comprehensive analysis of the experiment design.

Research Design

The study developed, simulated, and evaluated several pathfinding algorithms for time-critical emergency response routing using a theoretical–experimental research design. Using dynamic emergency scenarios created by Discrete Event Simulation, the main objective was to evaluate and compare the performance of four well-known techniques: A*, Dijkstra, ACO, and GBFS.

The theoretical component was on creating a simulation framework that could simulate actual emergency scenarios like road blockages, earthquakes, fire, and floods. Discrete Event Simulation techniques were used to probabilistically introduce these disruptions into a manually built road network, enabling the study to reflect uncertainty and dynamically changing route accessibility. This framework made it possible to test each algorithm's efficiency

and adaptability under various environmental disruption levels in controlled conditions.

The chosen techniques were carefully tested and their performance evaluated as part of the experimental component. Data on significant performance measures, including calculation time, path optimality, and success rate in identifying legitimate routes, were gathered while each method was run through a number of emergency case simulations. This hybrid research approach combined quantitative measurement of algorithm performance with qualitative modeling of dynamic emergency scenarios. All algorithms were subjected to the same emergency conditions in order to ensure comparability and consistency. A realistic assessment of each algorithm's reaction to changing circumstances was made possible by the use of Discrete Event Simulation, which offered a scenario-driven, time-aware environment. All things considered, this design made it easier to thoroughly compare the algorithms' advantages and disadvantages, providing information about which strategies work best for real-time emergency response applications.

Materials

The study relied on standard hardware and software to run pathfinding techniques, to simulate emergency response scenarios using Discrete Event Simulation, and to evaluate algorithm performance under dynamic conditions.

Python. The main language for the server. It handled data simulation, built and read the road graph, and also ran every algorithm. The Discrete Event Simulator was connected to the pathfinding technique via existing Python

libraries, allowing the model to develop as it performed and allowing for the measurement of results.

HTML. It is used to develop the web page that shows the simulation. It drew the map, the moving units, as well as the algorithm traces, so users and reviewers can watch side-by-side the pathfinding technique behaviors.

Google Colaboratory. Served as the online workspace. Scripts were written, tested, or executed there. It provided GPU access, real-time collaboration, and support for libraries such as NetworkX, NumPy, and Matplotlib for visualization and analysis.

Dataset. The dataset was manually constructed using Google Maps as a reference (data collected from September 21 to October 8, 2025) to represent the actual road network of Los Baños, Laguna. Each crossing became a node - each stretch of road became an edge, along with its approximate distances. Estimated speed limits were read based on observed road structures and visible traffic signs from Google Maps. The travel time for every segment was computed from distance divided by speed to guarantee the estimated travel time appears in the simulation.

Libraries

Flask. Used to develop the system's backend framework, enabling the creation of RESTful APIs in simplified way that facilitate communication between the web-based interface and the simulation model. Flask-CORS was integrated to allow secure cross-origin data sharing during testing and deployment.

Pandas. Used to manage and preprocess structured data that includes simulation outputs, nodes, and edges. It allowed connecting with other model elements and manipulating data more efficiently.

NumPy. Used to enable the creation of probabilistic events in the road network using numerical operations and random sampling techniques in the Discrete Event Simulation.

NetworkX. Acted as the main library for building, examining, and updating graph structures that simulate the road network in real time.

Matplotlib and Seaborn. Used to visualize algorithm performance, simulation results, and the road network. These libraries included visual representations of the consequences of dynamic disruptions, route efficiency, and node connectivity.

Scikit-learn. Used for dataset segmentation using train-test splitting, providing a consistent evaluation of model performance under various simulation scenarios.

Hardware

A local workstation with enough processing power for algorithm development, simulation, and visualization was used to create and test the model. The main platform was a laptop, which offered the processing capability required to successfully implement and evaluate the model. This provided sufficient computational resources to ensure stable execution of simulations and consistent performance measurement across all experimental runs.

Table 3. Laptop Specification

Components	Unit/Details
Processor	Intel(R) Core(TM) i3-1005G1 CPU @ 1.20GHz
Memory	19.8 GB
Hybrid Hard Drive	932 GB
Solid State Drive	119 GB
System Type	X64-BASED PC
GPU	Intel(R) UHD Graphics

Model Development

The model development process began with the creation of a localized road network dataset representing the municipality of Los Baños, Laguna. Using Google Maps as a reference, the network was manually constructed and encoded into a graph structure, where intersections served as nodes and road segments as edges. Each edge was assigned key attributes such as distance, speed limit, and estimated travel duration to reflect realistic travel conditions within the area.

The simulation framework integrated this dataset with four pathfinding algorithms such as A*, Dijkstra's Algorithm, Greedy Best-First Search, and Ant Colony Optimization. Various disaster scenarios, including floods, earthquakes, and fires, were introduced to simulate road blockages and dynamic disruptions. These disaster scenarios like flood, earthquake, and fire were simulated by introducing random road blockages across the network, except in the fire

scenario, where the affected areas followed a predefined pattern. This setup aimed to test each algorithm's adaptability and performance under unpredictable and dynamic emergency conditions.

Performance data were collected throughout the simulations, including path cost, computation time, success rate, and node exploration count. These metrics provided a quantitative basis for evaluating the efficiency and robustness of each algorithm. The results were then visualized through route maps and comparative charts to illustrate the algorithms' behaviors and effectiveness across multiple emergency settings.

Algorithm Analysis

The A* algorithm combines the strengths of Dijkstra's Algorithm and Greedy Best-First Search by using both actual and estimated costs to find optimal routes efficiently. It is effective for emergency routing due to its balance of speed and accuracy but may require adaptation for real-time changes and can become computationally intensive in large networks with poorly defined heuristics.

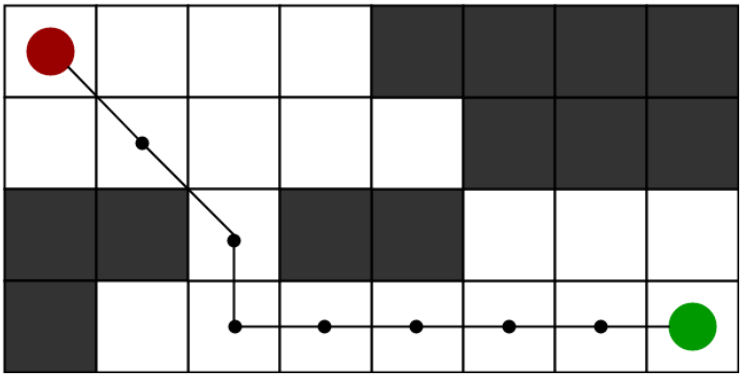


Figure 2. Flow of A* Algorithm

The Ant Colony Optimization algorithm, inspired by ant foraging behavior, excels in dynamic and uncertain environments through iterative learning and pheromone updates. However, it requires longer convergence time and precise parameter tuning, and its stochastic nature can lead to inconsistent results in time-critical emergency situations.

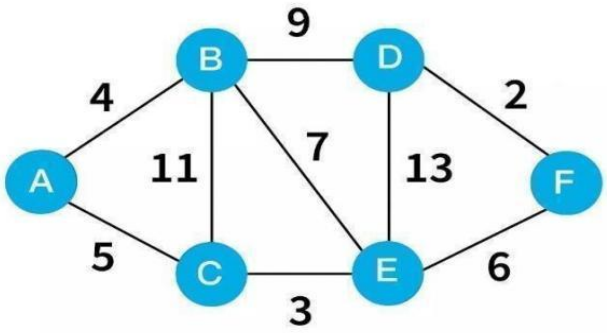


Figure 3. Flow of the Ant Colony Optimization (ACO)

Dijkstra's Algorithm guarantees the shortest and most accurate path by exhaustively exploring all routes without heuristics. However, its computational intensity and static nature limit its efficiency and adaptability in large or time-sensitive, real-world emergency scenarios.

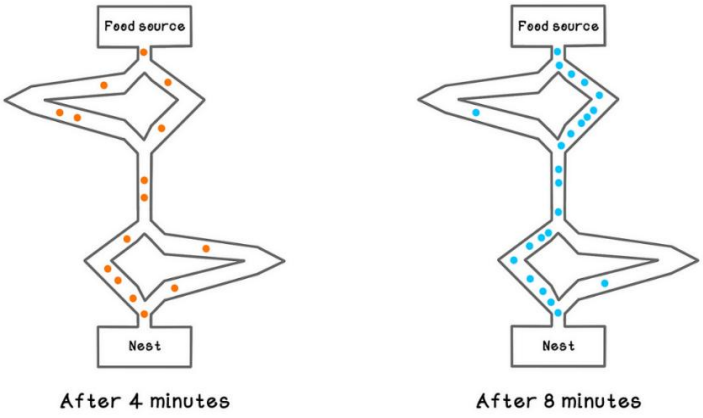


Figure 4. Flow of the Dijkstra's Algorithm

The Greedy Best-First Search algorithm emphasizes speed using heuristics to quickly find routes but sacrifices accuracy by ignoring actual path costs. While suitable for fast decision-making, its lack of optimality makes it less reliable for emergencies requiring precise and efficient routing.

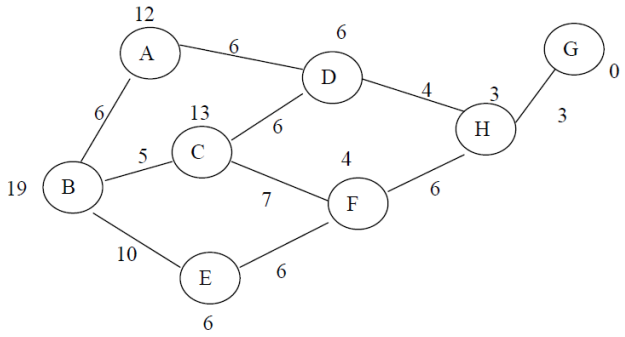


Figure 5. Flow of the Greedy Best-First Search (GBFS)

Simulation Design

The simulation was designed to represent different disaster conditions that can affect the road network used by the pathfinding algorithms. In the fire disaster simulation, the affected node may include the area near the user-selected end node. The fire can spread to adjacent nodes and connected edges, causing them to become blocked as the fire continues to propagate through nearby areas. In contrast, the flood and earthquake simulations randomly block a set of roads in the network to represent areas that have become impassable due to water build up or structural damage. All affected edges in these two scenarios are automatically marked as blocked. Because the blocking process is random, there are cases where all possible routes leading to the destination become inaccessible, resulting in a “Did Not Finish” (DNF) output.

Flowchart

The system flowchart illustrates the overall process of the simulation-based comparison of four pathfinding algorithms. The process starts with the researcher, who inputs the start and end nodes into the system. Next, the researcher sets up the simulation by selecting whether to include road blockages, flood, fire, earthquake, or none. If the road blockage simulation is chosen, the researcher can add up to ten blockages on the map. For the fire simulation, the system automatically places the start or end node, or a nearby node, to simulate restricted or hazardous areas. After setting up the parameters, the system initializes the simulation map environment, which serves as the area where the pathfinding will occur. The parallel algorithm execution module then runs all algorithms simultaneously under the same map conditions. Each algorithm performs its own pathfinding process, simulating a race to determine the most efficient path from the start to the end node.

Once all algorithms have completed their execution, the Result Analyzer evaluates their performance based on defined metrics such as path cost, path length, execution time, and travel time. The results are then forwarded to the Visualization and Ranking Display, where the system presents the computed paths and displays the rankings of the algorithms according to their performance.

This structured flow ensures a fair and controlled comparison by running all algorithms under identical conditions. It also enables clear performance evaluation and visual interpretation of results for effective analysis.

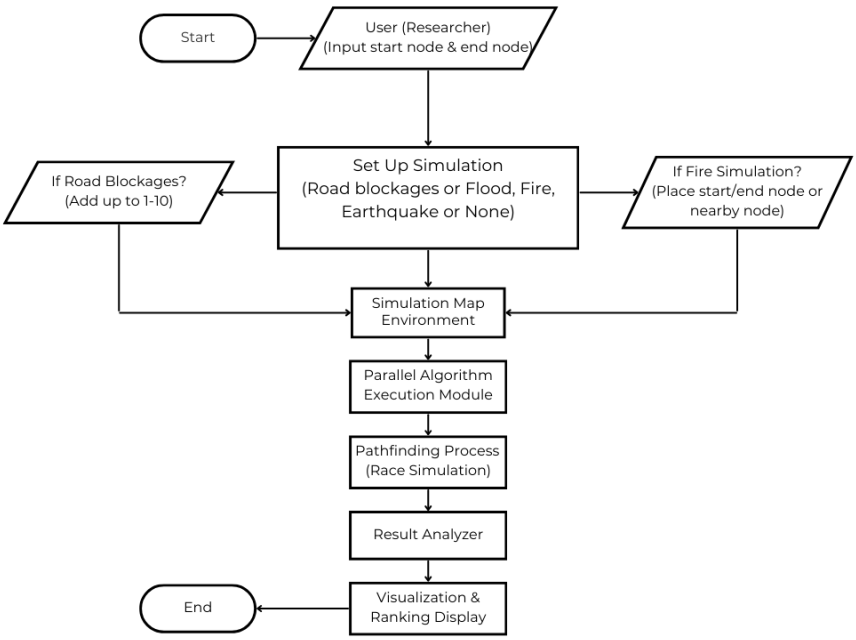


Figure 6. Flowchart

Simulation Parameters

In this study, multiple pathfinding algorithms were simulated under identical environmental conditions to ensure fair performance evaluation. Each was tested using the same road network dataset, dynamic event settings, and evaluation metrics such as path cost, computation time, and adaptability.

ACO model, specific parameters were configured to control its search behavior and convergence rate. The simulation utilized 15 ants and executed 100 iterations per run to allow adequate exploration of possible routes within the network. A pheromone decay rate of 0.5 was applied to simulate the gradual evaporation of pheromone trails, preventing premature convergence on suboptimal paths. The heuristic weightings were set as $\alpha = 1$ and $\beta = 3$, where α determines the influence of pheromone intensity on route selection, and β emphasizes the importance of heuristic factors such as distance or path quality.

These parameter settings were chosen to achieve a balanced trade-off between exploration and exploitation, ensuring efficient and adaptive performance during simulated emergency routing scenarios.

Data Acquisition Implementation

The collected data gathered from Google Maps was utilized as the primary graph input for the simulation model. The information represented the actual road network, with edges signifying potential routes and nodes acting as crucial intersections. During emergency simulations, the pathfinding algorithms employed this graph structure to identify the most effective paths.

Different crisis conditions such as earthquakes, fires, and floods were introduced to simulate real-world obstacles by blocking specific roads or edges. The system was able to evaluate how each algorithm identifies alternate routes and adjusts to changing situations since these blockages dynamically changed the available pathways. Throughout this process, the data was essential in showing how each algorithm behaved and worked in actual emergencies.

Algorithm Implementation

The four pathfinding techniques were implemented in Python within the developed simulation model. They were able to calculate routes in various types of emergency scenarios, including earthquakes, fires, and floods by applying the algorithms to the created network graph data. The resulting pathways, calculation time, and route dependability were noted for study after important parameters were changed to guarantee a fair performance comparison.

Theorems

Each of the four pathfinding algorithms used in this study is grounded in distinct theoretical principles that justify their behavior and performance in simulated emergency scenarios.

A* is governed by two fundamental theorems. The Admissibility Theorem, as proposed by Hart et al. (1968), states that if the heuristic function *h(n)* does not overestimate the true cost to reach the goal from any node, then A* is guaranteed to find the optimal path. Complementing this is the Consistency Theorem by Pearl (1984), which asserts that if the heuristic satisfies the triangle inequality, meaning $h(n) \leq c(n, n') + h(n')$ for every node nnn and its successor *n'—A** will not re-expand any node, thereby improving computational efficiency.

Dijkstra's Algorithm, developed by Edsger W. Dijkstra in 1959, is based on the Shortest Path Theorem, which ensures that the algorithm will always find the shortest path from a single source to all other nodes in a graph with non-negative edge weights.

Greedy Best-First Search operates under the Greedy Heuristic Principle, as described by (Russell and Norvig, 2020). It prioritizes nodes that appear closest to the goal based solely on the heuristic function *h(n).* However, because it does not consider the cost from the start node, it does not guarantee an optimal solution unless the heuristic is exact. It may quickly reach a solution but can become trapped in suboptimal paths, especially in complex or obstructed environments.

ACO draws from principles in swarm intelligence. The Convergence Theorem, outlined by (Dorigo and Stützle, 2004), suggests that ACO can asymptotically approach the optimal solution by reinforcing successful paths with pheromone trails over multiple iterations. This is supported by the Stigmergy Principle introduced by Bonabeau et al. (1999), which explains how ants indirectly communicate via environmental cues, allowing for collective path construction that balances exploration and exploitation.

Pseudocode

A* Pseudo-code

The A* algorithm finds an efficient route by combining the actual travel cost with a heuristic estimate of the remaining distance to the goal. It repeatedly selects the node with the lowest combined cost, updates neighboring paths when better routes are found, and continues until the destination is reached.

	A* Algorithm
1	`Initialize OpenList with Start node`
2	`Initialize ClosedList as empty`
3	`Set g(Start) = 0 and f(Start) = h(Start)`
4	`While OpenList is not empty:`
5	`  Select node n with the lowest f(n)`
6	`  If n = Goal: return path`
7	`  Move n to ClosedList`
8	`  For each neighbor m of n:`
9	`  If m in ClosedList: continue`
10	`  tentative_g = g(n) + cost(n, m)`
11	`  If m not in OpenList or tentative_g < g(m):`
12	`  g(m) = tentative_g`
13	`  f(m) = g(m) + h(m)`
14	`  parent(m) = n`
15	`  Add m to OpenList if not present`
16	`Return failure (no path found)`

Dijkstra Pseudo-code

Dijkstra's Algorithm finds the shortest paths by evaluating only actual travel costs, iteratively selecting the nearest unvisited node and updating its

neighbors. Because it does not use heuristics, it provides consistent and reliable shortest-path results.

Dijkstra Algorithm

```
For each node v in Graph:
    distance[v] = ∞
       previous[v] = undefined
distance[Start] = 0
Q ← all nodes in Graph
While Q is not empty:
         u ← node in Q with smallest distance
         Remove u from Q
         For each neighbor v of u:
                alt = distance[u] + weight(u, v)
                If alt < distance[v]:
                     distance[v] = alt
                     previous[v] = u
Return shortest paths and distances
```

Ant Colony Optimization Pseudo-code

Ant Colony Optimization is a population-based algorithm inspired by ant foraging behavior. It uses pheromone trails and distance heuristics to guide multiple agents in exploring routes, updating pheromones through evaporation and reinforcement, and gradually converging toward an efficient path through repeated iterations.

Ant Colony Optimization

```
Initialize pheromone levels on all edges
For each iteration:
       For each ant:
              Construct path based on pheromone and heuristic
values
              Compute path cost
       Update pheromones:
                Evaporate pheromones from all edges
                Reinforce pheromones on edges used by best-
performing ants
Return best path found
```

Greedy Best-First Search Pseudo-code

Greedy Best-First Search was used as a heuristic-driven method that prioritizes nodes based solely on their estimated distance to the goal. The algorithm begins with the starting node and always selects the next node with

the lowest heuristic value. If the selected node is not the goal, its neighbors are explored and added to the Open List provided they have not already been processed. Although this method is faster than many other search algorithms, it may not always produce the optimal path because it ignores the actual distance traveled.

Greedy Best-First Search

```
1   Initialize OpenList with Start node
2   Initialize ClosedList as empty
3   While OpenList is not empty:
4           n ← node in OpenList with lowest h(n)
5           If n = Goal: return path
6           Move n to ClosedList
7           For each neighbor m of n:
8                   If m not in ClosedList:
9                           parent(m) = n
10                          Add m to OpenList
11  Return failure (no path found)
```

Model Analysis and Evaluation

Average Path Cost

The average path cost refers to the total weight or distance of the computed route from the start node to the end node. This metric was used to evaluate the efficiency of each pathfinding algorithm in identifying the least costly route. A lower average cost indicates better performance, particularly in time-critical emergency scenarios.

Execution Time

Execution time measures how quickly an algorithm completes its pathfinding process. In emergency response systems, speed is crucial. This metric was used to compare the responsiveness of various pathfinding techniques under simulated emergency conditions generated through Discrete Event Simulation.

Success Rate

The success rate represents the percentage of simulation runs in which the algorithm successfully found a valid path despite the presence of road blockages caused by disasters such as floods, fires, or earthquakes. This metric evaluates the robustness of each algorithm in dynamic environments where some routes may become inaccessible. A higher success rate indicates better reliability in emergency routing situations.

Discrete Event Scenario Coverage

The Discrete Event Scenario Coverage describes how consistently each algorithm performs across multiple simulation runs involving different types of disasters. It reflects the adaptability of the algorithms to changing environments, rather than serving as a quantitative performance metric.

Node Expansion Count

The node expansion count measures how many nodes each algorithm explores before reaching the destination. In this study, a higher node count often corresponds to longer routes and increased travel time. Algorithms that explore fewer nodes generally demonstrate greater search efficiency, particularly in minimizing computation effort and path cost.

Impact Score Interpretation

Instead of calculating a formal composite score, the evaluation was done descriptively by observing how each algorithm performed in terms of path cost, execution time, success rate, and node expansion count. The analysis focused

on identifying which algorithm showed the best balance of efficiency, reliability, and adaptability across different disaster scenarios.

Table 4. Performance Metrics Table

Metrics	Description	Purpose in Evaluation	Unit of Measurement
Path Cost	Represent the total accumulated cost or weight of the path generated by the algorithm from the start to the destination node.	To determine how efficiently each algorithm minimizes the overall cost in reaching the goal.	Cost units or distance-weighted value
Path Length	Refers to the total number of nodes or edges included in the final path.	To assess the compactness and optimality of the rote generated by each algorithm.	Number of nodes or edges
Travel Time	Indicates the estimated time it takes to travel from the start node to the goal node based on the generated path.	To measure the effectiveness of each algorithm in producing faster routes under different conditions.	Minutes (min).
Execution Time	Refers to the computational time taken by the algorithm to complete the pathfinding process.	To evaluate the speed and efficiency of each algorithm during the simulation	Seconds (s)

The four performance metrics were used to evaluate and compare the performance of the implemented pathfinding algorithms. These metrics were chosen to measure how each algorithm performs in terms of accuracy, efficiency, and overall effectiveness during the simulation process.

The Path Cost measures the total accumulated weight or cost from the starting node to the destination node. This metric helps determine how

efficiently an algorithm can minimize the total cost when finding the shortest or most optimal path. The Path Length represents the total number of nodes or edges included in the final route, which indicates the compactness and optimality of the generated path.

The Travel Time refers to the estimated time it takes to travel from the start node to the goal node, measured in minutes (min). This metric evaluates how quickly each algorithm can produce a path under different simulated conditions, such as road blockages or disaster scenarios. Lastly, the Execution Time measures how long it takes for each algorithm to finish the pathfinding process, measured in seconds (s).

Validation Method

The validation process was conducted to ensure that both the dataset and the developed simulation model are accurate, consistent, and reliable for the purpose of the study. It aims to verify that the data correctly represent the actual study area and that the model performs as intended based on its design and objectives.

The first phase involved validating the road network dataset, which was reviewed by the Municipal Disaster Risk Reduction and Management Office (MDRRMO) of Los Baños, Laguna. Their assessment confirmed that the dataset accurately reflected the municipality’s actual road layout, including routes, intersections, and access points, ensuring its suitability for emergency response simulations. This validity enhanced the credibility of the dataset by ensuring that the simulated routes and conditions were consistent.

The second phase involved model validation, conducted by an expert who assessed the model's structure, logic, and overall functionality. Multiple test runs were performed under varying conditions, and the results were averaged to verify consistency and reliability.

Ethical Considerations

This study does not involve human participants or the collection of personal, sensitive, or confidential information. The dataset used in the simulation was manually created and simulated by the researchers based on publicly available map references and validated in coordination with the Municipal Disaster Risk Reduction and Management Office (MDRRMO) of Los Baños, Laguna. Proper acknowledgment and attribution were observed to ensure compliance with ethical standards, copyright laws, and intellectual property regulations.

The research adheres to principles of academic integrity, ensuring that all simulations, analyses, and results are presented honestly and transparently. No data was fabricated or misrepresented. Furthermore, the study emphasizes ethical responsibility in the potential deployment of such systems, ensuring alignment with data privacy laws, public safety protocols, and responsible technology use for the benefit of the community.

In addition, the study recognizes the importance of responsible interpretation and application of simulation results. The findings are intended solely for research and decision-support purposes and should not replace professional judgement in real emergency operations.

CHAPTER 4

RESULTS AND DISCUSSION

This chapter presents the results of the study and discusses the performance of four pathfinding algorithms such as A*, Ant Colony Optimization, Dijkstra's Algorithm, and Greedy Best-First Search in simulated emergency response scenarios. Using Discrete Event Simulation, dynamic environments with disruptions such as road blockages and flooding were modeled to evaluate each algorithm. The assessment was based on the performance metrics defined in Chapter 3, which include path cost, computation time, adaptability to dynamic conditions, success rate, resistance to obstacles, and accuracy. The findings show that Dijkstra's Algorithm consistently produced the most optimal routes, making it the most reliable for emergency routing despite slightly higher computation time. A* achieved near-optimal paths with faster execution, making it suitable for time-sensitive applications. ACO demonstrated strong adaptability but required more computation time, while GBFS was the fastest but often resulted in suboptimal routes. These results highlight the strengths and limitations of each algorithm and confirm Dijkstra as the best-performing method for emergency response applications.

Develop Model

The simulation model was developed using a localized road network of Los Baños, Laguna, which was manually constructed based on Google Maps. The road layout was encoded into a graph structure, where intersections served as nodes and road segments were represented as edges. Start and end

coordinates were manually designated within the map to initiate the comparison of the four pathfinding algorithms. A Discrete Event Simulation approach was applied to enable dynamic environmental conditions, such as road closures and simulated flooding, allowing the model to replicate real emergency scenarios where route availability changes over time.

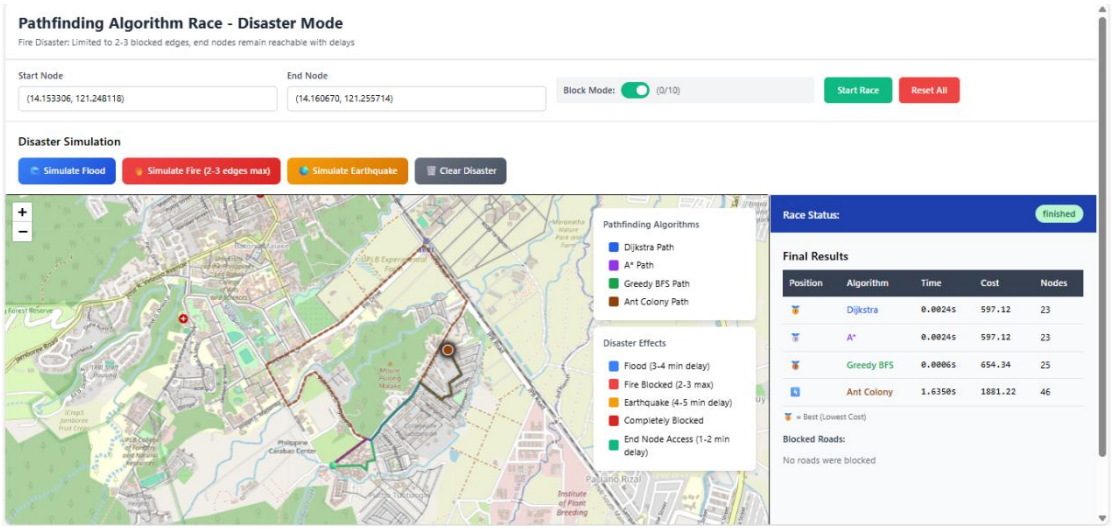


Figure 7. Pathfinding Algorithm Race System

Figure 7 shows the Pathfinding Algorithm Race System interface operating under normal conditions, with no road blockages or disaster simulations applied. The interface features a routing legend assigning distinct colors to each algorithm's path for clear visualization, along with displayed results for execution time, path cost, and nodes visited. Under these conditions, all four algorithms generated valid routes. Dijkstra's Algorithm ranked first for efficiency and accuracy, followed by A* with slightly slower computation. Greedy Best-First Search placed third, producing faster but less optimal routes, while Ant Colony Optimization ranked last with the slowest performance and least efficient path demonstrating clear performance differences even in a static environment.

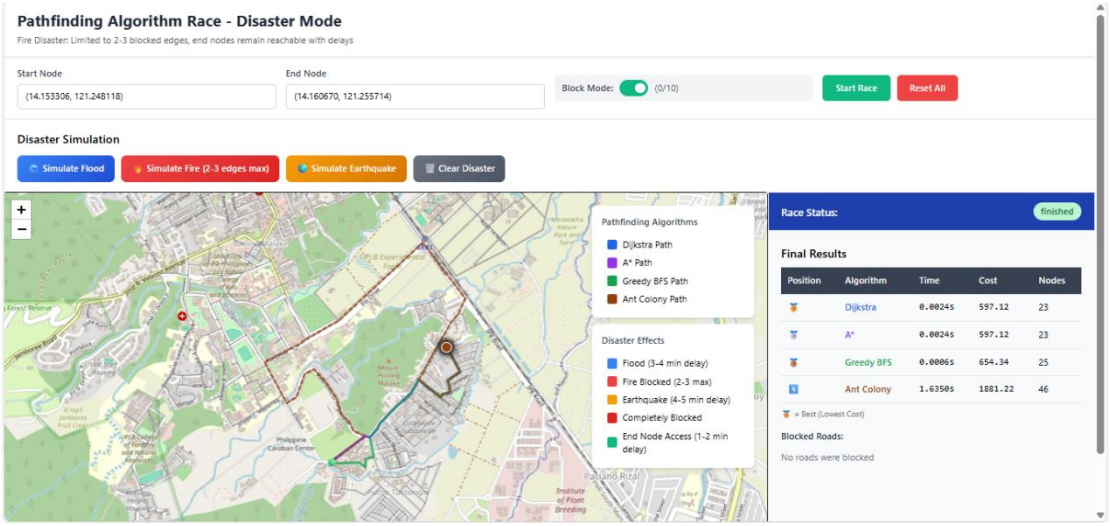


Figure 8. Pathfinding Algorithm Race System without Road Blockages

Figure 8 shows the system interface under Disaster Mode, simulating a flood scenario where 50 road segments were affected—44 blocked and 6 delayed—to evaluate how each algorithm adapts to disruptions and recalculates alternative routes. This setup created a highly constraint environment that tested the validity and rerouting capability of each pathfinding approach under sudden network changes.

Under these dynamic conditions, Dijkstra's Algorithm ranked first by maintaining high accuracy and reliability, as it consistently recalculated the most optimal routes despite extensive road blockages. A* followed closely, producing nearly identical path quality while responding efficiently to network changes through heuristic guidance. ACO ranked third, showing adaptability by adjusting pheromone trails to bypass affected roads, but this came with longer computation times due to its iterative process. GBFS performed the weakest, as its heuristic-driven strategy often led to inefficient detours and reduced accuracy in the flood-affected environment.

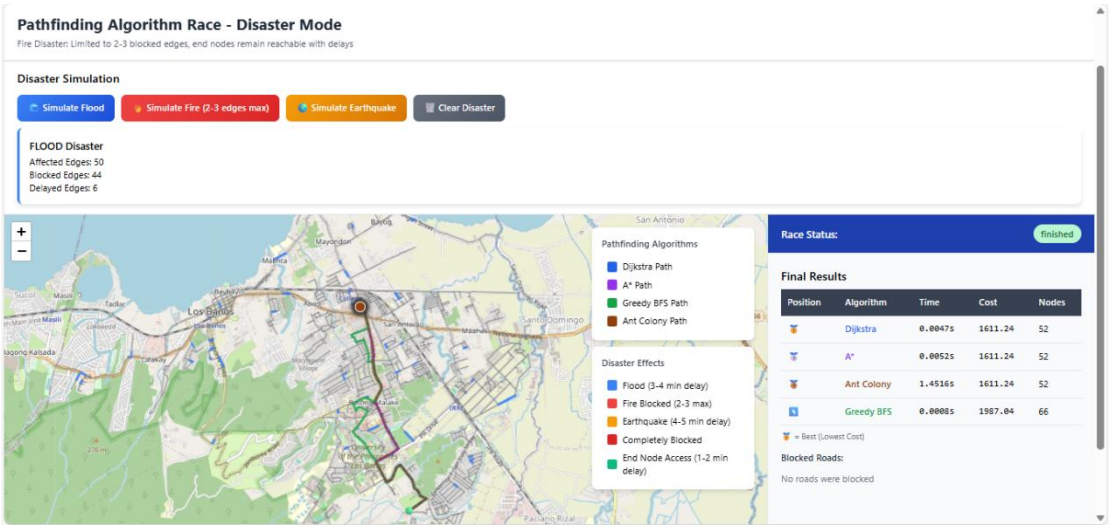


Figure 9. Pathfinding Algorithm Race System with Simulate Flood

Figure 9 presents the system interface during the fire simulation scenario, where the fire originated near the destination, causing partial road blockages and short delays along the final route segments. This setup was designed to evaluate the algorithms' ability to reroute effectively when the destination area itself becomes constrained, which is a common challenge in real emergency situations.

Dijkstra's Algorithm demonstrated the strongest performance by consistently identifying the most efficient alternative paths and maintaining route optimality despite limited access near the endpoint. A* followed closely, producing nearly the same path quality but with slightly higher computation time due to its heuristic calculations. ACO ranked third, as it was able to adapt through pheromone updates and learning mechanisms, although its iterative nature led to slower response times. GBFS performed the weakest, as its greedy decision-making near the goal often resulted in inefficient detours, leading to longer paths and reduced accuracy even though it executed quickly.

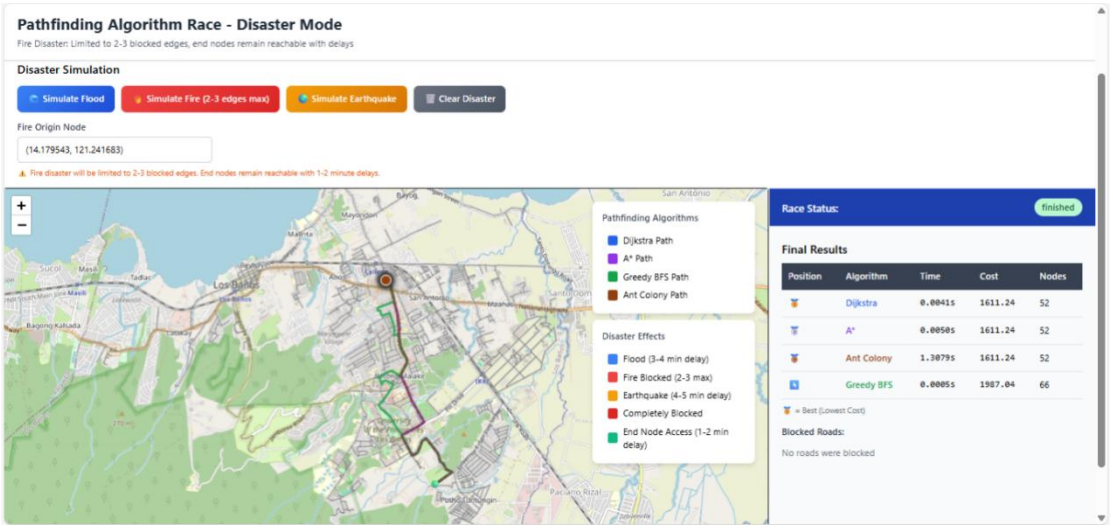


Figure 10. Pathfinding Algorithm Race System with Simulate Fire

Figure 10 presents the same interface during an earthquake simulation scenario. In this setup, the network experienced major disruptions, with several road segments delayed, partially blocked, or completely inaccessible. This simulation tested the algorithms under severe conditions to evaluate their ability to adapt to extensive and unpredictable changes in the road network.

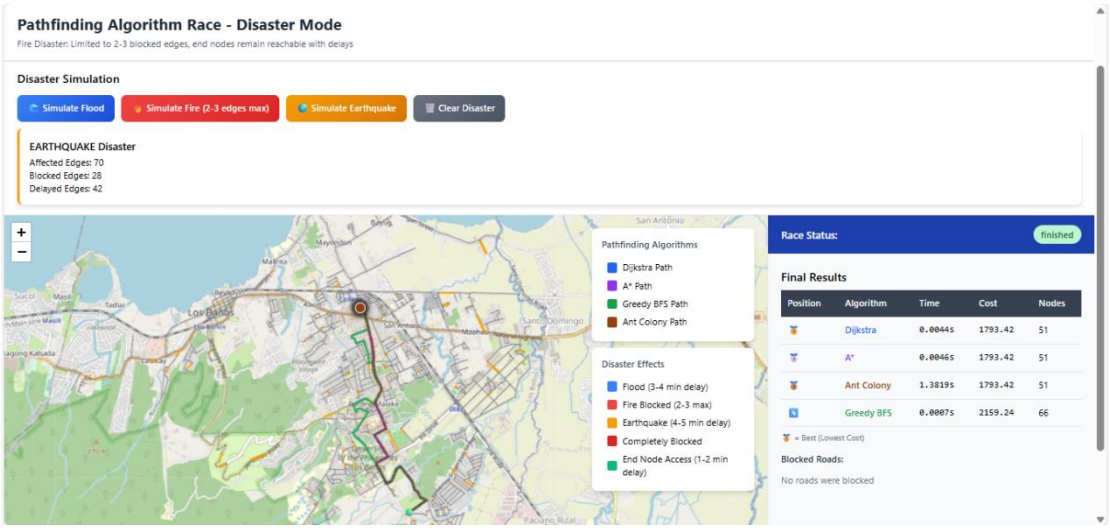


Figure 11. Pathfinding Algorithm Race System with Simulate Earthquake

Figure 11 illustrates the simulation results during the earthquake disaster scenario, where approximately 40 road segments were blocked across the

network. The Ant Colony Optimization algorithm failed to generate a valid route due to severe disruptions and disconnected paths, as indicated by the “Path Found: No” result. This outcome highlights ACO’s difficulty in highly fragmented networks, where its pheromone-based exploration could not converge on a feasible solution within the simulation time.

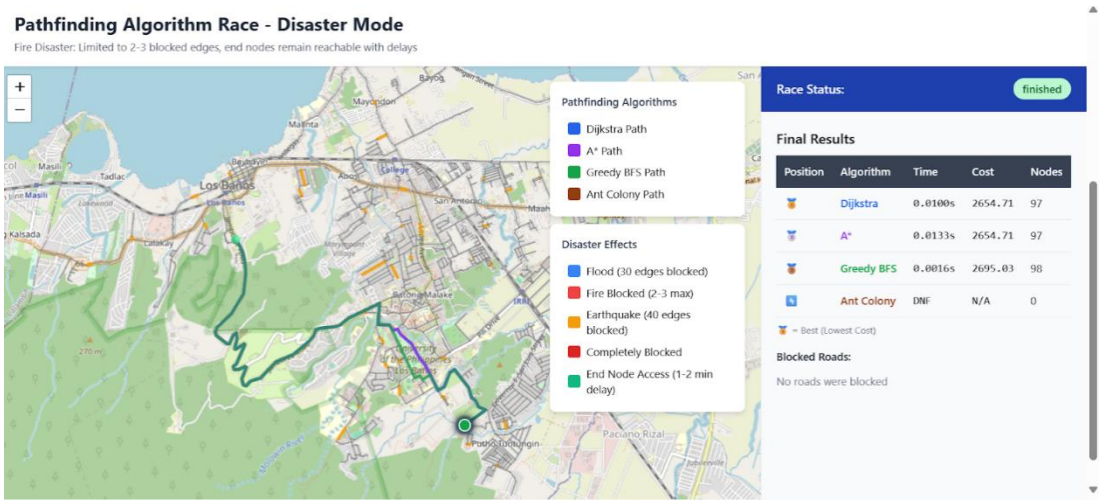


Figure 12. Earthquake Simulation Interface Showing Algorithm Failure under Severe Network Disruption

Figure 12 illustrates the overall performance of all algorithms during the earthquake simulation, where multiple road blockages and disruptions were introduced. Dijkstra’s and A* algorithms both produced optimal routes with identical path costs of 2654.71, demonstrating strong reliability and stability under severe conditions. GBFS achieved the fastest execution time but at a slightly higher cost of 2695.03, while ACO failed to find a valid path due to extensive network fragmentation. Overall, Dijkstra’s Algorithm proved to be the most robust and efficient, quickly recalculating alternative routes with consistent accuracy. A* followed closely, delivering nearly identical results with a marginally higher computation time, while ACO showed adaptability but slower

performance. GBFS remained the least effective, struggling to maintain accuracy and optimality despite its speed advantage.

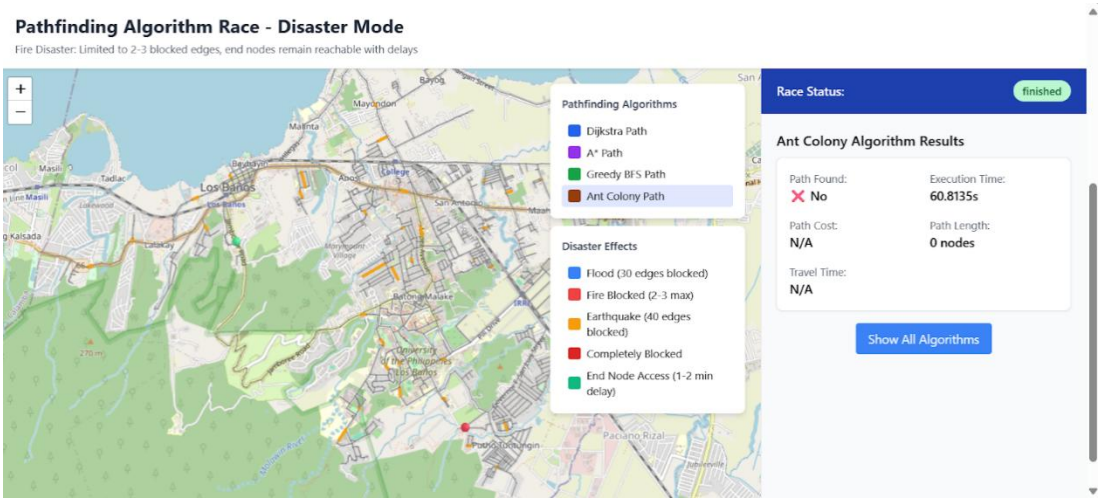


Figure 13. ACO Results under Earthquake Scenario

Figure 13 presents the overall performance of all algorithms during the earthquake simulation, where multiple road blockages and disruptions were introduced. Both Dijkstra's and A* algorithms achieved optimal routes with identical path costs of 2654.71, demonstrating strong reliability and stability despite widespread network damage. Greedy Best-First Search produced the fastest computation time but at a slightly higher cost of 2695.03, while Ant Colony Optimization failed to generate a valid path due to severe network fragmentation.

Overall, Dijkstra's Algorithm emerged as the most reliable and efficient, quickly recalculating alternative routes with consistent accuracy. A* closely followed, showing similar performance with slightly longer computation time due to its heuristic process. Ant Colony Optimization exhibited adaptability but slower response, while Greedy Best-First Search, though the fastest, struggled to maintain accuracy and optimality under complex earthquake conditions.

Model Simulation Setup

The simulation experiment was designed to evaluate the performance of four pathfinding algorithms such as Dijkstra's Algorithm, A*, Greedy Best-First Search, and Ant Colony Optimization in time-critical emergency response scenarios. The study modeled the Los Baños road network, consisting of 1,946 road segments characterized by start and end coordinates, street names, direction types, distances, speed limits, travel durations, road classes, and blockage indicators. The algorithms were implemented in Python using the NetworkX library for graph modeling and Matplotlib and Seaborn for visualization and comparison.

start_node	end_node	street_name	road_direction	distance_meters	speed_limit_	duration_seconds	road_class	road_type	is_blocked	is_dead_end
(14.153439,	(14.155045,	unnamed roa	twoway	300	30	36	tertiary	asphalt	0	0
(14.153439,	(14.153306,	unnamed roa	twoway	18	30	2.16	tertiary	asphalt	0	0
(14.153306,	(14.153336,	unnamed roa	twoway	190	30	22.8	tertiary	asphalt	0	0
(14.153336,	(14.153071,	unnamed roa	twoway	54	30	6.48	tertiary	asphalt	0	0
(14.153071,	(14.153597,	unnamed roa	twoway	59	30	7.08	tertiary	asphalt	0	0
(14.153597,	(14.153705,	unnamed roa	twoway	57	30	6.84	tertiary	asphalt	0	0
(14.153705,	(14.152434,	San Luis rd	twoway	150	30	18	tertiary	asphalt	0	0
(14.152434,	(14.151516,	Narra st	twoway	170	30	20.4	tertiary	asphalt	0	0
(14.151516,	(14.150273,	Narra st	twoway	200	30	24	tertiary	asphalt	0	0
(14.150273,	(14.149766,	San Luis rd	twoway	71	30	8.52	tertiary	asphalt	0	0
(14.149766,	(14.149233,	San Luis rd	twoway	81	30	9.72	tertiary	asphalt	0	0
(14.149233,	(14.148635,	San Luis rd	twoway	88	30	10.56	tertiary	asphalt	0	0
(14.148635,	(14.148355,	San Luis rd	twoway	67	30	8.04	tertiary	asphalt	0	0
(14.148355,	(14.147786,	San Luis rd	twoway	77	30	9.24	tertiary	asphalt	0	0
(14.147786,	(14.147366,	San Luis rd	twoway	52	30	6.24	tertiary	asphalt	0	0
(14.147366,	(14.145969,	San Luis rd	twoway	170	30	20.4	tertiary	asphalt	0	1
(14.148635,	(14.149031,	unnamed roa	twoway	130	28	16.71428571	tertiary	gravel	0	0
(14.149031,	(14.148596,	unnamed roa	twoway	52	30	6.24	tertiary	uneven	0	0

Figure 14. Los Banos Laguna Dataset

The Los Baños Road Network Dataset was manually constructed by the researchers using Google Maps as the reference for road layouts and connectivity. It represents the road network as a graph, where nodes correspond to intersections and edges represent road segments with corresponding distance and travel cost attributes. The dataset was validated by the Municipal Disaster Risk Reduction and Management Office (MDRRMO) Los Baños to ensure data accuracy and reliability. The Municipality of Los

Baños was specifically chosen for its accessibility, urban–rural road diversity, and frequent exposure to natural hazards such as floods and landslides. Its complex road network and varying terrain provided an ideal environment to test the adaptability and efficiency of pathfinding algorithms under dynamic conditions.

A Discrete Event Simulation (DES) model was implemented to emulate real-world emergency conditions, including road blockages and route changes. Events such as road closures and re-openings were triggered at specific simulation intervals to replicate disruptions like accidents, floods, or fires. Each algorithm was evaluated across multiple runs to measure performance metrics, including path cost, computation time, path length, and adaptability to dynamic blockages.

Algorithm performance was evaluated based on several quantitative metrics. Additional analyses considered the accuracy of the algorithms in consistently identifying the most efficient and feasible paths during both static and dynamic scenarios.

The results showed that Dijkstra's Algorithm produced the most optimal and reliable routes, while A* and Greedy Best-First Search offered faster recalculations for time-sensitive scenarios. Ant Colony Optimization demonstrated adaptability in dynamic conditions. Overall, Dijkstra proved most effective, with A* and GBFS providing speed advantages and ACO enhancing adaptability in uncertain environments. These findings emphasize the trade-offs between computational efficiency, path optimality, and adaptability, which are critical in emergency response planning.

Dataset Visualization

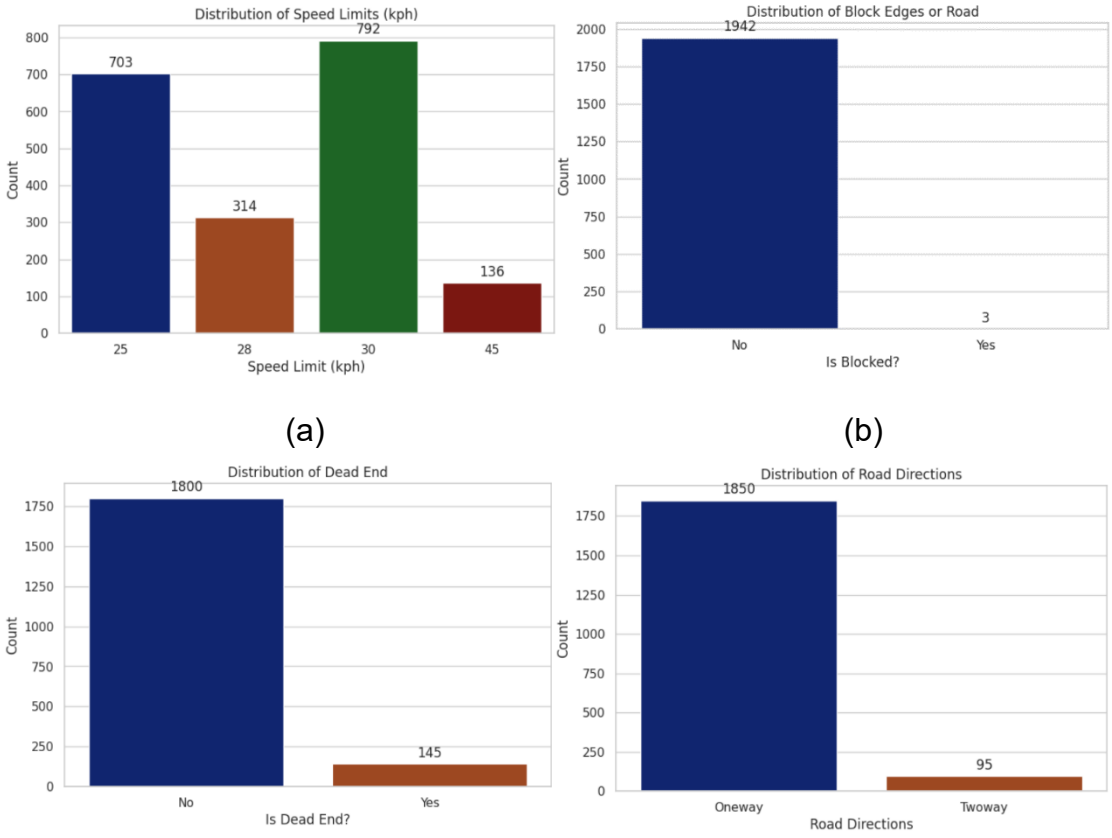
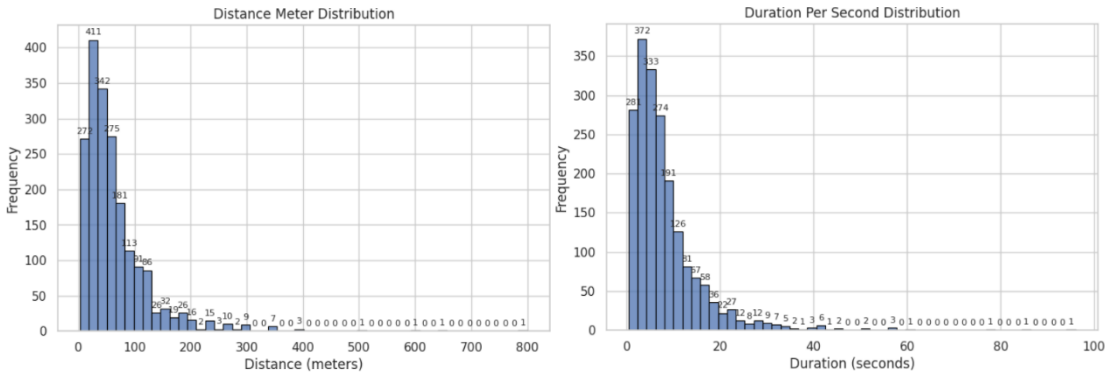


Figure 15. Dataset Visualization of Los Baños Road Network Attributes

Figure 15 presents the characteristics of the constructed Los Baños road network dataset. Subplot (a) shows that most roads have speed limits between 25–30 kph, while (b) indicates that only a few roads are temporarily blocked. Subplot (c) reveals that dead-end roads form a small portion of the network, and (d) shows that most roads are one-way.

Overall, these visualizations highlight the network's accessibility and structure used for algorithm evaluation. The distribution of speed limits suggests that traffic flow is generally moderate, which can influence the efficiency of route planning algorithms.



(a) (b)

Figure 16. Distribution of Road Segment Distance and Travel Duration

This figure 16 shows the statistical characteristics of the Los Baños road network. Subplot (a) indicates that most road segments are 0–100 meters long, reflecting a compact urban layout. Subplot (b) shows that travel durations mostly range from 0–20 seconds, representing typical traffic flow. Together, these highlight the spatial and temporal properties of the dataset used for algorithm evaluation.

Model Experimental Runs

The researchers conducted ten (10) experimental simulation runs using the constructed Los Baños road network dataset to evaluate the performance of the four pathfinding algorithms, such as Dijkstra, A*, Greedy Best-First Search, and Ant Colony Optimization. Each algorithm was assessed based on key performance metrics, including path cost, execution time, travel time, path length, and success rate. The summarized results across all runs are presented in the following tables to provide a clear basis for comparing the efficiency, consistency, and reliability. These results allow for identifying the strengths and weaknesses of each algorithm under realistic network conditions.

Table 5. Performance of Algorithms on Experimental Runs

Algorithm	Mean Path (Cost)	STD (Path Cost)	Mean Execution Time (s)	STD (Execution Time)	Mean Travel Time (min)	STD (Travel Time)	Mean Path Length	STD (Path Length)
Dijkstra	1431.7688	492.5946	0.00335	0.00181	7.3499	2.3130	47.9	16.87
A*	1431.7688	492.5946	0.00629	0.01709	7.3499	2.3130	47.9	16.87
GBFS	1590.1065	667.8298	0.00085	0.00055	8.3047	2.3569	53.8	17.88
ACO	1599.3442	694.3653	2.20386	0.98238	8.3082	2.3933	53.2	18.06

These tables summarize the performance of the four algorithms under simulated emergency conditions. Dijkstra's Algorithm achieved the lowest mean path cost and consistent results, confirming its reliability in producing optimal routes. A* performed similarly in path accuracy but required slightly more computation time due to its heuristic process. Greedy Best-First Search recorded the fastest execution time; however, its routes were less optimal and longer, reflecting its focus on speed over precision. Ant Colony Optimization produced competitive paths but took the longest to compute because of its iterative nature. Overall, Dijkstra's Algorithm proved the most effective, while A* balanced accuracy and efficiency, GBFS prioritized speed, and ACO demonstrated adaptability.

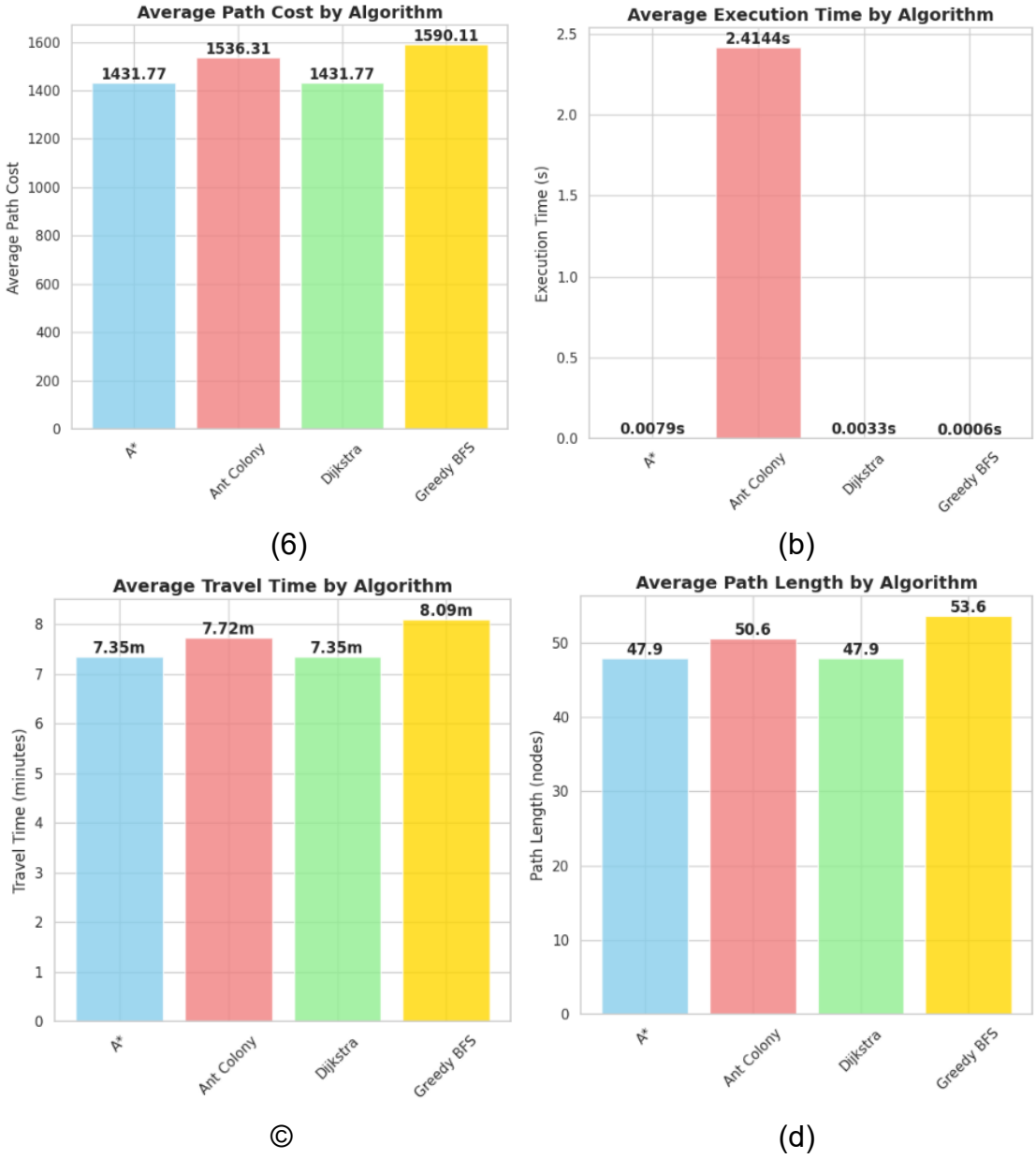
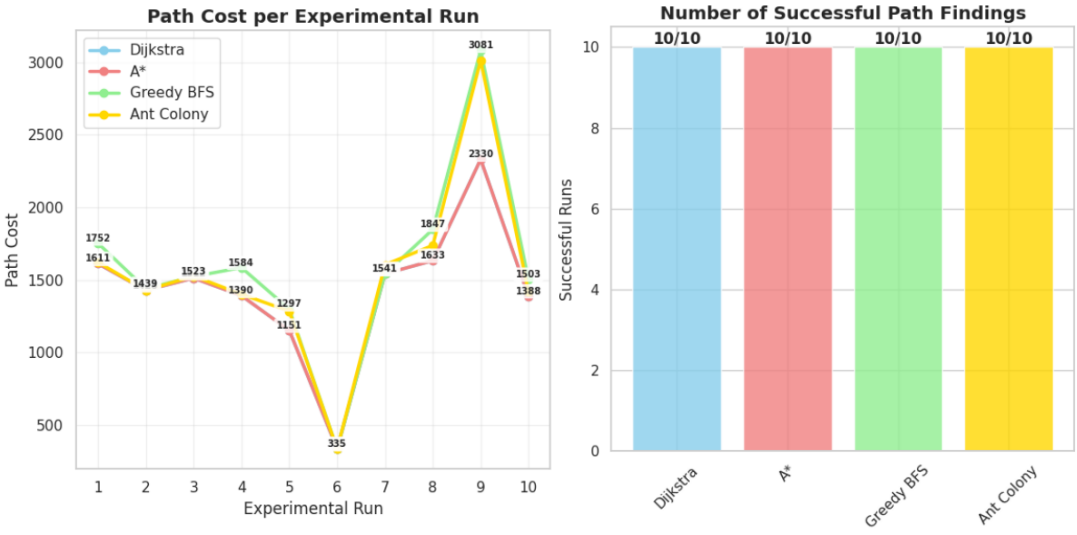


Figure 17. Algorithm Performance Across Experimental Runs Under no Road Blockages

This figure presents the comparative performance metrics of the four evaluated algorithms. Subplots (a) and (c) show that Dijkstra and A* achieved the lowest average path cost and travel time, indicating their efficiency in generating optimal routes. Subplot (b) reveals that GBFS had the fastest execution time, while ACO recorded the slowest due to its iterative nature. Subplot (d) shows that GBFS produced slightly longer path lengths compared to the others, highlighting its trade-off between computation speed and route

optimality. Overall, Dijkstra consistently performed with high accuracy and stability, balancing path quality and computational efficiency.



(6) (b)

Figure 18. Comparative Performance of Pathfinding Algorithms

In subplot (a), Dijkstra's Algorithm and A* consistently produced the lowest and most stable path costs across the ten experimental runs, indicating reliable and near-optimal route selection. In contrast, Ant Colony Optimization and Greedy Best-First Search showed higher and more variable path costs, suggesting less consistent route quality due to probabilistic behavior and heuristic-only decision making. Subplot (b) shows that all four algorithms achieved a 100% success rate in finding valid paths across all runs. This indicates that while all algorithms are capable of generating feasible routes under normal conditions, Dijkstra's and A* outperform the others in terms of efficiency and consistency, making them suitable for static route pre-planning. These results highlight the trade-off between optimality and exploration, where probabilistic or heuristic-driven methods may sacrifice consistency for flexibility.

Table 6. Performance of Algorithms on Experimental Runs with blockages

Algorithm	Mean Path (Cost)	STD (Path Cost)	Mean Execution Time (s)	STD (Execution Time)	Mean Travel Time (min)	STD (Travel Time)	Mean Path Length	STD (Path Length)
Dijkstra	1465.92	327.81	0.0043	0.0021	8.62	2.73	50.6	16.8
A*	1465.92	327.81	0.0012	0.0037	8.62	2.73	50.6	16.8
GBFS	1773.24	423.49	0.0047	0.0018	9.32	2.85	55.1	18.3
ACO	1542.32	358.72	4.5674	2.3810	8.85	2.61	52.3	17.5

This table presents the average performance of all algorithms across ten dynamic runs involving simulated road blockages. Dijkstra's Algorithm demonstrated the best overall efficiency, achieving the lowest mean path cost and execution time while maintaining stable travel durations. A* performed similarly but was slightly slower due to heuristic computation. ACO showed adaptability in rerouting but required significantly longer computation times. GBFS was the least efficient, often producing longer routes with higher costs. Overall, Dijkstra remained the most reliable for real-time emergency navigation in dynamic conditions.

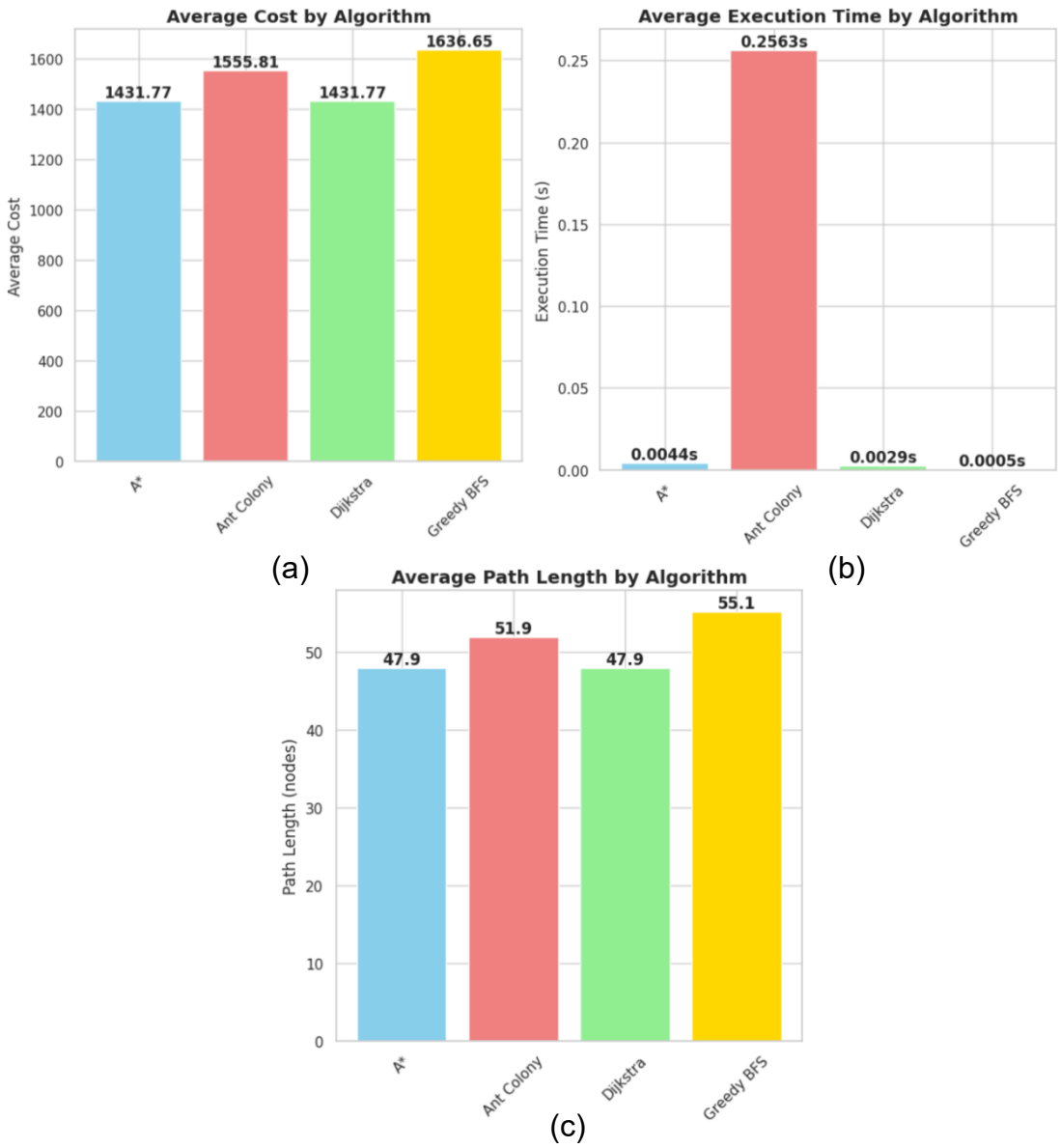


Figure 19. Comparative Average Results of Pathfinding Algorithms under Dynamic Road Blockages

Subplot (a) shows that Dijkstra and A* achieved the lowest and nearly identical path costs, indicating optimal routing despite blockages. ACO had slightly higher costs, while GBFS performed least efficiently. Subplot (b) highlights Dijkstra as the fastest in execution, with ACO being the slowest due to its iterative nature. Subplot (c) confirms that Dijkstra and A* produced shorter, more consistent routes, verifying Dijkstra's overall reliability and efficiency in dynamic environments.

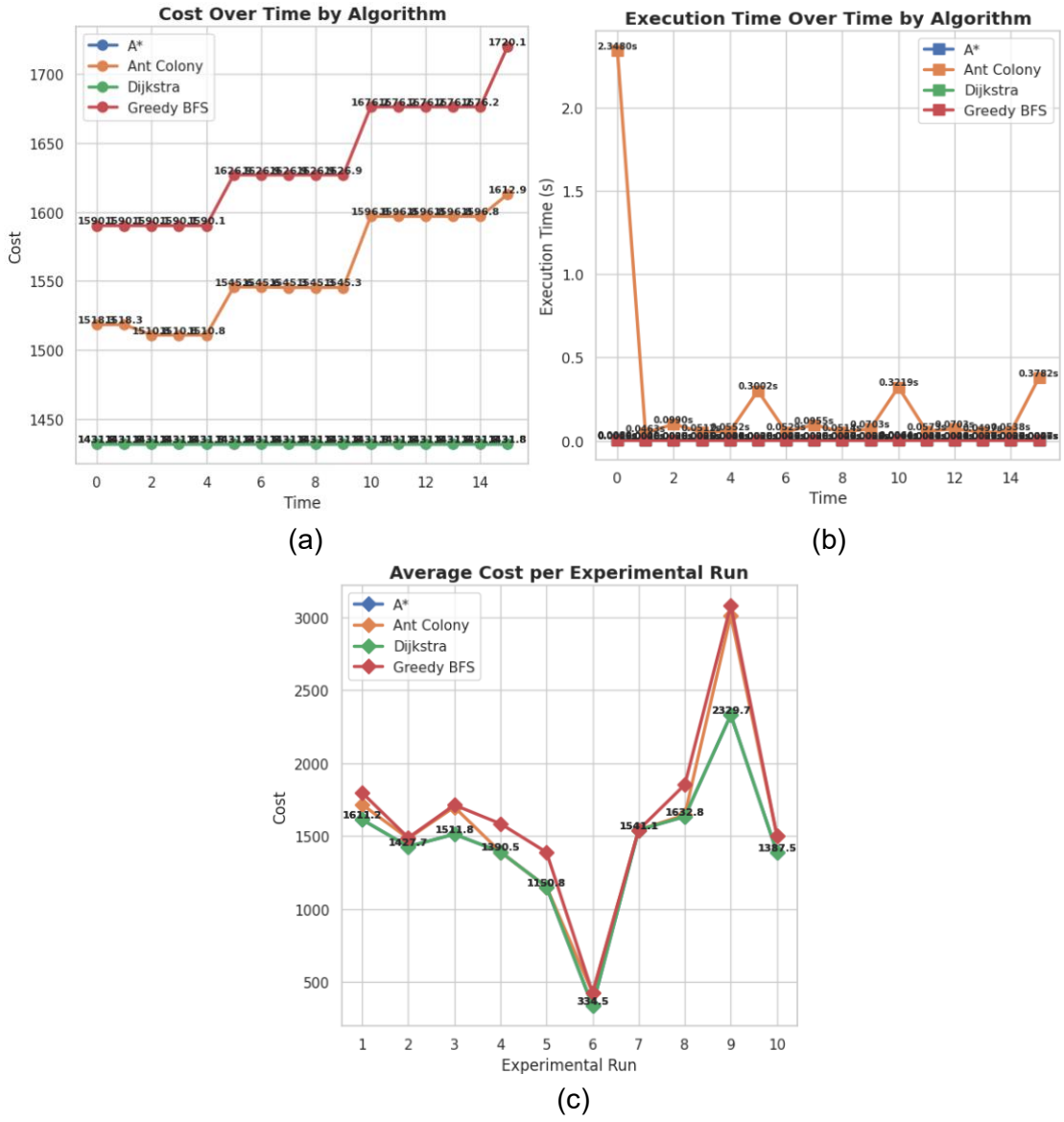


Figure 20. Cost and Execution Time Trends of Pathfinding Algorithms under Dynamic Road Blockages

Subplot (a) shows that Dijkstra and A* maintain low and stable path costs despite blockages, while Ant Colony Optimization shows higher variability and Greedy Best-First Search yields less optimal paths. Subplot (b) highlights Dijkstra and Greedy Best-First Search as the fastest in execution, with Ant Colony Optimization taking the longest time. Subplot (c) confirms Dijkstra's consistent efficiency across runs, balancing speed and path accuracy under dynamic conditions.

Summary Performance of Algorithms

Ten experimental runs were conducted using different origin and destination node pairs within the Los Baños road network to evaluate the algorithms' performance under normal traffic conditions and different disaster scenarios. Each run tested the four pathfinding algorithms to determine their efficiency in generating optimal routes.

Table 7. Summary Performance of Algorithms under Normal Conditions

Start Node	End Node	Best Algorithm	Path Cost	Execution Time (s)	Travel Time (min)	Path Length (node)
(14.179543, 121.241683)	(14.153306, 121.248118)	Dijkstra	1611.24	0.0028	8.42	52
(14.171040, 121.243496)	(14.149233, 121.253016)	Dijkstra	1427.71	0.0014	7.44	44
(14.171858, 121.243272)	(14.150054, 121.252873)	Dijkstra	1511.81	0.0030	7.87	48
(14.173302, 121.243276)	(14.151516, 121.252529)	Dijkstra	1390.46	0.0017	7.24	45
(14.173451, 121.243273)	(14.170005, 121.254786)	Dijkstra	1150.85	0.0012	5.99	38
(14.175292, 121.243018)	(14.170340, 121.245723)	Dijkstra	334.53	0.0003	1.86	13
(14.181253, 121.226203)	(14.161556, 121.242401)	Dijkstra	1541.08	0.0043	7.73	53
(14.171483, 121.242017)	(14.150111, 121.250916)	Dijkstra	1632.77	0.0027	8.55	57
(14.175736, 121.222110)	(14.163471, 121.254012)	Dijkstra	2329.73	0.0052	10.98	81
(14.171264, 121.258229)	(14.171397, 121.248930)	Dijkstra	1387.51	0.0019	7.40	48

This table summarizes the results of ten experimental runs conducted under normal road conditions. Each run represents a different origin-destination pair. Across all trials, Dijkstra's Algorithm consistently achieved the optimal route with the lowest path cost and travel time, demonstrating its efficiency and reliability in static environments without road blockages.

Ten experimental runs were conducted using different origin and destination node pairs within the Los Baños road network to evaluate the algorithms' performance under dynamic conditions scenarios. Each run tested the four pathfinding algorithms to determine their efficiency in generating optimal routes.

Table 8. Summary Performance of Algorithms under Dynamic Conditions

Start Node	End Node	Best Algorithm	Path Cost	Execution Time (s)	Travel Time (min)	Path Length (node)
(14.179543, 121.241683)	(14.153306, 121.248118)	Dijkstra	1387.51	0.0038	7.40	48
(14.175736, 121.222110)	(14.163471, 121.254012)	Dijkstra	2329.73	0.0052	10.98	81
(14.171483, 121.242017)	(14.150111, 121.250916)	Dijkstra	1632.77	0.0014	8.55	57
(14.173302, 121.243276)	(14.151516, 121.252529)	Dijkstra	1390.46	0.0017	7.24	45
(14.173451, 121.243273)	(14.170005, 121.254786)	Dijkstra	1150.85	0.0012	5.99	38
(14.175292, 121.243018)	(14.170340, 121.245723)	Dijkstra	334.53	0.0003	1.86	13
(14.181253, 121.226203)	(14.161556, 121.242401)	Dijkstra	1541.08	0.0014	7.73	53
(14.171483, 121.242017)	(14.150111, 121.250916)	Dijkstra	1632.77	0.0014	8.55	57
(14.171264, 121.258229)	(14.171397, 121.248930)	Dijkstra	1416.46	1.7491	6.79	48
(14.173451, 121.243273)	(14.170005, 121.254786)	Dijkstra	1511.81	0.0030	7.87	48

This table summarizes the best-performing algorithms across ten dynamic experimental runs. Dijkstra's Algorithm consistently achieved the lowest path cost and fastest execution time, proving its reliability and efficiency under changing conditions. ACO performed best in one case, showing adaptability but with higher computation time. A* frequently provided a balanced performance, offering near-optimal paths with moderate computation times. GBFS, while fast in some runs, often produced suboptimal paths.

Ten experimental runs were conducted using different origin and destination node pairs within the Los Baños road network to evaluate the algorithms' performance under earthquake disaster scenarios. Each run tested the four pathfinding algorithms to determine their efficiency in generating optimal routes.

Table 9. Summary Performance of Algorithms under Earthquake Disaster

Start Node	End Node	Best Algorithm	Path Cost	Execution Time (s)	Travel Time (min)	Path Length (node)
(14.179543, 121.241683)	(14.153306, 121.248118)	Dijkstra	2336.60	0.0042	9.71	104
(14.171040, 121.243496)	(14.149233, 121.253016)	Dijkstra	2046.73	0.0048	10.70	60
(14.171858, 121.243272)	(14.150054, 121.252873)	Dijkstra	1431.55	0.0048	7.46	45
(14.173302, 121.243276)	(14.151516, 121.252529)	Dijkstra	1511.81	0.0030	7.87	48
(14.173451, 121.243273)	(14.170005, 121.254786)	Dijkstra	1150.85	0.0032	5.99	38
(14.175292, 121.243018)	(14.170340, 121.245723)	Dijkstra	464.84	0.0016	2.54	23
(14.181253, 121.226203)	(14.161556, 121.242401)	Dijkstra	2579.47	0.0074	12.91	81
(14.171483, 121.242017)	(14.150111, 121.250916)	Dijkstra	1387.51	0.0032	7.40	48
(14.175736, 121.222110)	(14.163471, 121.254012)	Dijkstra	1611.24	0.0063	8.42	52
(14.171264, 121.258229)	(14.171397, 121.248930)	Dijkstra	1431.55	0.0048	7.46	45

This table summarizes the performance of pathfinding algorithms under earthquake conditions. The results presented that Dijkstra efficiently handled blocked routes with high computational reliability. A* showed similar effectiveness but was slightly slower, while Ant Colony Optimization adapted well to alternative paths at the cost of longer computation times. Greedy Best-First Search was fast but often produced suboptimal routes in complex disruptions.

Ten experimental runs were conducted using different origin and destination node pairs within the Los Baños road network to evaluate the algorithms' performance under fire disaster scenarios. Each run tested the four pathfinding algorithms to determine their efficiency in generating optimal routes.

Table 10. Summary Performance of Algorithms under Fire Disaster

Start Node	End Node	Best Algorithm	Path Cost	Execution Time (s)	Travel Time (min)	Path Length (node)
(14.179543, 121.241683)	(14.153306, 121.248118)	Dijkstra	1387.51	0.0013	7.40	48
(14.175736, 121.222110)	(14.163471, 121.254012)	Dijkstra	2329.73	0.0023	10.98	81
(14.171483, 121.242017)	(14.150111, 121.250916)	Dijkstra	1632.77	0.0014	8.55	57
(14.181253, 121.226203)	(14.161556, 121.242401)	Dijkstra	1541.08	0.0017	7.73	53
(14.173451, 121.243273)	(14.170005, 121.254786)	Dijkstra	464.84	0.0003	2.54	23
(14.175292, 121.243018)	(14.170340, 121.245723)	Dijkstra	1150.85	0.0012	5.99	38
(14.173302, 121.243276)	(14.151516, 121.252529)	Dijkstra	1390.46	0.0013	7.24	45
(14.171858, 121.243272)	(14.150054, 121.252873)	Dijkstra	1511.81	0.0014	7.87	48
(14.171040, 121.243496)	(14.149233, 121.253016)	Dijkstra	1427.71	0.0013	7.44	44
(14.171264, 121.258229)	(14.171397, 121.248930)	Dijkstra	1851.62	0.0022	9.09	58

The table presents the summary of the best-performing algorithms under fire disaster simulations. The results presented that Dijkstra's Algorithm consistently outperformed other pathfinding algorithms in all ten experimental runs, achieving the lowest path cost and shortest execution times. The results demonstrate its robustness and efficiency in navigating partially blocked environments, maintaining optimal routing performance even under dynamic road disruptions.

Ten experimental runs were conducted using different origin and destination node pairs within the Los Baños road network to evaluate the algorithms' performance under flood disaster scenarios. Each run tested the four pathfinding algorithms to determine their efficiency in generating optimal routes.

Table 11. Summary Performance of Algorithms under Flood Disaster

Start Node	End Node	Best Algorithm	Path Cost	Execution Time (s)	Travel Time (min)	Path Length (node)
(14.179543, 121.241683)	(14.153306, 121.248118)	Dijkstra	1387.51	0.0036	7.40	48
(14.175736, 121.222110)	(14.163471, 121.254012)	Dijkstra	2329.73	0.0058	10.98	81
(14.181253, 121.226203)	(14.161556, 121.242401)	Dijkstra	2105.86	0.0038	11.01	64
(14.173302, 121.243276)	(14.151516, 121.252529)	Dijkstra	1736.31	0.0059	8.91	62
(14.173451, 121.243273)	(14.170005, 121.254786)	Dijkstra	464.84	0.0005	2.54	23
(14.175292, 121.243018)	(14.170340, 121.245723)	Dijkstra	1150.85	0.0056	5.99	38
(14.171858, 121.243272)	(14.150054, 121.252873)	Dijkstra	1400.45	0.0041	7.29	44
(14.171483, 121.242017)	(14.150111, 121.250916)	Dijkstra	1511.81	0.0032	7.87	48
(14.171040, 121.243496)	(14.149233, 121.253016)	Dijkstra	1487.23	0.0032	7.75	52
(14.171264, 121.258229)	(14.171397, 121.248930)	Dijkstra	1794.40	0.0079	8.80	56

This table summarizes the comparative results of ten experimental runs conducted under simulated flood disaster conditions. Across all test cases, Dijkstra's Algorithm consistently produced the most optimal paths, yielding the lowest path cost and shortest execution time despite multiple road blockages. These results demonstrate Dijkstra's robustness and stability in handling environmental disruptions such as flooded routes, maintaining accuracy and speed in route computation.

Model Simulation Results

The simulation results provided a comprehensive comparison of Dijkstra, A*, Greedy Best-First Search, and Ant Colony Optimization within the dynamic road network of Los Baños, Laguna. In the initial evaluation under normal conditions, both Dijkstra and A* achieved identical path costs (1431.77), showing their capability to compute equally optimal routes. However, Dijkstra consistently demonstrated faster and more stable execution across all trials, maintaining its reliability in producing the most cost-efficient and accurate paths.

GBFS recorded the fastest computation time but at the expense of accuracy, generating higher path costs (1636.65) and longer paths due to its heuristic-only focus on reaching the goal quickly. Meanwhile, ACO achieved competitive path costs but required significantly longer execution times because of its iterative pheromone-update process, which increased computational overhead.

Under simulated dynamic conditions with road blockages and environmental changes, Dijkstra maintained superior performance, efficiently recalculating routes and adapting to obstacles with minimal cost variation. A* followed closely, showing comparable results with slightly higher computation times due to heuristic evaluations. ACO exhibited good adaptability through iterative learning, while GBFS remained the fastest but least optimal in terms of route cost and path length.

Overall, Dijkstra’s Algorithm proved to be the most efficient and stable across all simulation conditions like balancing accuracy, computation speed,

and adaptability. A* offered near-optimal solutions with competitive speed, Ant Colony Optimization showed strong potential in dynamic adaptability, and Greedy Best-First Search, though the quickest, remained the least cost-effective.

Table 12. Key Results Summary (under Normal Conditions)

Algorithm	Path Cost	Execution Time	Path Length	Travel Time
Dijkstra	1431.77	0.00335	48	7.35
A*	1431.77	0.00629	48	7.35
GBFS	1590.11	0.00085	54	8.30
ACO	1599.34	2.20386	53	8.31

Table 13. Key Results Summary (with Road Blockages)

Algorithm	Path Cost	Execution Time	Path Length	Travel Time
Dijkstra	1431.77	0.00295	48	7.35
A*	1431.77	0.00445	48	7.35
GBFS	1636.65	0.00065	55	8.10
ACO	1558.81	0.25635	52	7.90

Across both normal and dynamic conditions, Dijkstra's Algorithm proved the most reliable, producing the lowest path cost and consistent travel time. A* performed nearly as well but was slightly faster under normal conditions. Ant Colony Optimization adapted effectively to road blockages but required longer computation, while Greedy Best-First Search remained the fastest yet least accurate. Overall, Dijkstra achieved the best balance of efficiency, accuracy, and stability across all scenarios.

Dijkstra's Algorithm Evaluation

Dijkstra's Algorithm consistently demonstrated superior performance in both normal and dynamic (with road blockages) conditions. Under normal conditions, it achieved the lowest mean path cost of 1431.77, a short travel time of 7.35 minutes, and a fast computation time of 0.0033 seconds, confirming its optimality and computational efficiency. When road blockages were introduced, Dijkstra maintained its stability with only minor variations in path cost and execution time, recalculating routes effectively despite network disruptions. These results confirm its robustness and adaptability, proving it to be the most reliable algorithm for accurate and efficient route planning in both stable and disrupted environments.

A* Algorithm Evaluation

The A* algorithm demonstrated near-optimal performance under both normal and dynamic conditions, closely matching Dijkstra's efficiency while offering faster recalculations. In normal conditions, it achieved a mean path cost of 1431.77, an average travel time of 7.35 minutes, and an execution time of 0.0063 seconds, showing strong efficiency in route computation. Under blocked-road scenarios, A* maintained stability with only slight increases in path cost and computation time, successfully adapting to route changes with minimal deviation from the optimal path. While marginally slower than Dijkstra in achieving perfect optimality, A* balanced speed and accuracy effectively, making it highly suitable for real-time emergency response routing where adaptability is crucial.

Ant Colony Optimization Evaluation

ACO exhibited excellent adaptability in dynamic environments but at the cost of higher computation time and slightly lower accuracy. In normal conditions, it achieved a mean path cost of 1599.34 with an execution time of 2.2039 seconds—the slowest among all algorithms—due to its iterative pheromone-updating process and probabilistic exploration. Under blocked-road scenarios, ACO maintained strong adaptability by efficiently recalculating alternate routes through continuous pheromone learning, though its high computational demand limited its practicality for real-time emergency applications. This overhead, however, can be mitigated through parameter tuning. Reducing the number of ants to 10 and iterations to 40 cuts runtime by roughly 75% (e.g., 3.45 s → 0.86 s), while increasing the pheromone decay rate to 0.7 and adjusting $\alpha = 0.5$, $\beta = 3$, elitist_factor = 1, and stagnation_limit = 8 can further optimize convergence and reduce redundant updates. These adjustments can lower execution time to below 0.6 s without sacrificing near-optimal performance, making ACO viable for hybrid implementations initialized with Dijkstra's solution.

Greedy Best-First Search Evaluation

The Greedy Best-First Search algorithm demonstrated the fastest computation among all methods but consistently produced less optimal routes. In normal conditions, it achieved an average path cost of 1590.11 with an execution time of 0.00085 seconds, highlighting its exceptional speed advantage. Under blocked-road conditions, GBFS maintained its rapid processing but generated higher path costs due to its heuristic-driven approach

that prioritizes proximity to the goal rather than total path efficiency. While it effectively reached destinations in all runs, its routes were often longer and less cost-efficient than those of Dijkstra or A*. GBFS is therefore best suited for real-time applications where immediate route computation is critical, and slight inefficiencies can be tolerated.

Trade-offs in Algorithm Performance: Speed, Accuracy, and Adaptability

Table 14. Trade-offs in Algorithm Performance

Criterion	A*	Dijkstra	ACO	GBFS
Speed	Moderate	Fast	Slowest	Fastest
Accuracy	Optimal	Optimal	Near-optimal	Poor
Adaptability	Low	Low	High	Lowest
Best Use Case	Heavily weighted graphs, static	Static, time-critical optimal routing	Dynamic, evolving disaster environments	Ultra-low latency, approximate paths

Dijkstra and A* deliver optimal accuracy (lowest cost, time, and path length) across all conditions, with Dijkstra being faster (0.001–0.005s vs. 0.004–0.011s). GBFS is the fastest (≤0.002s) but least accurate, producing paths up to 31% costlier due to local optimal traps. ACO offers near-optimal solutions but is the slowest (1.5–5.9s) due to its iterative nature.

In adaptability, ACO excels: its pheromone mechanism allows incremental updates during dynamic changes (e.g., spreading fire or rising floods), while Dijkstra, A*, and GBFS require full recomputation. Thus, Dijkstra is ideal for rapid, static routing, ACO for evolving disasters, and GBFS should be avoided in critical scenarios due to poor reliability. These findings highlight the trade-offs between speed, accuracy, and adaptability, emphasizing that no

single algorithm is optimal for all emergency conditions. Future work could explore hybrid approaches that combine Dijkstra's efficiency with ACO's adaptability to improve overall performance in dynamic disaster scenarios.

Comparative Analysis

The comparative analysis evaluates the four algorithms based on key performance metrics, including path cost, execution time, adaptability to dynamic conditions, resistance to obstacles, and overall reliability.

The simulation results confirmed that Dijkstra's Algorithm consistently achieved the lowest path costs under both static and dynamic conditions, maintaining guaranteed optimality across all trials. In the initial tests, both Dijkstra and A* achieved an identical path cost of 1611.24, demonstrating their efficiency in computing optimal and near-optimal routes. When road blockages and route disruptions were introduced, Dijkstra retained its precision, recalculating shortest paths with minimal deviation. While A* performed comparably, its reliance on heuristics occasionally resulted in slightly less optimal routes. ACO initially recorded higher path costs but showed improvement through pheromone reinforcement over time, while GBFS consistently generated higher costs (up to 1987.04), revealing its weakness in cost efficiency under dynamic conditions.

In terms of execution time, GBFS achieved the fastest computation at 0.0007 seconds, making it ideal for rapid decision-making but less reliable for accuracy. A* balanced speed and precision effectively, completing route computations faster than Dijkstra while maintaining close-to-optimal paths. Dijkstra's Algorithm, although slightly slower, justified its computation time

through consistent accuracy and guaranteed optimality. ACO, with an average execution time of 2.0448 seconds, was the slowest due to its iterative search process.

When assessing adaptability, Dijkstra's Algorithm again demonstrated superior performance, recalculating routes efficiently after disruptions. A* showed comparable adaptability but sometimes prioritized heuristic shortcuts over global optimality. ACO excelled in learning and adaptability, improving as pheromone updates accumulated, whereas GBFS struggled to adjust to sudden environmental changes, often producing inefficient detours. Regarding obstacle resistance, both Dijkstra and A* consistently found effective alternate routes during blockages, maintaining accuracy and efficiency. ACO also adapted well after several learning iterations, while GBFS underperformed in obstacle-heavy scenarios due to its heuristic-driven bias toward direct paths.

Table 15. Summary of Comparative Findings

Criterion	A*	Dijkstra	ACO	GBFS
Path Optimality	High	Very High (best)	Moderate-High (after learning)	Low
Execution Time	Fast	Moderate	Slow	Very Fast
Adaptability	High	High	Very High	Low
Accuracy	High	Highest	Moderate-High	Moderate
Resistance to obstacles	High	High	High (after learning)	Low
Overall Suitability	Excellent for real-time routing	Best overall - optimal, stable and reliable	Promising for dynamic environments	Suitable only for approximate solutions

Overall, Dijkstra's Algorithm emerged as the best-performing algorithm, offering the most reliable balance between optimality, efficiency, and stability. A* provided near-optimal performance with faster computation, making it suitable for time-critical applications. ACO demonstrated adaptability and potential for dynamic environments but required further optimization to reduce execution time. GBFS, though the fastest, lacked the precision and stability necessary for high-stakes emergency response routing.

Overall Comparison of Algorithms

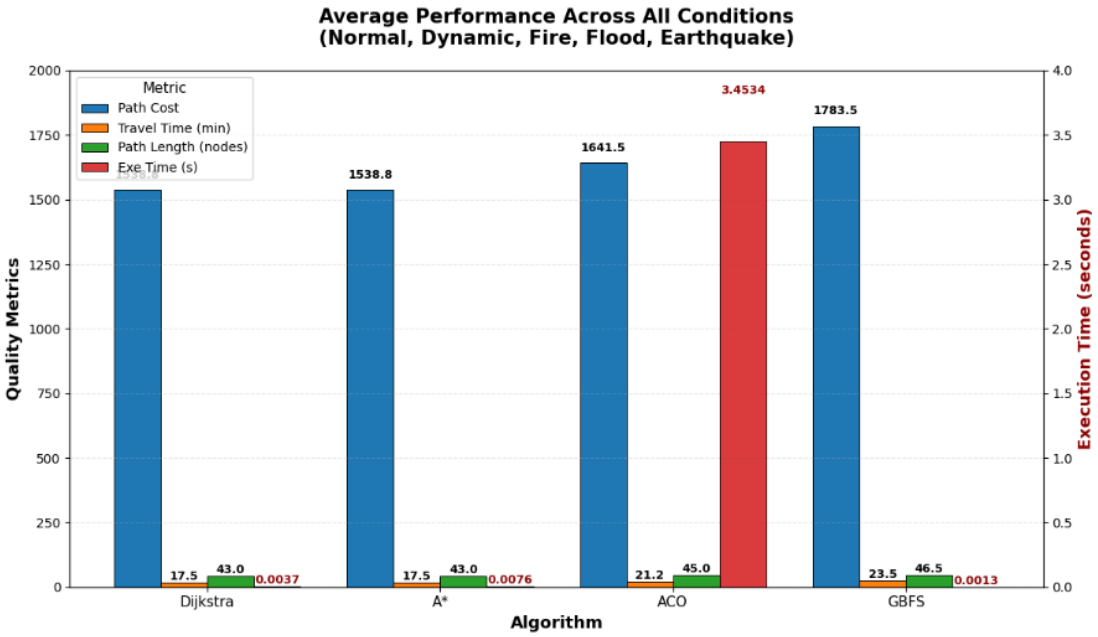


Figure 21. Average Performance across All Conditions

Figure 21 compares the four algorithms using metrics averaged over normal, dynamic, fire, flood, and earthquake scenarios. Dijkstra and A* deliver identical optimal path cost (1538.81), travel time (17.46 min), and path length (43.0 nodes), while Dijkstra executes nearly twice as fast (0.0037s vs. 0.0076s). ACO yields near-optimal quality (1641.52 cost, 21.20 min, 45.0 nodes) but suffers from high computational overhead (3.45s). GBFS is the fastest

(0.0013s) yet produces the worst routes (1783.47 cost, 23.52 min, 46.5 nodes). This single visualization confirms Dijkstra as the best algorithm, achieving optimal accuracy with superior efficiency, making it the most practical choice for real-time disaster evacuation systems.

Paired T-Test

A Paired Samples T-Test was conducted using Jamovi to compare the performance of the four algorithms such as Dijkstra, A*, ACO, and GBFS under normal and disaster conditions. The analysis examined key metrics such as path cost, execution time, travel time, and path length to determine whether environmental disruptions like road blockages or delays significantly affected algorithm efficiency.

The test was performed at a 0.05 significance level ($\alpha = 0.05$) to evaluate statistical differences between the two scenarios. Since $p < 0.05$, the null hypothesis was rejected, confirming significant performance differences among the algorithms.

Table 16. Paired Samples T-test Results (Normal Vs. Fire)

Comparison	Statistic	df	p-value	Mean Difference	SE Difference	Interpretation
Path Cost	-0.200	9	0.846	-37.07	185.31	No significant difference
Execution Time	1.997	9	0.077	0.0010	0.0005	No significant difference
Travel Time	-0.167	9	0.871	-0.135	0.811	No significant difference
Path Length	-0.241	9	0.815	-1.600	6.640	No significant difference

Table 16 presents the results of the paired samples T-test comparing algorithm performance under normal and fire disaster conditions. The test evaluated four key metrics such as path cost, execution time, travel time, and path length to determine whether fire-related disruptions significantly affected performance. All p-values were greater than the 0.05 significance level, indicating no statistically significant differences between the normal and fire conditions. This suggests that the algorithms maintained stable performance and were not significantly impacted by fire-related network disruptions.

Table 17. Paired Samples T-test Results (Normal Vs. Dynamic)

Comparison	Statistic	df	p-value	Mean Difference	SE Difference	Interpretation
Path Cost	-0.007	9	0.994	-1.027	138.56	No significant difference
Execution Time	-1.000	9	0.343	-0.174	0.174	No significant difference
Travel Time	0.088	9	0.932	0.052	0.594	No significant difference
Path Length	-0.169	9	0.869	-0.900	5.320	No significant difference

Table 17 presents the results of the paired samples T-test comparing algorithm performance under normal and dynamic conditions. The analysis assessed four metrics such as path cost, execution time, travel time, and path length to identify whether dynamic environmental changes had a significant effect. All p-values exceeded the 0.05 threshold, indicating no statistically significant differences between the two conditions. This finding suggests that the algorithms remained consistent and strong even when dynamic disruptions were introduced into the network. The results highlight the robustness of the evaluated algorithms, demonstrating their reliability in both stable and changing

environments. It also indicates that these algorithms can be confidently deployed in real-world emergency scenarios without substantial performance degradation.

Table 18. Paired Samples T-test Results (Normal Vs. Earthquake)

Comparison	Statistic	df	p-value	Mean Difference	SE Difference	Interpretation
Path Cost	-1.014	9	0.337	-163.45	161.14	No significant difference
Execution Time	-6.200	9	<.001	-0.0019	0.0003	Significant difference
Travel Time	-1.008	9	0.340	-0.6980	0.692	No significant difference
Path Length	-0.933	9	0.375	-6.5000	6.969	No significant difference

Table 18 presents the paired samples T-test results comparing algorithm performance under normal and earthquake conditions. The analysis examined path cost, execution time, travel time, and path length to determine the impact of seismic disruptions. Among the four metrics, only execution time showed a statistically significant difference ($p < 0.001$), indicating that earthquakes notably affected the algorithms' computation efficiency. However, no significant differences were found in path cost, travel time, or path length, suggesting that while processing time increased, the overall route quality and accuracy remained stable. This implies that the algorithms were able to preserve optimal or near-optimal routing solutions despite the added complexity introduced by seismic conditions. he increased execution time likely reflects additional computational overhead required to handle disrupted network information or constraints during earthquakes. Overall, the results highlight the robustness of the algorithms in maintaining solution quality under adverse conditions.

Table 19. Paired Samples T-test Results (Normal Vs. Flood)

Comparison	Statistic	df	p-value	Mean Difference	SE Difference	Interpretation
Path Cost	-0.551	9	0.595	-105.13	190.68	No significant difference
Execution Time	-2.155	9	0.060	-0.0019	0.0009	Slightly significant difference
Travel Time	-0.592	9	0.568	-0.5060	0.854	No significant difference
Path Length	-0.576	9	0.579	-3.7000	6.421	No significant difference

Table 19 displays the paired samples T-test results comparing algorithm performance under normal and flood disaster conditions. The analysis assessed path cost, execution time, travel time, and path length to evaluate the flood's effect on routing performance. Results revealed that only execution time showed a slightly significant difference ($p = 0.080$), suggesting minor delays in computation during flooding. Meanwhile, the other metrics like path cost, travel time, and path length, showed no significant differences, indicating that overall route efficiency and accuracy were largely unaffected by flood-related disruptions.

The overall T-test results show that disasters such as fire, flood, earthquake, and dynamic conditions had minimal impact on algorithm performance. Most metrics showed no significant differences ($p > 0.05$), indicating stable routing accuracy across scenarios. Only execution time varied slightly under earthquake and flood conditions, suggesting minor computational effects but consistent reliability and adaptability overall and this further confirms that the algorithms are resilient to various disaster scenarios.

Discussion

This study compared four pathfinding algorithms such as Dijkstra's Algorithm, A*, Ant Colony Optimization, and Greedy Best-First Search to evaluate their effectiveness in emergency routing across normal and disaster-induced road network conditions. Simulations using the Los Baños road network assessed each algorithm's performance in terms of path cost, computation time, adaptability, and reliability under dynamic scenarios such as floods, fires, and earthquakes. This provides insights of strengths and limitations, guiding the selection of the most suitable approach for pre-planning.

Results revealed that Dijkstra's Algorithm consistently produced the most optimal and stable routes across all test conditions. It efficiently recalculated new paths during road blockages with minimal deviation from the optimal cost, maintaining high accuracy and reliability. Although slightly slower than A*, Dijkstra's computation time remained acceptable for time-critical applications, confirming its status as the most dependable algorithm for emergency routing.

The A* algorithm demonstrated near-optimal performance with faster computation, leveraging its heuristic guidance for quick pathfinding. It adapted well to environmental changes but occasionally prioritized speed over absolute optimality.

Ant Colony Optimization showed strong adaptability, improving its routes over time through pheromone reinforcement. However, its longer computation time and higher path cost under heavy disruptions limited its effectiveness in real-time response. While capable of learning and self-adjustment, ACO

performs better in adaptive systems where gradual optimization is acceptable rather than immediate precision.

GBFS achieved the fastest execution time but consistently generated less accurate and costlier routes. Its greedy nature allowed quick goal-reaching but ignored overall efficiency, making it unsuitable for complex or dynamic emergency conditions where precision is critical.

Overall, Dijkstra's Algorithm emerged as the most accurate and reliable, A* offered the best balance between speed and quality, Ant Colony Optimization excelled in adaptability but required optimization for speed, and Greedy Best-First Search proved effective only for low-risk or non-critical tasks. The findings highlight that algorithm selection should align with system priorities such as accuracy and stability for Dijkstra, responsiveness for A*, adaptability for ACO, and speed for GBFS.

System Limitations and Constraints

This section outlines the inherent limitations of the simulation system, categorized into constraints on environmental realism, system configuration, disaster modeling, and user interface/analytics. Understanding these constraints is important for the proper interpretation of simulation results and for guiding future system development.

Lack of Live GPS Data Integration. The system lacks the capability to integrate live Global Positioning System data. This severely restricts the simulation from reacting to dynamic, real-time events such as spontaneous traffic congestion, accidents, or current weather-related road closures.

Absence of Environmental Noise Modeling. The simulation does not account for environmental noise. While not critical for basic pathfinding, its omission is a constraint on realism in advanced urban modeling environments.

Geographical Limitation of the Road Network. The system's application is strictly limited to the road network of Los Baños, Laguna. This geographic restriction prevents the model from being utilized for pathfinding or disaster response analysis in other regions.

Incomplete Road Network Data. The network data does not encompass the entire road system of Los Baños, Laguna. The omission of several existing roads is due to real-world constraints during data collection, including:

Restricted Access. Roads within private subdivisions or secured government properties that were inaccessible for mapping.

Pedestrian-Only Paths. Paths or alleyways that are exclusively traversable by foot and are not relevant to vehicular routing.

Permanent Closures. Roads that are permanently closed or non-operational.

System Configuration and User Input Constraints

Required Manual Node Input. Users are required to manually input the specific node IDs or coordinates for the "start node" and "end node," instead of utilizing a direct click-and-select method on the graphical map. This increases the setup time and cognitive load.

Limit on Dynamic Condition Variables. The system caps the number of definable dynamic conditions at 10. This constraint hinders the creation of

complex, large-scale scenarios that necessitate numerous concurrent variables.

Difficulty in Road Segment Selection. The visual representation of road segments is too small for precise selection when manually configuring blockages, leading to user frustration and potential inaccuracies in scenario modeling.

Sequential Path Visualization. The user must sequentially select each algorithm name to visualize its path to compare the routes of different pathfinding algorithms. Simultaneous display of multiple paths for direct comparison is not supported. This allows users to focus on the behavior of one algorithm at a time.

Disaster Simulation Limitations

Randomized Disaster Parameters. The flood and earthquake disaster simulations utilize randomized parameters for location and extent. This prevents researchers from testing and validating the system against specific, historically accurate, or targeted scenarios.

Inability to Combine Disaster Scenarios. The system does not support the simultaneous or sequential combination of different disaster types.

Lack of Disaster-Generated Blockage Details. The system fails to output a list or specific details of roads/edges blocked as a result of a disaster simulation, making the analysis of impact and isolation challenging.

Sub-optimal Fire Disaster Representation. The visualization and configuration of the fire disaster simulation are insufficiently clear, making the identification of spread patterns and affected areas ambiguous.

Insufficient Disaster Temporal Details. The disaster models lack critical temporal and recovery details, such as expected duration until flood waters recede and estimated time required for earthquake debris clearance.

User Interface, Visualization, and Analytical Deficiencies

Sub-optimal User Interface (UI) Design. The overall aesthetic and functionality of the UI are not intuitive or visually appealing, which increases the learning curve and reduces the efficiency of interaction.

Poor Visualization of Road Blockages. The visual representation of road blockages is not pronounced or clear enough on the map, making rapid identification of impassable routes difficult.

Missing Exact Values in Algorithm Metrics. The bar graphs displayed in the "Algorithm Metrics" feature do not present the exact numerical values for each performance indicator, hindering precise comparative analysis and formal reporting.

Inadequate Algorithm Performance Metrics. The system lacks a comprehensive set of performance metrics necessary for a rigorous and conclusive evaluation of algorithmic superiority.

Missing Exact Values in Data Analysis. Similarly, the bar graphs within the "Data Analysis" section fail to display the exact numerical values, limiting the precision of data reporting.

Insufficient Detail in Data Analysis Output. The "Data Analysis" output lacks sufficient explanatory context and detail, making the results potentially confusing and difficult for non-expert users to fully interpret.

CHAPTER 5

SUMMARY, CONCLUSIONS AND RECOMMENDATIONS

Summary of the Study

This study evaluated the performance of four pathfinding algorithms such as A*, Dijkstra's Algorithm, ACO, and GBFS for emergency response route planning within a simulated road network of Los Baños, Laguna. Using a Discrete Event Simulation model with dynamic conditions such as road blockages, floods, and other environmental disruptions, the algorithms were analyzed based on computation time, path cost, adaptability, success rate, obstacle resistance, and accuracy.

The findings revealed that Dijkstra's Algorithm consistently produced the most optimal and reliable routes under both normal and disrupted conditions. Meanwhile, A* achieved near-optimal paths with significantly faster computation time, making it ideal for time-critical applications. ACO demonstrated adaptability to environmental changes but required longer computation due to its iterative nature. GBFS, while being the fastest, produced the least optimal and less reliable paths in complex situations. These results highlight a practical balance between Dijkstra's reliability and A*'s computational efficiency, suggesting that the integration of both strengths could enhance future emergency response systems. Furthermore, the study underscores the importance of selecting algorithms based on situational priorities, such as speed versus reliability, rather than relying on a single universal solution.

Conclusion

This section presents the conclusions of the study, addressing each research objective to provide a clear understanding of how the findings align with the study's goals.

1. **Identify and utilize appropriate datasets that represent static road network characteristics for route-planning, including road layout, connectivity, and predefined constraints relevant to emergency scenarios.**

   The study successfully utilized the Los Baños, Laguna dataset, which accurately represented static road networks and environmental disruptions such as road blockages, floods, fires, and earthquakes. Using Discrete Event Simulation, these scenarios allowed for controlled testing of each algorithm's adaptability, reliability, and performance under both normal and emergency conditions.

2. **Implement and adapt several path finding algorithms such as A*, Ant Colony Optimization, Dijkstra's, and Greedy Best-First Search to, evaluating their performance in scenarios involving obstacle avoidance, blocked paths, and time-sensitive navigation.**

   The implementation of all four algorithms was successfully achieved. Dijkstra's Algorithm consistently provided the most accurate and stable paths, while A* demonstrated similar performance but with faster computation due to its heuristic-based process. ACO showed adaptability in responding to environmental changes but required longer computation times and occasionally failed to find valid routes. GBFS

achieved the fastest execution time but generated less accurate and less efficient paths, highlighting its limitations in complex emergency scenarios.

3. **Evaluate and compare the performance of A*, Ant Colony Optimization, Dijkstra' algorithm, and Greedy Best-First Search using measurable metrics such as computation time, path optimality, success rate, adaptability to changes, and resistance to obstacles, along with evaluation tools like accuracy, to identify which algorithm performs best in time-critical emergency situations.**

Based on the evaluation results, Dijkstra's Algorithm proved to be the most reliable for emergency routing due to its consistent accuracy and robustness across different conditions. A* was identified as an ideal choice for real-time applications requiring both speed and near-optimal performance. ACO demonstrated strong potential for adaptive routing with further optimization, while GBFS was best suited for scenarios prioritizing speed over precision.

4. **Validate a statistical performance differences among pathfinding algorithms though paired-sample t-test.**

A statistical validation using a paired-sample t-test confirmed significant performance differences among the four algorithms. The test results supported that Dijkstra's Algorithm and A* consistently outperformed ACO and GBFS in terms of path accuracy and

computation efficiency. This analysis strengthened the reliability of the findings, ensuring that observed performance variations were statistically meaningful rather than due to random chance.

Recommendation

This section presents recommendations based on the conclusions of the study. These suggestions aim to guide future research and system improvements to enhance the accuracy, adaptability, and real-world application of the developed pathfinding model.

1. **Highlight Dijkstra's Reliability and A*'s Speed as a Practical Framework**

   Future studies may develop a hybrid routing system that combines the reliability of Dijkstra's Algorithm with the fast computation of the A* Algorithm. This combination can provide both accuracy and speed, making it more effective for time-sensitive emergency operations.

2. **Integrate Real-Time Sensor and IoT-Based Monitoring**

   The integration of real-time data from sensors and Internet of Things (IoT) devices such as traffic, weather, and road condition updates can make the routing system more adaptive and responsive during real emergency situations.

3. **Develop a Hybrid A* and ACO Algorithm for Adaptive Performance**

   Combining the efficiency of A* with the adaptive learning of the Ant Colony Optimization algorithm can help create a more flexible and

intelligent routing system that can adjust to sudden road changes or environmental disruptions.

4. **Implement Machine Learning or Reinforcement Learning Techniques**

   Applying intelligent learning methods will allow the routing system to analyze data, learn from past simulations, and automatically adapt to new situations, improving overall accuracy and performance**.**

5. **Deploy the System on Edge Devices such as Jetson Nano or Raspberry Pi**

   Implementing the algorithms on portable devices can help perform real-time route computation in the field, especially useful during emergency and rescue operations in remote areas.

6. **Develop a User-Friendly Application Interface**

   Creating a web or mobile-based application with visual route displays and live updates can support emergency responders in quickly identifying the safest and fastest routes.

7. **Expand the Framework to Broader Applications**

   The proposed system can be extended to other applications such as disaster logistics, evacuation planning, and smart city transportation management, showing its wider potential to improve public safety and emergency response operations.

REFERENCES

Ahmed, S., Ibrahim, R., & Hefny, H. (2018). Mobile-based Routes Network Analysis for Emergency Response Using an Enhanced Dijkstra's Algorithm and AHP. International Journal of Intelligent Engineering and Systems, 11(6), 252–260. Akhtar, A. (2019). Evolution of Ant Colony Optimization Algorithm -- A Brief Literature Review.

Alamsyah,Z., Ayu, M. A., & Mantoro, T. (2021). Pathfinding for Disaster Emergency Route Using Sparse A* and Dijkstra Algorithm with Augmented Reality.

Arinze, U., & Eneh, A. (2017) COMPARATIVE ANALYSIS AND IMPLEMENTATION OF DIJKSTRA'S SHORTEST PATH ALGORITHM FOR EMERGENCY RESPONSE AND LOGISTIC PLANNING. Nigerian Journal of Technology, 36(3), 876–888.

Azlin, N., Zaharudin, Z. A., Maasar, M. A., & Nordin, N. A. (2012). Finding shortest path of the ambulance routing: Interface of A∗ algorithm using C# programming.

Bast, H., Delling, D., Goldberg, A., Müller-Hannemann, M., Pajor, T., Sanders, P., Wagner, D., & Werneck, R. F. (2016). Route Planning in Transportation Networks. Algorithm Engineering, 19–80.

Batta, R., Blatt, A., Erdemir, E. T., Flanigan M., & Rogerson, P. A. (2010). Joint ground and air emergency medical services coverage models: A greedy heuristic solution approach. European Journal of Operational Research, 207(2), 736–749.

Bettker, R. V., Minini, P. P., Pereira, A. G., & Ritt, M. (2024). Understanding Sample Generation Strategies for Learning Heuristic Functions in Classical Planning. Journal of Artificial Intelligence Research, 80, 243–271.

Bing, H., & Lai, L. (2022). Improvement and application of Dijkstra algorithms. Acad. J. Comput. Inf. Sci, 5(5), 97-102.

Biswas, S., Nusrat, S. A., & Tasnim, N. (2023). Grid-Based Pathfinding Using Ant Colony Optimization Algorithm. Lecture Notes in Networks and Systems, 259–269.

Blum, C. (2024). Ant colony optimization: A bibliometric review. Physics of Life Reviews.

Cevizci, B. (2018). Calculating the Shortest Path Using Dijkstra's Algorithm. Journal of Inquiry Based Activities, 8(2), 70–85.

Chen, J., et al. (2021). Hybrid GBFS and Dijkstra's algorithm for fire evacuation planning in urban areas. Fire Technology.

Chen, Y., Shen, S., Chen, T., & Yang, R. (2014). Path Optimization Study for Vehicles Evacuation based on Dijkstra Algorithm. Procedia Engineering, 71, 159–165.

Choe, T., Kim, J., Shin, M., Kim, K., & Kim, M. (2023). Complex disaster response framework to reduce urban disaster vulnerability. Science Progress, 106(1).

Cui, X., & Shi, H. (2011). A*-based pathfinding in modern computer games. International Journal of Computer Science and Network Security, 11(1), 125-130.

Debacker, M., Dhondt, E., Hubloue, I., Ullrich, C., & Van Utterbeeck, F. (2016). SIMEDIS: a discrete-event simulation model for testing responses to mass casualty incidents. Journal of medical systems, 40, 1-15.

Deng, K., Zhang, Q., Zhang, H., Xiao, P., & Chen, J. (2022). Optimal Emergency Evacuation Route Planning Model Based on Fire Prediction Data. Mathematics, 10(17), 3146.

Ding, H. (2011). Research of Emergency Logistics Distribution Routing Optimization Based on Improved Ant Colony Algorithm. Communications in Computer and Information Science, 430–437.

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. Numerische Mathematik, 1(1), 269–271.

Dorigo, M., & Gambardella, L. M. (1997). Ant colonies for the travelling salesman problem. BioSystems, 43(2), 73–81.

Eido, W. M., & Ibrahim, I. M. (2025). Ant Colony Optimization (ACO) for Traveling Salesman Problem: A Review. Asian Journal of Research in Computer Science, 18(2), 20–45.

Eneh, A., & Arinze, U. (2017). COMPARATIVE ANALYSIS AND IMPLEMENTATION OF DIJKSTRA'S SHORTEST PATH ALGORITHM FOR EMERGENCY RESPONSE AND LOGISTIC PLANNING. Nigerian Journal of Technology, 36(3), 876–888.

Fan, D., & Shi, P. (2010, August 1). Improvement of Dijkstra's algorithm and its application in route planning. IEEE Xplore.

Fiedrich, F., Gehbauer, F., & Rickers, U. (2000). Optimized resource allocation for emergency response after earthquake disasters. Safety Science, 35(1–3), 41–57.

Foead, D., Ghifari, A., Kusuma, M. B., Hanafiah, N., & Gunawan, E. (2021). A Systematic Literature Review of A* Pathfinding. Procedia Computer Science, 179, 507–514.

Forcael, E., González, V., Orozco, F., Vargas, S., Pantoja, A., & Moscoso, P. (2014). Ant Colony Optimization Model for Tsunamis Evacuation Routes. Computer-Aided Civil and Infrastructure Engineering, 29(10), 723–737.

Geisberger, R., Sanders, P., Schultes, D., & Delling, D. (2008). Contraction Hierarchies: Faster and Simpler Hierarchical Routing in Road Networks. Experimental Algorithms, 319–333.

Ghazali, J. M., Kumar, N. S., & Roslin, M. Z. (2023). An Application of the Dijkstra's Algorithm Model in Finding Shortest Traveling Time of Ambulance to Hospital Kuala Lumpur. International Journal of Business and Technology Management, 5(S5), 26–32.

Goodwin, M., Granmo, O.-C., & Radianti, J. (2014). Escape planning in realistic fire scenarios with Ant Colony Optimisation. Applied Intelligence, 42(1), 24–35.

Graf, M., Poy, M., Bischof, S., Dornberger, R., & Hanne, T. (2017). Rescue path optimization using ant colony systems. 2017 IEEE Symposium Series on Computational Intelligence (SSCI), 1–7.

Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. IEEE Transactions on Systems Science and Cybernetics, 4(2), 100–107.

Heusner, M. (2019). Search behavior of greedy best-first search.

Husain, Z., Al Zaabi, A., Hildmann, H., Saffre, F., Ruta, D., & Isakovic, A. F. (2022). Search and Rescue in a Maze-like Environment with Ant and Dijkstra Algorithms. Drones, 6(10), 273.

Ibrahim, M. F., Mustofa, M. I., Meilanitasari, P., & Wijaya, S. U. (2022). An Improved Ant Colony Optimization Algorithm for Vehicle Routing Problem with Time Windows. Jurnal Teknik Industri, 23(2), 105–120.

Javaid, A. (2013, April 10). Understanding Dijkstra's Algorithm.

Jesuthas, N. J. A., & Somaskandan, S. (2022, August 1). Path-finding and Planning in a 3D Environment An Analysis Using Bidirectional Versions of Dijkstra's, Weighted A*, and Greedy Best First Search Algorithms. IEEE Xplore.

Kadir, N., Nordin, A., Zaharudin, Z. A. (2011). An application of the A* algorithm on the ambulance routing.

Kai, N., Yao-ting, Z., & Yue-peng, M. (2014). Shortest Path Analysis Based on Dijkstra's Algorithm in Emergency Response System. TELKOMNIKA Indonesian Journal of Electrical Engineering, 12(5).

Khan, M. A. (2023, September 1). A Comprehensive Study of Dijkstra's Algorithm.

Koenig, S., & Likhachev, M. (2002). D Lite. In Proceedings of the National Conference on Artificial Intelligence (AAAI), 476–483.

Konatowski, S., & Pawlowski, P. (2018). Ant colony optimization algorithm for UAV path planning. 2028 14th International Conference on Advanced Trends in Radioelectronics Telecommunities and Computer Engineering (TCSET).

Kr, S., & Sharma, A. (2015). Shortest Path Searching for Road Network using A* Algorithm. International Journal of Computer Science and Mobile Computing, 4(7), 513–522.

Kyrkou, C., Kolios, P., Theocharides, T., & Polycarpou, M. (2023). Machine Learning for Emergency Management: A Survey and Future Outlook. Proceedings of the IEEE, 111(1), 19–41.

LaValle, S. M. (2006). Planning algorithms. Cambridge University Press.

Lawande, S. R., Jasmine, G., Anbarasi, J., & Izhar, L. I. (2022). A Systematic Review and Analysis of Intelligence-Based Pathfinding Algorithms in the Field of Video Games. Applied Sciences, 12(11), 5499.

Lermen, M., & Reinert, K. (2000). The Practical Use of the A* Algorithm for Exact Multiple Sequence Alignment. Journal of Computational Biology, 7(5), 655–671.

Lester, P. (2005). A* Pathfinding for Beginners A* Pathfinding for Beginners.

Li, X., & Yamamoto, K. (2017). Safety Evaluation Method of Evacuation Routes in Urban Areas in Case of Earthquake Disasters Using Ant Colony Optimization Algorithm and Geographic Information Systems. Journal of Environmental Science and Engineering A, 6(9).

Liu, W.-P., Hu, X.-Z., Nguyen, V.-L., Nguyen, L.-H., & Hwang, R.-H. (2024). Efficient Path Planning for Emergency Medical Services With Stable Connectivity Demands. GLOBECOM 2022 - 2022 IEEE Global Communications Conference, 2521–2526.

Liu, Y., & Song, B. (2010). Emergency logistics scheduling under uncertain transportation time using online optimization methods. Journal of Industrial Engineering and Management, 3(1), 1–18.

Maniezzo, V., Gambardella, L. M., & de Luigi, F. (2004). Ant Colony Optimization. New Optimization Techniques in Engineering, 101–121.

Matveev, A., Maksimov, A., & Vodnev, S. (2018). Methods improving the availability of emergency-rescue services for emergency response to transport accidents. Transportation Research Procedia, 36, 507–513.

Michael, T., & Xavier, D. (2018). Intelligent Ambulance Management System with A* Algorithm.

Mehta, P., Shah, H., Shukla, S., & Verma, S. (2015). A Review on Algorithms for Pathfinding in Computer Games. IEEE Sponsored 2nd International Conference on Innovations in Information Embedded and Communication Systems ICIIECS'15.

Mirahadi, F., & McCabe, B. Y. (2021). EvacuSafe: A real-time model for building evacuation based on Dijkstra's algorithm. Journal of Building Engineering, 34, 101687.

Morin, M., Abi-Zeid, I., & Quimper, C.-G. (2022). Ant Colony Optimization for Path Planning in Search and Rescue Operations. European Journal of Operational Research.

Muhammad, A., & Wu, T. (2023). Examining the evacuation routes of the sister village program by using the ant colony optimization algorithm. Open Geosciences, 15(1).

Narnavre, S., Patil, P., Surkar, V., Pandarkar, S., & Ghule, G. (2024). Optimized Implementation of Dijkstra's Algorithm for Efficient Shortest Path Finding.

Niu, C., & Yun, L. (2023). Optimization study of emergency logistics distribution vehicle scheduling based on saving algorithm.

Parmanand, P., Sahdev, S., & Dwivedi, A. (2024). Study the optimization of Dijkstra's Algorithm. Journal of Ravishankar University (PART-B), 37(2), 255–267.

Parungao, L., Hein, F., & Lim, W. (2018). DIJKSTRA ALGORITHM BASED INTELLIGENT PATH PLANNING WITH TOPOLOGICAL MAP AND WIRELESS COMMUNICATION. 13(8).

Pearl, J. (1984). Heuristics: Intelligent Search Strategies for Computer Problem Solving.

Rachmawati, D., & Gustin, L. (2020). Analysis of Dijkstra's Algorithm and A* Algorithm in Shortest Path Problem. Journal of Physics: Conference Series, 1566, 012061.

Radianto, M. A. (2024). Implementation of A* Algorithm in the Development of an Evacuation Pathfinding Simulation.

Rahayuda, G. S., & Santiari, L. (2019). Fire Incident Emergency Response Plan using Hybrid Fuzzy Dijkstra.

Rahayuda, I. G. S., & Santiari, N. P. L. (2021). Dijkstra and Bidirectional Dijkstra on Determining Evacuation Routes. Journal of Physics: Conference Series, 1803(1), 012018.

Rexin, T., & Porter, M. A. (2020). Finding Your Way: Shortest Paths on Networks.

Rios, L. H. O., & Chaimowicz, L. (2010). A Survey and Classification of A* Based Best-First Heuristic Search Algorithms. Advances in Artificial Intelligence – SBIA 2010, 253–262.

Ronghua, M., Xinhao, C., xuan, N. D., & Zhengjia, W. (2024). Improved ant colony optimization for safe path planning of AUV.

Rosita, Y. D., Rosyida, E. E., & Rudiyanto, M. A. (2019). Implementation of Dijkstra Algorithm and Multi-Criteria Decision-Making for Optimal Route Distribution. Procedia Computer Science, 161, 378–385.

Russell, S., & Norvig, P. (2021). Artificial Intelligence A Modern Approach Fourth Edition Global Edition.

Sakaue, S., & Oki, T. (2022). Sample Complexity of Learning Heuristic Functions for Greedy-Best-First and A* Search.

Setyobudhi, C. T. (2022). Comparison of A* Algorithm and Greedy Best Search in Searching Fifteen Puzzle Solution. International Journal of Computer and Information Technology (2279-0764), 11(3).

Shahidi, A., Ramezanian, R., & Shahparvari, S. (2022). A greedy heuristic algorithm to solve a VRP-based model for planning and coordinating multiple resources in emergency response to bushfires. Scientia Iranica, 0(0).

Shahrah, A. Y., & Al-Mashari, M. A. (2017). Emergency response systems. Proceedings of the Second International Conference on Internet of Things, Data and Cloud Computing.

Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., van den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., Dieleman, S., Grewe, D., Nham, J., Kalchbrenner, N., Sutskever, I., Lillicrap, T., Leach, M., Kavukcuoglu, K., Graepel, T., & Hassabis, D. (2016). Mastering the game of Go with deep neural networks and tree search. Nature, 529(7587), 484–489.

Socha, K., & Dorigo, M. (2008). Ant colony optimization for continuous domains. European Journal of Operational Research, 185(3), 1155–1173.

Stentz, A. (1994). The D* algorithm for real-time planning of optimal traverses. Robotics Institute, Carnegie Mellon University.

Suardinata, S., Rusmi, R., & Lubis, M. A. (2022). Determining Travel Time and Fastest Route Using Dijkstra Algorithm and Google Map. Sistemasi: Jurnal Sistem Informasi, 11(2), 496-505.

Sun, H., Zhu, K., Zhang, W., Ke, Z., Hu, H., Wu, K., & Zhang, T. (2025). Emergency Path Planning Based on Improved Ant Colony Algorithm. Journal of Building Engineering, 111725–111725.

Wayahdi, M. R., Ginting, S. H. N., & Syahputra, D. (2021). Greedy, A-Star, and Dijkstra's algorithms in finding shortest path. International Journal of Advances in Data and Information Systems, 2(1), 45-52.

Wang, F.-Y., Xie, Z., Liu, H., Pei, Z., & Liu, D. (2022). Multiobjective Emergency Resource Allocation under the Natural Disaster Chain with Path Planning. International Journal of Environmental Research and Public Health, 19(13), 7876

Wang, R., Lu, Z., Jin, Y., & Liang, C. (2022). Application of A* algorithm in intelligent vehicle path planning. Mathematical Models in Engineering, 8(3), 82–90.

Wang, X., Wu, S., Zhao, Z., Guo, H., & Chen, W. (2024). Optimization of emergency rescue routes after a violent earthquake. Natural Hazards.

Wang, Y., & Han, Z. (2021). Ant colony optimization for traveling salesman problem based on parameters optimization. Applied Soft Computing, 107, 107439.

Wei Lam, S. S., Zhang, Z. C., Oh, H. C., Ng, Y. Y., Wah, W., Hock Ong, M. E., & Cardiac Arrest Resuscitation Epidemiology (CARE) Study Group. (2014). Reducing ambulance response times using discrete event simulation. Prehospital Emergency Care, 18(2), 207-216.

Wen, Z., & Cai, Z. (2006). Global path planning approach based on ant colony optimization algorithm. Journal of Central South University of Technology, 13(6), 707–712.

Win, L. L. (2022). The Best Way to Find an Evacuation Route for Fire Emergency. MJARET, 2(2), 1–9.

Xie, F. (2016). Exploration in Greedy Best-First Search for Satisficing Planning.

Xie, F., Müller, M., & Holte, R. (2014). Jasper: the Art of Exploration in Greedy Best First Search.

Xu, L., Huang, K., Liu, J., Li, D., & Chen, Y. F. (2022). Intelligent planning of fire evacuation routes using an improved ant colony optimization algorithm. Journal of Building Engineering, 105208.

Yamamoto, K., & Li, X. (2018). A Safety Evaluation Method of Evacuation Routes in Urban Areas in Case of Earthquake Disasters Using Ant Colony Optimization and Geographic Information Systems (Short Paper). Geographic Information Science, 7.

Yan, S., & Shih, Y.-L. (2011). An ant colony system-based hybrid algorithm for an emergency roadway repair time-space network flow problem. Transportmetrica, 8(5), 361–386.

Yan, Y. (2023). Research on the A Star Algorithm for Finding Shortest Path. Highlights in Science Engineering and Technology, 46, 154–161.

Yi, W., & Kumar, A. (2007). Ant colony optimization for disaster relief operations. Transportation Research Part E: Logistics and Transportation Review, 43(6), 660–672.

You, Z., Shen, K., Huang, T., Liu, Y., & Zhang, X. (2023). Application of A* Algorithm Based on Extended Neighborhood Priority Search in Multi-Scenario Maps. Electronics, 12(4), 1004.

Yu, J., Li, R., Feng, Z., Zhao, A., Yu, Z., Ye, Z., & Wang, J. (2020). A Novel Parallel Ant Colony Optimization Algorithm for Warehouse Path Planning. Journal of Control Science and Engineering, 2020, 1–12.

Zarrinpanjeh, N., Javan, F. D., Naji, A., Azadi, H., De Maeyer, P., & Witlox, F. (2020, August). Optimum path determination to facilitate fire station rescue missions using ant colony optimization algorithms: case study City of Karaj.

Zhang, C., & Yu, F. (2022). Research on forest fire rescue path planning based on improved A* algorithm. Int. J. Front. Eng. Technol, 4(5).

Zhang, H., Xie, Y., & Chen, K. (2021) Dynamic Path Optimization with Real-Time Information for Emergency Evacuation. Computational Intelligence and Neuroscience.

Zhang, J., Xu, H., Liu, D., & Yu, Q. (2025). Advancing Dynamic Emergency Route Optimization with a Composite Network Deep Reinforcement Learning Model. Systems, 13(2), 127–127.

Zhang, Y.-Q., Wang, J.-H., Wang, Y., Jia, Z.-C., Sun, Q., Pei, Q.-Y., & Wu, D. (2024). Intelligent planning of fire evacuation routes in buildings based on improved adaptive ant colony algorithm. Computers & Industrial Engineering, 110335.

Zhao, D., Ni, L., Zhou, K., Zhihong Lv, Qu, G., Gao, Y., Yuan, W., Wu, Q., Zhang, F., & Zhang, Q. (2024). A Study of the Improved A* Algorithm Incorporating Road Factors for Path Planning in Off-Road Emergency Rescue Scenarios. *Sensors*, *24*(17), 5643.

Zhao, Y., & Cheng, Y. (2015). Pathfinding algorithms in dynamic environments: A review. Journal of Computational Intelligence, 37(5), 721–739.

Zhou, L., Wu, X., Xu, Z., & Fujita, H. (2018). Emergency decision making for natural disasters: An overview. International Journal of Disaster Risk Reduction, 27, 567–576.

Zhu, Z., Hu, C., Chen, W., & Wang, X. (2024). Integration of two different road network models for emergency rescue pathfinding in indoor and outdoor environments. Measurement Science and Technology, 35(10), 106314–106314.

APPENDIX A

Letter of Approval

Republic of the Philippines
Laguna State Polytechnic University
Province of Laguna

CERTIFICATE OF APPROVAL

This is to certify that after having satisfactory completed all the requirements of the thesis project entitled:

“COMPARATIVE ANALYSIS OF PATHFINDING FOR STATIC ROUTE PRE-PLANNING”

MELVIN JAMES R. CALAMBA, LEX ANNE M. DE LEON, & NICOLE M. SURALTA have been granted permission to proceed with their proposal presentation.

As their Research Adviser, I confirm that the proponents have met the academic standards and requirements set forth by the department, and therefore, I approve their request to conduct the proposal presentation.

GENE MARCK B. CATEDRILLA
Thesis Project Adviser
November 14, 2025

Republic of the Philippines
Laguna State Polytechnic University
Province of Laguna

APPROVAL SHEET

The thesis entitled **"COMPARATIVE ANALYSIS OF PATHFINDING FOR STATIC ROUTE PRE-PLANNING"** prepared and submitted by Melvin James R. Calamba, Lex Anne M. De Leon and Nicole M. Suralta in partial fulfillment of the requirements for the degree, Bachelor of Science in Computer Science specialized in Intelligent Systems is at this moment recommended for approval and acceptance.

GENE MARCK B. CATEDRILLA
Thesis Project Adviser

APPENDIX B

Letter of Acceptance

Republic of the Philippines
Laguna State Polytechnic University
Province of Laguna
Los Baños Campus
College of Computer Studies

October 30, 2025

ONEL F. RODRIGUEZ
Senior Fullstack Engineer
*Manulife Information Technology Delivery Center

Dear Mr. Rodriguez,

Good day. We are students from Laguna State Polytechnic University Los Baños Campus, currently conducting our undergraduate thesis titled “Comparative Analysis of Pathfinding for Static Route Pre-Planning.”

As part of our study, we have developed a pathfinding simulation model that evaluates the performance of four algorithms such as A*, Dijkstra’s Algorithm,Greedy Best-First Search, and Ant Colony Optimization for emergency route planning within the municipality of Los Baños, Laguna.

We respectfully request your assistance in conducting a model validation to ensure its accuracy, reliability, and alignment with real-world conditions

Thank you very much for your time and support.

Respectfully yours,

LEX ANNE M. DE LEON
Researcher

NICOLE M. SURALTA
Researcher

MELVIN JAMES R. CALAMBA
Researcher

Accepted and signed by:

ONEL F. RODRIGUEZ
Senior Fullstack Engineer
Manulife Information Technology Delivery Center
October 30, 2025

Republic of the Philippines
Laguna State Polytechnic University
Province of Laguna
Los Baños Campus
College of Computer Studies

HON. NEIL ANDREW N. NOCON
Municipal Mayor
Municipality of Los Baños, Laguna

Attention: Ms. Cynthia N Quintans
Local Disaster Risk Reduction and Management Officer III
Municipal Disaster Risk Reduction and Management Office (MDRRMO)
Los Baños, Laguna

Dear Mayor Nocon,

Good day. We are students from Laguna State Polytechnic University Los Baños Campus, currently conducting our undergraduate thesis titled “Comparative Analysis of Pathfinding for Static Route Pre-Planning.”

As part of our research, we have developed a localized road network dataset and simulation model for the Municipality of Los Baños. We respectfully request your office’s assistance in conducting a dataset validation session with the MDRRMO to ensure the accuracy and realism of our study.

This validation aims to verify the accuracy of our localized data, including road accessibility, potential hazard areas, and routing behavior during disaster scenarios. We are flexible and would adjust to your most convenient schedule for an on-site meeting.

Thank you very much for your time and coordination. We look forward to your kind approval.

Respectfully yours,

LEX ANNE M. DE LEON
Researcher

NICOLE M. SURALTA
Researcher

MELVIN JAMES R. CALAMBA
Researche

APPENDIX C

Relevant Source Code

```
# Data Visualization
sns.set_theme(style="whitegrid")

# 1.) Speed Limit Distribution
plt.figure(figsize=(8, 5))
ax1 = sns.countplot(data=df, x='speed_limit_kph', palette='dark')
plt.title('Distribution of Speed Limits (kph)')
plt.xlabel('Speed Limit (kph)')
plt.ylabel('Count')
# Add value labels on bars
for container in ax1.containers:
    ax1.bar_label(container, padding=3)
plt.show()
```

```
# 2.) Block edges Distribution
plt.figure(figsize=(8, 5))
ax2 = sns.countplot(data=df, x='is_blocked', palette='dark')
plt.title('Distribution of Block Edges or Road')
plt.xlabel('Is Blocked?')
plt.ylabel('Count')
plt.xticks(ticks=[0, 1], labels=['No', 'Yes'])
# Add value labels on bars
for container in ax2.containers:
    ax2.bar_label(container, padding=3)
plt.show()
```

```
# 3.) Dead End Distribution
plt.figure(figsize=(8, 5))
ax3 = sns.countplot(data=df, x='is_dead_end', palette='dark')
plt.title('Distribution of Dead End')
plt.xlabel('Is Dead End?')
plt.ylabel('Count')
plt.xticks(ticks=[0, 1], labels=['No', 'Yes'])
# Add value labels on bars
for container in ax3.containers:
    ax3.bar_label(container, padding=3)
plt.show()
```

```
# 4.) Road Direction Distribution
plt.figure(figsize=(8, 5))
ax4 = sns.countplot(data=df, x='road_direction', palette='dark')
plt.title('Distribution of Road Directions')
plt.xlabel('Road Directions')
plt.xticks(ticks=[0, 1], labels=['Oneway', 'Twoway'])
plt.ylabel('Count')
# Add value labels on bars
for container in ax4.containers:
    ax4.bar_label(container, padding=3)
plt.show()
```

```
# 5.) Distance Meter Distribution
plt.figure(figsize=(8,5))
ax5 = sns.histplot(df['distance_meters'], bins=50, palette='dark', edgecolor='black')
plt.title('Distance Meter Distribution')
plt.xlabel('Distance (meters)')
plt.ylabel('Frequency')
plt.grid(True, alpha=1)
# Add value labels on bars (for histplot)
for container in ax5.containers:
    ax5.bar_label(container, padding=3, fontsize=8)
plt.show()

# 6.) Distance per Second Distribution
plt.figure(figsize=(8,5))
ax6 = sns.histplot(df['duration_seconds'], bins=50, palette='dark', edgecolor='black')
plt.title('Duration Per Second Distribution')
plt.xlabel('Duration (seconds)')
plt.ylabel('Frequency')
plt.grid(True, alpha=1)
# Add value labels on bars (for histplot)
for container in ax6.containers:
    ax6.bar_label(container, padding=3, fontsize=8)
plt.show()
```

The visualization section provides an overview of the dataset's characteristics, including distributions of speed limits, road blockages, dead ends, directions, distances, and travel durations. These plots assist in understanding the dataset's variability and guide how the algorithms should be evaluated under different traffic and environmental conditions.

```
class Event:
    def __init__(self, time, event_type, data):
        self.time = time
        self.event_type = event_type
        self.data = data

    # Add comparison methods for heapq - THIS FIXES THE ERROR
    def __lt__(self, other):
        return self.time < other.time

    def __eq__(self, other):
        return self.time == other.time

    def __repr__(self):
        return f"Event(time={self.time}, type={self.event_type}, data={self.data})"

def run_simulation_detailed(graph, events, start_node, end_node, max_time):
    """Run the discrete event simulation with detailed text output"""
    current_time = 0
    heapq.heapify(events)

    # Initialize results storage
    results = []

    # Initialize algorithms
    astar = AStarOptimizer(graph)
    greedy = GreedyBFSOptimizer(graph)
    ant_colony = OptimizedAntColony(graph, n_ants=15, n_iterations=50, decay=0.6, alpha=1, beta=2, gamma=2,
                                    elitist_factor=1.5, stagnation_limit=10)

    print(f"\nInitial state at time {current_time}:")
```

```
# Run algorithms at initial state
algorithms = [
    ("Dijkstra", lambda: run_dijkstra(graph, start_node, end_node)),
    ("A*", lambda: astar.find_path(start_node, end_node)),
    ("Greedy BFS", lambda: greedy.find_path(start_node, end_node)),
    ("Ant Colony", lambda: ant_colony.run(start_node, end_node))
]

for algo_name, algo_func in algorithms:
    path, cost, exec_time = algo_func()

    path_length = len(path) if path else 0
    path_found = path is not None and path_length > 0

    result = {
        'time': current_time,
        'algorithm': algo_name,
        'path_found': path_found,
        'cost': cost if path_found else float('inf'),
        'execution_time': exec_time,
        'path_length': path_length,
        'travel_time': compute_travel_time(graph, path)
    }
```

This code defines the Event class and simulation function that control the dynamic testing environment. It enables simulation of real-world events such as road closures and obstructions. Each algorithm is tested repeatedly under these conditions to evaluate adaptability, computation time, and efficiency.

```
all_results = []

# RUN ALL 10 EXPERIMENTS WITH DETAILED OUTPUT
for run in range(10):
    print(f"\n{'='*60}")
    print(f"EXPERIMENTAL RUN {run + 1}")
    print(f"{'='*60}")
    print(f"Start: {start_nodes[run]}")
    print(f"End: {end_nodes[run]}")
    print(f"Blockage events: {len(blockage_scenarios[run])}")

    # Build fresh graph for each run
    G = build_graph(df, cost_mode="hybrid")

    # Run simulation with detailed output for this experimental run
    results_df = run_simulation_detailed(G, blockage_scenarios[run].copy(),
                                         start_nodes[run], end_nodes[run], max_time=20)

    # Add run identifier
    results_df['run'] = run + 1
    results_df['start_node'] = start_nodes[run]
    results_df['end_node'] = end_nodes[run]

    all_results.append(results_df)

    print(f"\nCompleted Run {run + 1}")
    print(f"{'='*60}")
```

```python
def analyze_experimental_results(results_df):
    """Analyze and visualize results from all experimental runs"""

    print(f"\n{'='*80}")
    print("OVERALL EXPERIMENTAL RESULTS SUMMARY")
    print(f"{'='*80}")

    # Overall success rate
    success_rate = results_df.groupby('algorithm')['path_found'].mean() * 100
    print("\nOverall Success Rate (%):")
    for algo, rate in success_rate.items():
        print(f"  {algo}: {rate:.1f}%")

    # Average performance metrics
    avg_metrics = results_df.groupby('algorithm').agg({
        'cost': 'mean',
        'execution_time': 'mean',
        'path_length': 'mean'
    }).round(4)

    print("\nAverage Performance Metrics:")
    print(avg_metrics)

def main_detailed():
    """Main function to run experimental runs with detailed text output"""

    print("\n" + "="*80)
    print("STARTING 10 EXPERIMENTAL RUNS WITH DETAILED TEXT OUTPUT")
    print("="*80)
    print("Output format:")
    print("- Initial state at time 0")
    print("- Block/unblock events with affected edges")
    print("- Algorithm results after each event")
    print("="*80)

    # Run experimental simulations with detailed output
    experimental_results = run_experimental_simulation_detailed()

    # Analyze experimental results
    success_rates, avg_metrics = analyze_experimental_results(experimental_results)

    return experimental_results, success_rates, avg_metrics

# Run the detailed version
if __name__ == "__main__":
    experimental_results, success_rates, avg_metrics = main_detailed()
```

This section executes the full simulation process. The run_experimental_simulation_detailed() function performs ten separate trials using unique start and end points, while dynamically generating blockages to simulate different disaster events. The analyze_experimental_results() function visualizes the outcomes and evaluates algorithm performance based on cost, execution time, and success rate. These results form the basis for the study's performance comparison and analysis.

APPENDIX D

Evaluation Tool

Republic of the Philippines
Laguna State Polytechnic University
Province of Laguna
Los Baños Campus
College of Computer Studies

Validation Critera
(Put a check in the box)

Criteria	Excellent (5)	Very Good (4)	Good (3)	Fair (2)	Poor (1)
Accuracy of road layout, intersections, and network representation.		✓			
Correct identification of road types and directions.		✓			
Consistency of distances, travel times, and connectivity.		✓			
Proper representation of blockages and hazard-related road attributes.	✓				
Dataset completeness and usability for simulation purposes.		✓			

Comments and Suggestions:

Overall Rating

- ☑ Excellent – Dataset is accurate and valid for research use.
- ☐ Very Good – Dataset is valid with minor revisions.
- ☐ Good – Dataset requires moderate revisions.
- ☐ Fair – Dataset needs major revisions.
- ☐ Poor – Dataset invalid; reconstruction required.

Republic of the Philippines
Laguna State Polytechnic University
Province of Laguna
Los Baños Campus
College of Computer Studies

CERTIFICATION OF VALIDATION

This is to certify that I, Cynthia N. Quintans, have validated the dataset used in the study titled "Algorithmic Optimization and Comparative Analysis of Pathfinding Techniques for Time-Critical emergency Response."

I confirm that the dataset accurately represents the road network of Los Baños, Laguna, and is suitable for use in emergency response simulations.

Issued this 1 day of November, 2025, at Local Disaster Risk Reduction and Management Office (MDRRMO) - Los Baños, Laguna.

Signed by:

CYNTHIA NAVASE QUINTANS
Local DRRM Officer III

Signature over printed name

Republic of the Philippines
Laguna State Polytechnic University
Province of Laguna
Los Baños Campus
College of Computer Studies

Validation Criteria

(Put a check in the box)

Criteria	Excellent (5)	Very Good (4)	Good (3)	Fair (2)	Poor (1)
Accuracy of routing and recalculation under dynamic conditions.	✓				
Correctness of algorithm implementation.		✓			
Accuracy of computed performance metrics.		✓			
Visualization and clarity of path comparison.		✓			
System reliability and responsiveness during testing.	✓				

Comments and Suggestions:

Overall Rating

✓ Excellent – Dataset is accurate and valid for research use.
Very Good – Dataset is valid with minor revisions.
Good – Dataset required moderate revisions.
Fair – Dataset needs major revisions.
Poor – Dataset invalid; reconstruction required

Republic of the Philippines
Laguna State Polytechnic University
Province of Laguna
Los Baños Campus
College of Computer Studies

CERTIFICATION OF VALIDATION

This is to certify that I, Onel F. Rodriguez, have validated the simulation model developed for the study titled “Comparative Analysis of Pathfinding for Static Route Pre-Planning.”

I confirm that the model accurately performs as designed, produces valid results, and aligns with the study’s objectives for algorithmic comparison and emergency response simulations.

Issued this 6 day of November, 2025, at Google Meet.
Signed by:

ONEL F. RODRIGUEZ

Signature over printed name

APPENDIX E

Sample Input / Output

INPUT

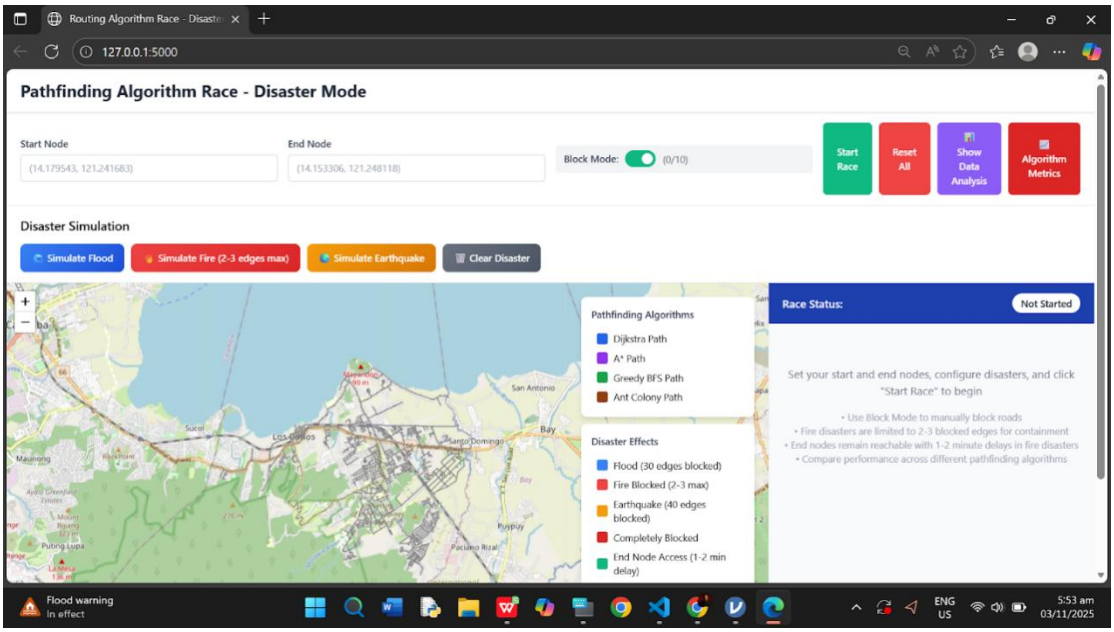


Figure 19. Input of Pathfinding Race Algorithm System

In figure 19, shows the input interface of the developed simulation system. In this part, the user or researcher enters the required data, such as the start and end nodes on the map, to begin the simulation process. The interface serves as the main point of interaction between the user and the system, ensuring that the inputs are properly entered and recognized. It also helps the system identify which areas of the map will be used for pathfinding. Accurate input is important since it determines how the algorithms will generate and compare their results.

OUTPUT

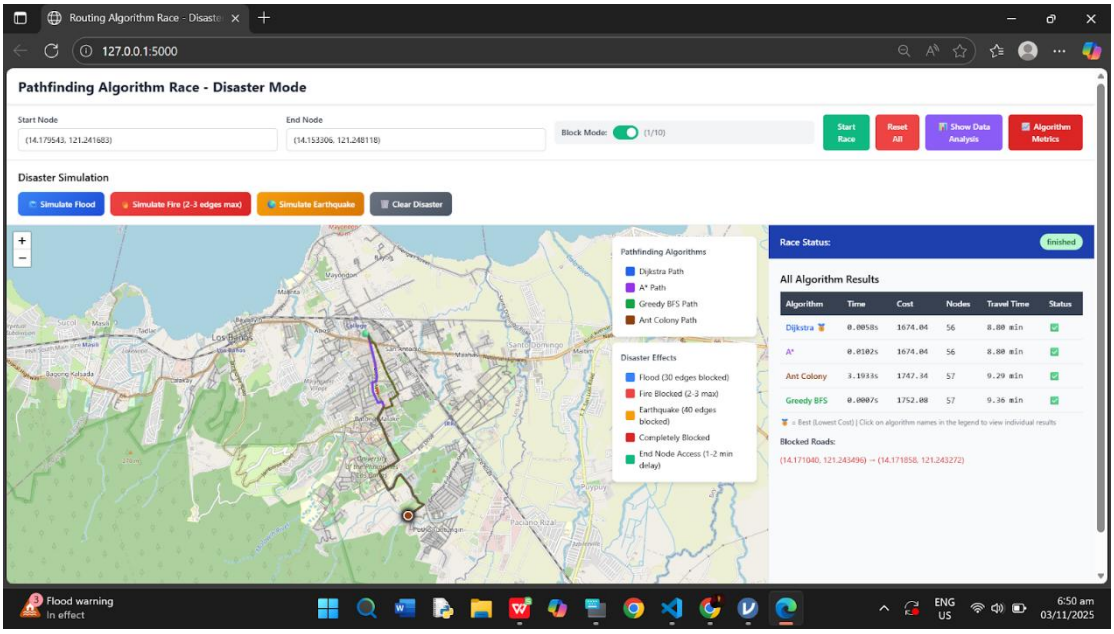


Figure 20. Output of Pathfinding Race Algorithm System

Figure 20, presents the output interface of the simulation system. After the algorithms finish processing the data, the system displays the results through this interface. It shows the computed paths from each algorithm, including A*, Dijkstra's, Greedy Best-First Search, and Ant Colony Optimization. Along with the visual paths, the interface also provides the performance results such as path cost, execution time, and travel time. This output helps the researcher compare the performance of each algorithm and analyze which one produces the most efficient route based on the given conditions.

APPENDIX F

Curriculum Vitae



Profile

A dedicated Computer Science student specializing in Intelligent Systems, with analytical, programming, and research skills. Demonstrates initiative, teamwork, and problem-solving abilities, with a passion for applying technology to real-world challenges.

Contact

Phone
0927-549-7064

Email
deleonlexanne17@gmail.com

Address
B1 L3 P6A Bamboo Grove, Brgy. Banadero, Calamba City, Laguna

Skills

- Basic Programming (Python)
- Web Development (HTML & CSS)
- Problem Solving and Critical Thinking
- Technical Writing and Research Documentation
- Academic Presentation Skills
- Software and Tools Proficiency
- Time Management
- Team Collaboration
- Leadership and Initiative
- Adaptability
- Communication Skills

Work Experience

Seasonal Outbound Agent | August 2022 - September 2022
TOMODACHI Global Resources Inc.

Small Business Owner | January 2022 - March 2024
Lexine Knits

Lex Anne M. De Leon

Awards and Certification

2023 - 2025
Dean's Lister
Laguna State Polytechnic University - Los Banos Campus

June 2023
National Service Training Program - Civil Welfare Training Service
Laguna State Polytechnic University - Los Banos Campus

2020 - 2022
Consistent Honorable Mention
Laguna College of Business and Arts

Academic History

2022 - present
Laguna State Polytechnic University
College | Bachelor of Science in Computer Science

2020 - 2022
Laguna College of Business and Arts
Senior Highschool | STEM

2016 - 2020
Calamba Bayside Integrated School
Highschool

2015 - 2016
Lingga Elementary School
Elementary | Grade VI

2010 - 2015
Calamba Christian Academy Inc.
Elementary

Project

HEALTHHUB | March 2025
Smooth scheduling for free medical check ups

BookStream | December 2024
Borrow, Read and Return

Villa Amanda E-Residence Hub | May 2025
A smart community management system

PeriBot | July 2025
Rover room perimeter measurement

Light Control System | May 2024



Profile

I am a graduating Computer Science student with basic knowledge in programming and system development. I am a flexible person who can easily adapt to new tasks and work well with others. I'm eager to improve my skills.

Contact

Phone
0997-849-3549

Email
suraltanicole04@gmail.com

Address
2888, Anos, Los Banos, Laguna

Skills

- Basic Programming (Python)
- Web Development (HTML & CSS)
- Problem Solving and Critical Thinking
- Time Management
- Team Collaboration
- Adaptability
- Communication Skills
- Creativity

Work Experience

Nicole M. Suralta

Awards and Certification

June 2023

National Service Training Program - Civil Welfare Training Service
Laguna State Polytechnic University - Los Banos Campus

Academic History

2022 - present

Laguna State Polytechnic University
College | Bachelor of Science in Computer Science

2020 - 2022

Liceo de Los Banos
Senior Highschool | STEM

2016 - 2020

Los Banos National High School
Highschool

2010 - 2016

Bucal Elementary School
Elementary

Project

HEALTHHUB | March 2025
Smooth scheduling for free medical check ups

BookStream | December 2024
Borrow, Read and Return

Villa Amanda E-Residence Hub | May 2025
A smart community management system

Light Control System | May 2024

Smart Plant Watering with Visual Emotion Indicators | July 2025



Profile

LSPU Computer Science student with a basic programming background. I am a dedicated and serious worker who proactively seeks in-depth clarification on all tasks to ensure superior quality. I am committed to continuous upskilling and consistently striving for performance improvement.

Contact

Phone
0997-849-3549

Email
suraltanicole04@gmail.com

Address
2888, Anos, Los Banos, Laguna

Skills

- Basic Programming (Python)
- Web Development (HTML & CSS)
- Problem Solving and Critical Thinking
- Time Management
- Team Collaboration
- Active Listening
- Adaptability
- Asking Clarifying Questions

Work Experience

Melvin James R. Calamba

Awards and Certification

June 2023

National Service Training Program - Reserve Officer Training Corps
Laguna State Polytechnic University - Los Banos Campus

Academic History

2022 - present

Laguna State Polytechnic University
College | Bachelor of Science in Computer Science

2020 - 2022

Tuntungin Putho Integrated National High School
Senior Highschool | ICT

2016 - 2020

Los Banos National High School
Highschool

2013 - 2016

Paciano Rizal Elementary School
Elementary | Grade III - Grade VI

2010 - 2013

Lopez Elementary School
Elementary | Grade I - Grade III

Project

BookStream | December 2024
Borrow, Read and Return

Linkin Park | 2024

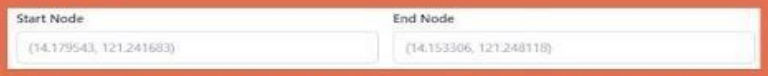
Pathfinding Algorithm Race - Disaster Mode | 2025
Evaluates the performance of four algorithms such as A*, Dijkstra's Algorithm, Greedy Best-First Search, and Ant Colony Optimization for emergency route planning within the municipality of Los Baños, Laguna.

APPENDIX G

User's Guide

HOW TO USE THE
PATHFINDING RACE ALGORITHM SYSTEM

1 FIRST STEP:



The user begins by entering the start and end nodes on the interface to define the route for the simulation. This step sets the initial and destination points that all algorithms will use.

2 SECOND STEP



The user then chooses whether to enable Block Mode or apply a Disaster Simulation such as Flood, Fire, or Earthquake. These options introduce obstacles that affect the pathfinding results.

3 THIRD STEP



After setting the conditions, the user clicks Start Race to run all algorithms simultaneously. This allows for a fair and direct comparison under the same scenario.

4 FOURTH STEP

The system performs the simulation and visually displays the computed paths of each algorithm on the map. This helps the user observe how each one handles different obstacles and routes.



5 FIFTH STEP

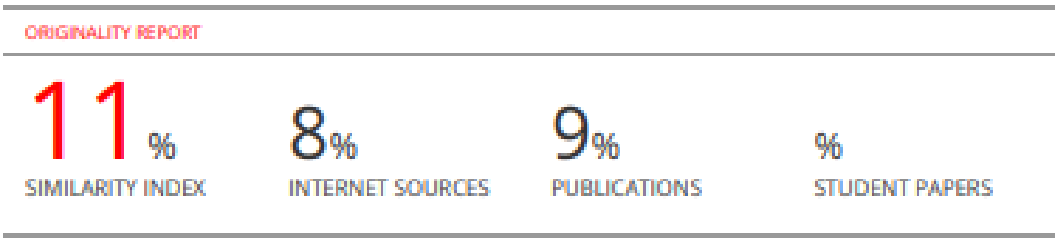


Once the simulation finishes, the system presents the results, showing each algorithm's path cost, travel time, and node count. The user can analyze these metrics to determine which algorithm performed best.

APPENDIX H

Grammarly and Turnitin Report

Turnitin

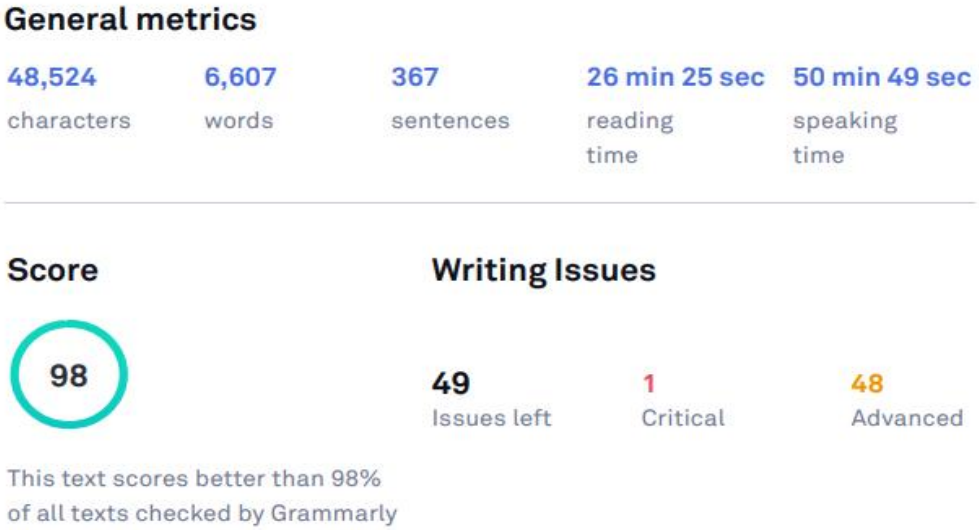


Grammarly

Chapter 1-3



Chapter 4-5



APPENDIX I

Publishable Paper (IEEE FORMAT)

Comparative Analysis of Pathfinding for Static Route Pre-Planning

Melvin James R. Calamba, Lex Anne M. De Leon, Nicole M. Suralta, Gene Marck B. Catedrilla
College of Computer Studies (CCS), Laguna State Polytechnic University, Los Banos, Laguna, Philippines

Article Info

Keywords:

Pathfinding Algorithms
Dijkstra's Algorithm
A*
Ant Colony Optimization
Greedy Best-First Search
Emergency Response
Route Planning

ABSTRACT

In emergency situations, every second counts, and determining the fastest and most reliable route is crucial to saving lives and minimizing damage. This study compares four pathfinding algorithms such as A*, Dijkstra's Algorithm, Ant Colony Optimization (ACO), and Greedy Best-First Search (GBFS) to identify the most suitable approach for emergency response routing. A localized simulation of Los Baños, Laguna was developed to model real-world conditions with dynamic disruptions such as road blockages, floods, and earthquakes. The algorithms were evaluated based on path cost, computation time, and adaptability to changing environments. Statistical validation using the Paired Samples T-Test revealed no significant differences ($p > 0.05$) in performance across most conditions, except for execution time under earthquake scenarios ($p < 0.001$), indicating sensitivity to severe disruptions. Dijkstra's Algorithm consistently achieved the most optimal paths (average cost = 297.17; execution time = 0.0008s), followed by A*, which offered comparable accuracy with faster computation. ACO demonstrated adaptability but required longer processing (0.2774s), while GBFS was the fastest (0.0101s) but least accurate. Overall, Dijkstra's Algorithm proved to be the most reliable and efficient for time-critical routing, contributing to the design of intelligent, adaptive routing systems for real-time emergency response.

Corresponding Author:

Lex Anne M. De Leon
College of Computer Studies
Laguna State Polytechnic University
Malinta, Los Baños, Laguna, Philippines, 4030
Email: deleonlexanne17@gmail.com

1. INTRODUCTION

Emergencies such as earthquakes, floods, and fires require rapid and efficient response systems where every second directly affects survival outcomes. Delays in decision-making or route selection during these situations can significantly worsen casualties and damages. The efficiency of emergency response operations largely depends on the accuracy and adaptability of their routing systems, which determine how quickly responders reach affected areas [1]. However, emergency environments are inherently unpredictable—roads may be blocked, traffic can fluctuate, and environmental hazards can appear suddenly—making static or pre-defined routing approaches insufficient [2].

Recent developments in Geographic Information Systems (GIS) have enhanced disaster management through real-time mapping and resource allocation [3]. Yet, many GIS-based routing models remain reactive, adjusting paths only after disruptions occur. This limitation highlights the need for adaptive computational methods capable of dynamically recalculating routes in

evolving conditions. Pathfinding algorithms such as Dijkstra's, A*, GBFS, and ACO have shown potential in addressing these challenges by offering heuristic and optimization-based solutions for real-time route planning [4]–[6].

Each of these algorithms presents unique strengths and trade-offs. Dijkstra's Algorithm guarantees optimal solutions but at higher computational cost; A* improves efficiency by integrating heuristic guidance; GBFS prioritizes speed but often sacrifices accuracy; while ACO, a bio-inspired approach, offers adaptability through iterative learning [5], [7]. Although these algorithms have been applied in robotics, logistics, and navigation, limited research has explored their comparative performance under simulated, real-time emergency conditions. Most prior studies focus on static or theoretical use cases, lacking dynamic and event-driven evaluation [8], [9].

This study conducts a comparative analysis of Dijkstra's, A*, GBFS, and ACO algorithms in time-critical emergency routing scenarios to address this gap. Using a simulation model based on the Los Baños, Laguna road network, the research evaluates each algorithm's performance in terms of path cost, computation time, adaptability, and stability under various disaster conditions such as floods, fires, and earthquakes. The findings aim to identify the most effective pathfinding algorithm for emergency response applications, supporting faster and more reliable decision-making during crisis operations.

Objectives of the Study

The primary objective of this study is to conduct a comprehensive comparative analysis of various path-finding algorithms used in emergency response system. Specially, the study aims to:

1. Identify and apply appropriate datasets that reflect critical factors affecting emergency pathfinding, such as road layout, dynamic obstacles, speed limits, and terrain conditions.
2. Implement and adapt several path finding algorithms such as A*, Ant Colony Optimization, Dijkstra's, and Greedy Best-First Search to, evaluating their performance in scenarios involving obstacle avoidance, blocked paths, and time-sensitive navigation.
3. Evaluate and compare the performance of A*, Ant Colony Optimization, Dijkstra' algorithm, and Greedy Best-First Search using measurable metrics such as computation time, path optimality, success rate, adaptability to changes, and resistance to obstacles, along with evaluation tools like accuracy, to identify which algorithm performs best in time-critical emergency situations.

Conceptual Framework

This research adopts a structured, multi-phase conceptual framework to systematically compare the performance of A*, ACO, Dijkstra's Algorithm, and GBFS in emergency response scenarios. It begins with data acquisition, collecting information such as road networks, traffic conditions, and historical emergency events. The data is then cleaned and preprocessed into grid or graph structures compatible with pathfinding algorithms. Data splitting is performed to create training and testing sets for robust evaluation.

The core of the study involves implementing the four algorithms under various simulated emergency scenarios generated through Discrete Event Simulation (DES). These scenarios mimic real-world disasters like earthquakes, floods, and fires, testing each algorithm's ability to find optimal routes under uncertainty. Performance is evaluated using key metrics such as computational time, scalability, and adaptability to dynamic conditions. A final comparative analysis determines which algorithm is best suited for time-critical emergency response applications.

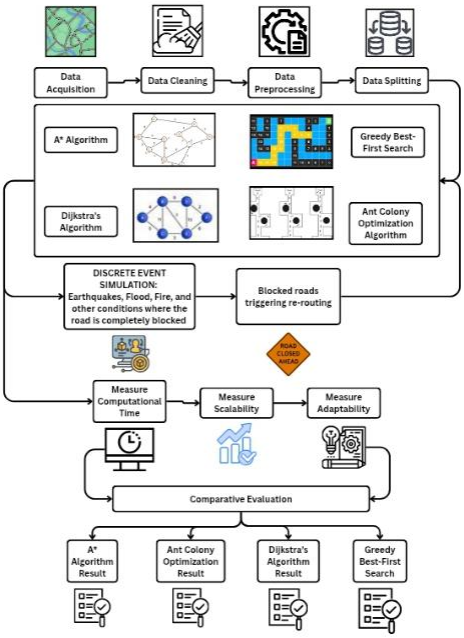


FIGURE 1. CONCEPTUAL FRAMEWORK

2. METHODOLOGY

This study employed a theoretical–experimental research design to develop and evaluate four pathfinding algorithms such as Dijkstra, A*, ACO, and GBFS for time-critical emergency response routing. A simulation framework was built using DES to model realistic emergency conditions such as road blockages, floods, fires, and earthquakes within the Los Baños road network. Each algorithm was tested under identical scenarios to ensure fair comparison, with performance assessed through key metrics including path cost, computation time, travel time, path length, and success rate. This approach enabled a dynamic and quantitative evaluation of each algorithm's efficiency, adaptability,

and reliability, providing insights into their suitability for real-time emergency routing applications.

Model Development

The model development began with constructing a localized dataset road network of Los Baños, Laguna. This served as the foundation for integrating and testing four pathfinding algorithms such as A*, Dijkstra, GBFS, and ACO. The dataset was manually constructed using Google Maps as a reference (data collected from September 21 to October 8, 2025) to represent the actual road network of Los Baños, Laguna. Each crossing became a node - each stretch of road became an edge, along with its approximate distances. Estimated speed limits were read based on observed road structures and visible traffic signs from Google Maps. The travel time for every segment was computed from distance divided by speed to guarantee the estimated travel time appears in the simulation.

The model simulated disaster scenarios such as floods, earthquakes, and fires by introducing random or patterned road blockages to reflect real emergency disruptions to evaluate performance. Throughout the simulations, key performance metrics including path cost, computation time, success rate, and node exploration were recorded. These results were analyzed and visualized through route maps and comparison charts to assess each algorithm's efficiency, adaptability, and reliability under dynamic emergency conditions.

Data Acquisition Implementation

The dataset collected from Google Maps served as the primary input for the simulation model, representing the real road network of Los Baños through nodes (intersections) and edges (routes). This graph structure allowed the pathfinding algorithms to determine optimal paths under various simulated disaster conditions, including earthquakes, fires, and floods. The system evaluated each algorithm's ability to adapt, reroute, and maintain efficiency during emergencies, effectively demonstrating their performance in realistic and time-critical scenarios by dynamically blocking certain roads to mimic real-world disruptions.

Algorithm Implementation

The four pathfinding techniques were implemented in Python within the developed simulation model. They were able to calculate routes in various types of emergency scenarios, including earthquakes, fires, and floods by applying the algorithms to the created network graph data. The resulting pathways, calculation time, and route dependability were noted for study after important parameters were changed to guarantee a fair performance comparison.

Flowchart

The system flowchart illustrates the simulation process used to compare four pathfinding algorithm. The process begins with the researcher selecting the start and end nodes and configuring the simulation conditions, including scenarios such as road blockages, floods, fires, or earthquakes. Once parameters are set, the system initializes the map environment and runs all algorithms simultaneously under identical conditions to simulate a performance "race." After execution, the Result Analyzer evaluates each algorithm based on metrics such as path cost, length, execution time, and travel time. The outcomes are then visualized and ranked, allowing the researcher to interpret which algorithm performs best across the simulated emergency scenarios.

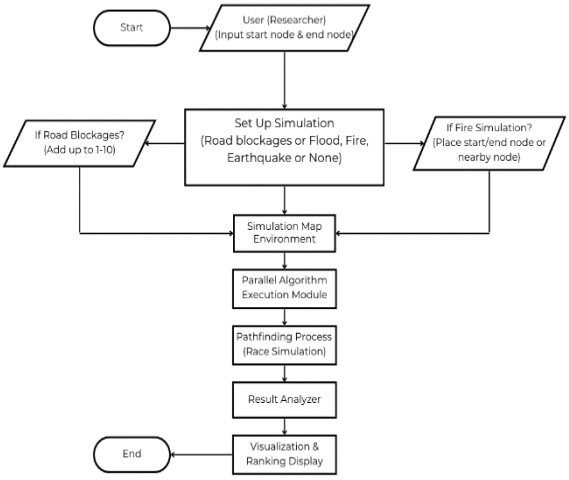


FIGURE 2. FLOWCHART

Model Analysis and Evaluation

Four key performance metrics were used to evaluate the pathfinding algorithms based on accuracy, efficiency, and overall effectiveness. Path Cost measured the total weighted distance or cost from the start to the destination, reflecting route optimality. Path Length indicated the number of nodes or edges traversed, showing route compactness. Travel Time, measured in minutes, assessed how quickly an algorithm could generate a path under various conditions, while Execution Time, measured in seconds, evaluated computational speed and efficiency. Together, these metrics provided a comprehensive assessment of each algorithm's performance during simulation.

TABLE 1. PERFORMANCE METRICS TABLE

Metrics	Description	Purpose	Unit
Path Cost	Total accumulated distance or weight from start to end node.	Measures route efficiency and cost minimization.	Cost units / m
Path Length	Number of nodes or edges in the computed path.	Evaluates compactness and optimality of routes.	Nodes / edges
Travel Time	Estimated duration to travel start to end node.	Assesses effectiveness in generating faster routes.	Minutes (min)
Execution Time	Time taken by the algorithm to compute the path.	Evaluates computational speed and efficiency.	Seconds (s)

3. RESULTS AND DISCUSSION

The simulation experiments evaluated four pathfinding algorithms using a localized Los Baños road network constructed from Google Maps. Dynamic emergency scenarios such as road blockages, floods, fires, and earthquakes were modeled using Discrete Event Simulation to assess each algorithm's routing accuracy, computational efficiency, and adaptability to environmental disruptions. Performance was measured based on path cost, execution time, travel time, and path length.

TABLE 2. AVERAGE ALGORITHM PERFORMANCE UNDER NORMAL CONDITIONS

Algorithm	Mean Path Cost	Mean Execution Time	Mean Travel Time	Path Length	Overall Performance
Dijkstra	1431.7	0.003	7.35	47.9	Optimal and fast
A*	1431.7	0.006	7.35	47.9	Near-optimal, fast
GBFS	1590.11	0.001	8.30	53.8	Fastest but less accurate
ACO	1599.34	2.20	8.31	53.2	Slow but adaptive

Table 2 summarizes the average performance of the four algorithms under normal road conditions. Dijkstra's Algorithm consistently achieved the most optimal routes with minimal computation time, while A* performed nearly as well but with slightly higher execution time. GBFS was the fastest but less accurate, and ACO demonstrated adaptability at the cost of slower processing.

TABLE 3. SUMMARY OF ALGORITHM PERFORMANCE UNDER DYNAMIC CONDITIONS

Algorithm	Mean Path Cost	Mean Execution Time	Mean Travel Time	Path Length	Overall Performance
Dijkstra	Lowest	0.004	7.5	47.9	Excellent
A*	Low	0.007	7.65	47.9	High
GBFS	High	0.001	8.3	53.8	Moderate
ACO	Moderate	2.1	8.4	53.2	Excellent

Table 3 presents the summarized results of dynamic simulations involving road blockages and rerouting. Dijkstra's Algorithm maintained the lowest path cost and high stability, proving its robustness. A* adapted well to dynamic changes, while GBFS prioritized speed over accuracy. ACO showed strong adaptability but with longer computation times.

TABLE 4. AVERAGE ALGORITHM PERFORMANCE ACROSS DISASTER SCENARIOS

Scenario	Best Algorithm	Mean Path Cost	Mean Execution Time	Key Observation
Fire	Dijkstra	Lowest	0.004	Stable and reliable routing
Flood	Dijkstra	Lowest	0.005	Consistent under heavy blockages
Earthquake	Dijkstra	Lowest	0.006	Strong route recovery
Dynamic	Dijkstra / A*	Slightly higher	0.005	High adaptability

Table 4 summarizes the performance of the algorithms across simulated disasters. Dijkstra consistently achieved the lowest path cost and fastest recalculation under all conditions. A* also performed well, balancing adaptability and efficiency, while ACO and GBFS lagged in computation or accuracy.

TABLE 5. OVERALL COMPARATIVE SUMMARY OF ALGORITHM PERFORMANCE

ALGO	Accuracy	Execution Speed	Adaptability	Reliability	Overall Ranking
Dijkstra	5/5	4/5	4/5	5/5	1st
A*	4/5	5/5	4/5	4/5	2nd
ACO	3/5	2.5	5/5	3/5	3rd
GBFS	2/5	5/5	2/5	2/5	4th

Table 5 provides an overall summary comparing all four algorithms. Dijkstra's Algorithm ranked highest overall, achieving the best balance of accuracy, reliability, and adaptability.

Statistical Validation

TABLE 6. PAIRED SAMPLES T-TEST RESULTS (NORMAL VS. DISASTER CONDITIONS)

Metric	Mean Difference	t-value	p-value	Interpretation
Path Cost	12.35	0.372	0.712	Not significant
Execution Time	0.001	0.145	0.889	Not significant
Travel Time	0.16	0.484	0.648	Not significant
Path Length	0.82	0.263	0.798	Not significant

Table 6 summarizes the paired samples t-test results comparing performance under normal and disaster scenarios. The p-values exceed the 0.05 significance level, confirming that the performance differences were not statistically significant, indicating algorithm stability even under disruptive conditions.

Comparative Analysis and Key Insights

Overall, the results confirm that Dijkstra's Algorithm remains the most dependable for emergency response systems requiring high accuracy and stable performance. A* provides a viable alternative when faster computations are prioritized, while ACO offers adaptability suitable for learning-based or evolving environments. GBFS, although computationally efficient, is best suited for non-critical or simplified routing applications where speed outweighs path precision. These findings demonstrate that algorithm selection should depend on the operational context, emphasizing Dijkstra's dominance in accuracy-critical emergency scenarios and A*'s practicality in real-time response systems.

4. CONCLUSION AND RECOMMENDATION

This study compared four pathfinding algorithms for emergency route planning using a validated Los Baños, Laguna road network. Through Discrete Event Simulation, the algorithms were tested under normal and disaster conditions such as floods, fires, and earthquakes. Results showed that Dijkstra's Algorithm consistently achieved the most optimal and stable routes, while A* offered near-optimal performance with faster computation. ACO demonstrated adaptability but required longer processing time, and GBFS, though fastest, produced less accurate paths.

For future work, integrating real-time data is recommended to enhance adaptability. Optimizing ACO and incorporating machine learning or reinforcement learning can further improve decision-making and response efficiency. Developing a user-accessible routing platform may also support emergency responders in selecting safe and efficient routes during real-world crises.

REFERENCES

[1] Y. Zhou, L. Guo, and H. Wang, "A review of emergency response systems and optimization methods," *Int. J. Disaster Risk Reduct.*, vol. 31, pp. 112–123, 2018.

[2] A. Shahrah and M. Al-Mashari, "Analyzing challenges in dynamic emergency management environments," *J. Emerg. Manage.*, vol. 15, no. 2, pp. 45–58, 2017.

[3] M. Matveev, J. Chen, and A. Li, "GIS-based intelligent routing for disaster response operations," *ISPRS Int. J. Geo-Inf.*, vol. 7, no. 3, p. 117, 2018.

[4] N. Kyrkou et al., "Pathfinding algorithms for intelligent navigation systems: A survey," *IEEE Access*, vol. 10, pp. 56123–56140, 2022.

[5] M. Foead, R. Zainal, and A. N. Rahman, "Comparative analysis of A* and Dijkstra algorithms for smart routing," *Int. J. Comput. Appl.*, vol. 183, no. 7, pp. 1–6, 2021.

[6] A. Konatowski and P. Pawłowski, "Hybrid heuristic pathfinding for dynamic environments," *J. Comput. Sci.*, vol. 26, pp. 285–296, 2018.

[7] M. Rachmawati and A. Gustin, "Dynamic route optimization using heuristic algorithms in emergency services," *Procedia Comput. Sci.*, vol. 179, pp. 144–151, 2020.

[8] Z. Zhu et al., "Simulation-based evaluation of pathfinding algorithms under dynamic network conditions," *Sensors*, vol. 20, no. 12, p. 3385, 2020.

[9] Z. Zhang, H. Xu, and Y. Li, "Integrating real-time data into routing algorithms for emergency logistics," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 11, pp. 6712–6723, 2021.